

Genral Information	ID	1
	Title	The Impact of Cryptographic Algorithm Performance on Smart Contracts in WebAssembly
	Link (DOI)	https://doi.org/10.1007/978-981-95-3480-7_7
	Year	2026
	Country	China
	Authors	Yuesin Kang, Jing Zeng, Qing Xia, Zhilin Wu, and Yucheng Li
	Venue it was published (Journal/Conference)	Springer Nature Singapore
	Reviewers (Reviewer 1/Reviewer 2)	Reviewer 1
Introduction	RQ0: In which application domains and use cases is WebAssembly (Wasm) currently used?	Smart contracts in blockchains (e.g., Ethereum 2.0/Ewasm), Decentralised Finance (DeFi) for liquidity pools, privacy protection via zero-knowledge proofs, and account verification. It also notes general use in serverless computing, embedded systems, edge computing, and IoT.
Adversarial models and attacks	RQ1: What types of attacks have been discovered so far for WASM?	Not Defined
	RQ2: What adversarial models have been used in this paper? (Honest/Semi-honest/malicious)	Not Defined
	RQ3: Which attacks are considered the most severe, and why?	Not Defined
Component vulnerability	RQ4: Which component of WASM may have vulnerabilities that can be exploited to perform a cyber attack?	Standalone WebAssembly runtimes (like Wasmer, Wasmtime, WAMR, and WAVM) which lack the extensive optimization and maturity of browser engines.
	RQ5: In each Wasm application domain or use case, which component of WASM may have vulnerabilities that can be exploited to perform a cyber attack?	In smart contracts and blockchain, the cryptographic algorithms within runtimes are highlighted as performance-sensitive areas that can lead to system-wide throughput issues and even render the application unavailable in this context.
RQ6: What types of attacks have been investigated for each Wasm use case?	RQ6-1: What types of attacks have been investigated for IoT in WASM?	Not Defined
	RQ6-2: What types of attacks have been investigated for Edge computing and cloud in WASM?	Not Defined
	RQ6-3: What types of attacks have been investigated for industrial control systems in WASM?	Not Defined
	RQ6-4: What security flaws and gaps exist in WASM for smart contracts?	Investigated performance gaps in cryptographic hashes (sha3, sha256) and signatures (ec_sign, ec_verify), noting they are significantly slower than native code.
	RQ6-5: Any other use-case?	Not Defined
Mitigation Tecniques	RQ7: What method is used to mitigate attack or detect vulnerabilities in this paper?	Performance characterisation and benchmarking (startup time, execution time, memory overhead) across different runtimes.
	RQ8: What mitigation techniques have been proposed or evaluated for each Wasm use case?	Not Defined
Research Gaps Domain and Use-case specific	RQ9: What research gaps remain in each Wasm application domain or use case?	Lack of optimization in runtime.
Detection Techniques	RQ10: What detection techniques have been proposed or evaluated for WASM?	Automated testing using shell scripts to measure performance metrics
RQ11: What mitigation techniques have been proposed or evaluated for each Wasm component?	RQ11-1: What mitigation techniques have been proposed or evaluated for binaries?	Use of dynamic linking to address startup time issues related to large binary sizes
	RQ11-2: What mitigation techniques have been proposed or evaluated for memory safety issues?	Not Defined
	RQ11-3: What types of attacks have been investigated for WASM compilers?	Not Defined
Future Work/Research gap	RQ12: Is there any future work suggested in this paper? Or the research gap mentioned?	Aims to promote the research and development of encryption algorithms and smart contracts in the WebAssembly ecosystem.
Framework/Tool	RQ13: Is there any security framework or tools for WASM brought in this paper?	Mentions OpenSSL toolkit and OpenZeppelin's smart contracts as test cases.
Survey or First-hand research	RQ14: Is this paper a first-hand research or a review (survey) paper?	first-hand research
Status	Finalized or Work in progress	Finalized

2	3	4	5
A Comprehensive Study of WebAssembly Runtime Bugs	WASMSEPA: Effectively Protecting WebAssembly Through Privilege Separation	WasmChecker: Effectively Detecting WebAssembly Bugs via Static Program Analysis	WATZ: A Trusted WebAssembly Runtime Environment with Remote Attestation for TrustZone
DOI: 10.1109/SANER56733.2023.00041	DOI: 10.1109/ORS65678.2025.00074	DOI: 10.1109/ICEMCE64157.2024.10861926	DOI: 10.1109/ICDCS54860.2022.00116
2023	2025	2024	2022
China	China	China	Switzerland
Yue Wanga, Zhide Zhoua, Zhilei Rena, Dong Liua, He Jiang	Zhuochen Jiang and Baojian Hua	Junjie Zhuang , Hao Jiang, Baojian Hua	James Menetrey , Marcelo Pasin , Pascal Felber , Valerio Schiavoni
IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)	International Conference on Software Quality, Reliability and Security (QRS)	International Conference on Electrical, Mechanical and Computer Engineering (ICEMCE)	International Conference on Distributed Computing Systems (ICDCS)
Reviewer 1	Reviewer 2	Reviewer 2	Reviewer 2
Web browsers (V8, SpiderMonkey), blockchain platforms, cryptocurrency, edge computing, IoT, and wearables.	WebAssembly (Wasm) is used in edge computing, smart contracts, cloud computing, server-side computing, and the Web	Blockchain (smart contracts), edge computing, cloud computing, IoT, gaming, media processing, and databases	Wasm insidea TEE, oT, edge computing, cloud computing, smartphones, and automotive applications
While focusing on bugs, it notes these can lead to "security vulnerabilities". Identified root causes include incorrect algorithm implementation, memory handling errors, and API misuse, Type Problem, Incorrect Condition Logic	Double-free bugs, buffer overflows (stack and heap), heap metadata corruption, and vulnerabilities originating from unsafe foreign function interface (FFI) calls	Integer overflows, memory safety issues (double-free, use-after-free, buffer overflows, memory leaks), and cryptojacking	The attacks are not directly on Wasm. But using Wasm + TEE prevents vertical privilege escalations and lateral attacks
Not explicitly named, but the study focuses on 867 real-world bugs, suggesting an "honest but buggy" implementation model.	The paper assumes Wasm VMs and RPC mechanisms are secure and trustworthy, while source code and Wasm binaries are untrusted and potentially unreliable	Source code and compilation toolchains are untrusted, while the Wasm runtime is assumed to be safe	Assumes attacker has physical access to devices but cannot subvert hardware protections. Secure world (monitor, bootloader, TEE OS) is trusted; Normal world (Rich OS) is untrusted
The paper identifies "Crash" as the most prevalent symptom (56.86%), which significantly degrades user experience and robustness. Root causes like "Memory" (17.07% of bugs) are critical because they involve illegal accesses or memory leaks	Memory vulnerabilities (like double-free) are severe because they propagate from unsafe source languages into Wasm binaries, potentially compromising the entire system's integrity	Integer overflows and memory safety issues are most significant because they lead to undefined behavior, system takeover, or theft of sensitive information	Not on Wasm, but on TEE (Trustzone). Attacks compromising the authenticity of software handling sensitive tasks/data (e.g., machine learning at the edge) are severe
Vulnerabilities exist in the Wasm Runtime infrastructure, specifically its AOT/JIT compilers, interpreters, APIs (C-API, Rust-API), and the WebAssembly System Interface (WASI)	Linear memory (susceptible to overwrites due to lack of protection for sensitive data), Wasm binaries (inheriting source flaws), and Wasm compilers	Linear memory design and Wasm binaries compiled from insecure languages	Not mentioned
"Incorrect Algorithm Implementation" in the code generation phase of runtimes like Wasmtime is highlighted.	Not defined	Critical applications like cryptographic functions or access control mechanisms are highly vulnerable to linear memory corruption	Not mentioned
Not defined	Not defined	Not defined	The paper runs Wasm inside a TEE. This prevents Malicious firmware updates and software flaws for IoT devices
Not defined	Not defined	Not defined	Validity of machine learning algorithm results (again this by considering running Wasm in a TEE)
Not defined	Not defined	Not defined	Trusted Execution Environment (TEE) using Arm TrustZone and a remote attestation protocol
Not defined	Not defined	Not defined	Not defined
Not defined	Not defined	Mentioning the Cryptojacking attacks	Wasm for creating sandboxed environment to execute codes in a TEE
The paper performs an empirical analysis of 867 real bugs (root causes, symptoms, fixing time) to characterize their root causes and symptoms to aid in future detection	Privilege separation by isolating Wasm binaries from unsafe libraries in separate processes using Remote Procedure Calls (RPC)	Static program analysis based on abstract interpretation using numerical (interval) and symbolic abstract domains	Isolation via the Wasm sandbox inside TrustZone for IoT and automotive machine learning enclaves
Not defined	Privilege separation via RPC is proposed to protect linear memory across general Wasm execution environments	Not defined	Isolation via the Wasm sandbox inside TrustZone for IoT and automotive machine learning enclaves
There is a need for automated techniques to detect and locate bugs caused by incorrect algorithm implementation and memory issues	Lack of architectural support for privilege separation in current VMs and challenges handling diverse programming languages and toolchains	Not defined	The paper mentions some gaps for TEE but not related to Wasm
The paper discusses fuzzing (e.g., Wasmfuzzer) and differential testing as promising directions	Identification of potentially unsafe function calls during binary generation and runtime sanity checking	Static program analysis	Not defined
Not defined	Binaries: Instrumented with RPC wrappers to replace direct procedural calls Memory Safety: Isolation of unsafe APIs to prevent direct access to linear memory Compilers: Not defined.	program analysis	Measured boot/secure boot to verify Wasm bytecode
Not defined		Tracking symbolic abstract values (e.g., malloc/free counts) to detect deallocation issues	Software fault isolation and control-flow integrity within the Wasm sandbox
Not defined		Not defined	Not specifically investigated
Lack of automated techniques to detect and locate bugs caused by incorrect algorithm implementations	Fine-grained unsafe operation collection, analyzing control-flow graphs (CFGs) via static analysis, and supporting concurrency semantics	supporting more languages (Rust, Go), and detecting vulnerabilities like indirect call redirection.	Not related to Wasm
Wasmfuzzer	WASMSEPA (software prototype)	WasmChecker	WATZ runtime
First-hand research (empirical analysis)	First-hand research	First-hand research	First-hand
Finalized	Finalized	Finalized	Finalized

6	7	8	9
Bringing the Web up to Speed with WebAssembly	MUTATE: Fast and effective binary diversification for WebAssembly	Drive-By Key-Extraction Cache Attacks from Portable Code	Iris-MSWasm: Elucidating and Mechanising the Security Invariants of Memory-Safe WebAssembly
http://dx.doi.org/10.1145/3062341.3062363	https://doi.org/10.1016/j.cose.2024.103731	https://doi.org/10.1007/978-3-319-93387-0_5	https://doi.org/10.1145/3689722
2017	2024	2018	2024
USA, Germany	Sweden, USA	USA, Israel, Australia	Denmark, Germany
Andreas Haas Andreas Rosberg Derek L. Schuff Ben L. Titzer, Michael Holman, Dan Gohman Luke Wagner Alon Zakai, JF Bastien	Javier Cabrera-Arteaga , Nicholas Fitzgerald, Martin Monperrus, Benoit Baudry	Daniel Genkin, Lev Pachmanov, Eran Tromer, and Yuval Yarom	Maxime Legoupi, June Rousseau, Aina Linn Georges, Jean Pichon-Pharabod, Lars Birkedal
ACM, Barcelona, Spain	ELSEVIER Computers & Security	Springer Nature	Proc. ACM Program. Lang. 8, OOPSLA2, Article 282
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Demanding web applications like interactive 3D visualization, audio/video software, and games. Designed to be a portable compilation target for languages like C and C++.	Software protection, moving target defence, optimization, malware evasion, and fuzzing compilers	Cryptographic libraries (ElGamal, ECDH, RSA), secure email browser extensions (Google End-to-End, OpenPGP.js), and locked-down platforms like Chromebooks.	Browser execution, cloud computing (Fastly's Compute@Edge, Ferra's Spin), edge computing, and microservices
Not Defined	Specifically investigates timing side-channel attacks like Spectre (BTB, PHT, and RSBit/spec types). Also mentions ROP (Return-Oriented Programming) and function-reuse attacks.	Microarchitectural cache side-channel attacks (specifically Prime+Probe), and drive-by attacks launched via malicious web advertisements.	Classical memory safety attacks (buffer overflows, use-after-frees) targeting programs not written for module isolation, and attacks from untrusted/adversarial library modules.
Assumes code originates from "untrusted sources" (malicious web content)	Malicious actors attempting to identify binaries via signatures or exploit speculative execution to exfiltrate data.	A malicious actor executes non-privileged code (Wasm or PNaCl) in a sandboxed browser tab to monitor cache contention from other tabs or processes on the same machine.	Malicious (unknown, potentially adversarial code)
Not Defined	Spectre attacks are treated as a significant threat due to their ability to bypass Wasm's control flow integrity through speculative execution.	Cache attacks capable of extracting secret cryptographic keys (e.g., 252-bit ECDH keys) in as little as 3 minutes.	Attacks that invalidate encapsulation guarantees due to mistakes in brittle security invariants.
Since the client language VM is trusted, a bug in that layer could allow an incorrect program to read or write memory arbitrarily or exhibit other undefined behavior.	The binary structure (which can be fingerprinted) and execution traces (which can leak info via cache).	The CPU's shared Last Level Cache (LLC) and microarchitectural resources like branch predictors.	Linear memory (due to all-or-nothing sharing), the module system (coarse-grained encapsulation), and potential incorrect compiler optimisations.
Untrusted code execution	Side-channels in browser environments that leak data from memory or ports.	Browser-based cryptographic implementations in JavaScript that introduce key-dependent control flow or memory accesses.	In edge computing, sharing a linear memory with adversarial libraries exposes all data within that memory.
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Investigates Spectre attacks on binaries used in these environments.	Not Defined	Interference from third-party adversarial library modules.
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Web: Investigates Spectre attacks on binaries used in these environments	Web/Chromebook: Investigates full key extraction from ElGamal, ECDH, and RSA implementations running in a browser environment.	combination of known code with unknown code compiled to MSWasm.
Formal semantics for execution and validation, a robust type system, and structured control flow to prevent arbitrary jumps.	using an e-graph data structure and 135 rewriting rules to create functionally identical but behaviorally different variants.	Constructing precise eviction sets from a portable environment to recover memory-to-cache mappings without huge pages.	Iris-MSWasm, a foundational reasoning framework using separation logic and logical relations to formally validate robust capability safety.
Sandboxing using linear memory (disjoint from code space) and dynamic bounds checks.	Loop unrolling, peephole optimizations, "phantom" instruction injection, and dead code removal to disrupt attack patterns.	Suggests delegating cryptographic operations to native implementations like the WebCrypto API.	Edge computing: fine-grained memory sharing via "handles" (capabilities) that guarantee spatial and temporal safety.
WebAssembly does not (yet) have threads, and therefore no non-determinism arising from concurrent memory access. Adding threads and a memory model is the subject of ongoing work beyond the scope of this paper.	Automation of rewriting rule creation and hardening against ret2spec/PHT attacks which are more complex than BTB.	Implementing side-channel resistant techniques in JIT-compiled environments is identified as an unexplored area.	Proving integrity properties of local state when using unknown code.
A validation algorithm checkable in a single linear pass that ensures type safety and prevents undefined behavior	Uses IntelPNaCl to collect and compare instruction and memory traces between original programs and variants.	Monitoring timing variability in memory access via Wasm/PNaCl to detect cache hits vs. misses.	Not Defined
Compact representation designed for efficient validation	Diversification of sections (type, function, custom) to prevent static analysis.	JIT compilation of high-level code can inadvertently introduce key-dependent control flow even in "constant-time" algorithms.	The MSWasm extension with segment memory and unforgeable handles.
Software Fault Isolation (SFI) where a module cannot access memory outside its allocated linear array	Injecting artificial global values to create divergent memory traces.	Not Defined	Use of the MSWasmCert mechanisation and Iris-MSWasm logic to prove safety invariants.
Uses JIT/AOT compilation with parallelization to improve safety and performance	Fuzzing compilers by generating semantically equivalent variants to stress components like spill/reload.	JIT compilation of high-level code can inadvertently introduce key-dependent control flow even in "constant-time" algorithms.	Mention of production compilers requiring patches to support capabilities.
Suggested additions include zero-cost exceptions, threads, SIMD instructions, and integration with advanced garbage collectors	Suggested future work includes using rule mining to automate rule generation and multivariant execution to harden against ret2spec attacks.	Need for side-channel mitigation research in the context of modern, JIT-compiled environments.	Proving confidentiality via binary logical relations, defining semantics that capture confidentiality, and solving issues with optimising capability-safety-preserving compilers.
Introduces a reference interpreter implemented in OCaml for testing production implementations	Introduces WASM-MUTATE, an open-source diversification engine.	Evaluates Elliptic, Google's End-to-End, and OpenPGP.js libraries	MSWasmCert (mechanised language definition) and Iris-MSWasm (mechanised program logic).
First-hand research	First-hand research	First-hand research	first-hand research
Finalized	Finalized	Finalized	Finalized

10	11	12	13
positional Information Flow Analysis for WebAssembly Pro	MSWasm: Soundly Enforcing Memory-Safe Execution of Unsafe Code	Developers Struggle with Authentication in Blazor WebAssembly	Angers, Assemble! Survey of WebAssembly Security Solutions
10.1109/SCAM51674.2020.00007	https://doi.org/10.1145/3571208	DOI: 10.1109/ICSM55016.2022.00045	10.1109/CLOUD55607.2022.00077
2020	2023	2022	2022
Belgium	USA, UK, Italy, Netherlands	Switzerland, Belgium, Germany	Republic of Korea
Quentin Stievenart, Coen De Roover	ALEXANDRA E. MICHAEL, ANITHA GOLLAMUDI, JAY BOSAMIYA, EVAN JOHNSON, AIDAN DENLINGER, CRAIG DISSELKOEN, CONRAD WATT, BRYAN PARNO, MARCO PATRIGNANI, MARCO VASSENÀ, DEIAN STEFAN	Pascal Marc André, Quentin Stievenart, Mohammad Ghafari	Minseo Kim, Hyerean Jang, Youngjoo Shin
IEEE 20th International Working Conference on Source Code Analysis and Manipulation (SCAM)	Proc. ACM Program. Lang. 7, POPL, Article 15	IEEE International Conference on Software Maintenance and Evolution (ICSM)	IEEE 15th International Conference on Cloud Computing (CLOUD)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Web browsers, cross-platform desktop applications, server applications, IoT backends, and embedded systems.	Web, edge clouds, IoT platforms, serverless computing, and embedded platforms (eWASM)	Cross-platform applications and Web UIs (specifically Blazor WebAssembly)	Cloud-native development, container alternatives, web applications, desktop, mobile, IoT, and embedded systems.
Cryptojacking, traditional attacks like buffer overflows, and remote code execution.	Buffer overflows, use-after-free, double-free vulnerabilities, and pointer integrity violations.	Not Defined. (The paper focuses on developer struggles/intentions regarding security implementation).	Cryptojacking, memory bugs (buffer overflows), type-confusion, side-channel attacks (timing/cache), and Spectre.
Malicious (exposure to malicious code/instructions).	Malicious (Wasm-level attacker linking arbitrary malicious code)	Not Defined.	Malicious (cybercriminals/attackers exploiting Wasm for resource stealing or data theft).
Cryptojacking is noted as the most common application of Wasm in a 2019 study, causing resource stealing.	Exploits that allow attackers to confuse host code into performing unsafe actions, such as cross-site scripting (XSS) via a sandboxed library.	Not Defined.	Cryptojacking, as it causes massive financial losses to cloud users and is a primary driver for malicious Wasm binaries.
Linear memory (prone to buffer overflows), and indirect function calls (allowing RCE).	Linear memory (lacks fine-grained protection), Pointers (compiled to forgeable 32-bit integers), and Compilers (industrial compilers lack memory-safety checks).	Not Defined. (The focus is on the Blazor framework and user authentication implementation)	Compilers (lacking security mechanisms), Linear memory (susceptible to memory corruption), and Function Table (complex type validation hurdles).
Not Defined.	In edge clouds, memory unsafety can allow one client to interfere with another in a single process.	Not Defined.	Cloud environments: lacks robust vulnerability analysis to protect benign applications from malicious exploitation.
Not Defined.	Not Defined.	Not Defined.	Not Defined.
Not Defined.	Not Defined.	Not Defined.	Resource stealing and financial losses in cloud systems
Not Defined.	Not Defined.	Not Defined.	Not Defined.
Not Defined.	Not Defined.	Not Defined.	Not Defined.
Web browser vulnerabilities (cryptojacking).	Web application attacks like XSS resulting from library buffer overflows.	Web UI: Authentication hassles, third-party social provider integration, and user role assignment.	Web browsers (DOM access risks via JavaScript APIs)
Automated compositional static program analysis focused on information flow to identify potential security issues before execution.	MSWasm extension (segment memory and unforgeable handles) and a formal C-to-MSWasm compiler to soundly enforce memory safety.	Web UI: Authentication hassles, third-party social provider integration, and user role assignment.	Systematic survey classifying state-of-the-art security techniques into static, dynamic, and hybrid approaches.
Not Defined.	Edge Cloud: Using language-level abstractions (handles) to isolate client memory.	Proposing improved documentation, troubleshooting guides for certificates/deployment, and demo applications for role-based access control.	Cloud/Web: Systems like MINOS, RAPID, and Outguard for cryptojacking detection.
Lack of static analysis tools to protect runtimes from malicious instructions.	Lack of industrial compilers that insert required memory-safety checks for Wasm.	Lack of studies regarding the security concerns faced by developers adopting Wasm.	Security in cloud-native Wasm applications have not been extensively investigated.
Static information flow analysis that derives function summaries to approximate information flow and unwanted data leaks.	An abstract, language-independent color-based memory-safety monitor.	Not Defined.	Static (MINOS, CFG-based MineRay), Dynamic (Resource monitoring RAPID, instruction-frequency-based MineThrottle), and Hybrid (SEISMIC).
Use of static analysis to identify malicious binaries before execution.	Introduction of segment-based load/store opcodes in MSWasm bytecode.	Not Defined.	Binary instrumentation (Wasabi), code encryption (SELWasm)
Not Defined.	Unforgeable handles with integrity tags and the slice instruction for intra-object safety.	Not Defined.	MS-Wasm (hardware isolation) and VeriWasm (static software isolation verification).
Not Defined.	A formalized C-to-MSWasm compiler ensuring that well-typed modules are robustly memory safe.	Not Defined.	Compiler-based protections like Swivel (against microarchitectural attacks).
Refining memory modeling to include precise modelling of numerical values, and modeling traps/exceptional control flow.	Memory unsafety in C++ programs, type confusion vulnerabilities, and temporal safety for hardware-backed CHERI.	Investigating issues across different source languages/target platforms, and developing automated models to identify developer sentiments/themes.	Techniques to detect malware bypassing obfuscation, vulnerability detection for non-C/C++ languages (Rust, Python), and extending research beyond web applications to standalone cloud platforms.
A compositional static analysis framework (implemented in OCaml); MINWASM	MSWasm compilers (AOT rWasm-based, JIT GraalVM-based) and LLVM-to-MSWasm compiler.	Blazor WebAssembly framework.	Wasabi, TaintAssembly, Swivel, MS-Wasm, CT-Wasm, and Vivienne
first-hand research	first-hand research	First-hand research	Survey paper
Finalized	Finalized	Finalized	Finalized

14	15	16	17
SwFuzz: Structure-Sensitive WebAssembly Fuzzing	WAPPLER: Sound Reachability Analysis for WebAssembly	Towards Least-Privilege WebAssembly Applications	Security Risks of Porting C Programs to WebAssembly
10.1109/APSEC65559.2024.00045	10.1109/CSF61375.2024.00025	10.1145/3801119.3801125	10.1145/3477314.3507308
2024	2024	2026	2022
China	Austria and Denmark	Belgium	Belgium and New Zealand
Jiahui Wang, Ziyi Guo, Xinlei Ying, Peng Qian, and Yan Chen	Markus Scherer, Jeppe Fredsgaard Blaabjerg, Alexander Sjösten, Magdalena Solitro, and Matteo Maffei	Olav Blaak and Tom Van Cutsem	Quentin Stiévenart, Coen De Roover, and Mohammad Ghafari
Conference; APSEC (Asia-Pacific Software Engineering Conference)	Conference; CSF (IEEE Computer Security Foundations Symposium)	Conference; Programming Companion '26	Conference; SAC '22 (ACM/SIGAPP Symposium on Applied Computing)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
web browsers, server-side applications, blockchain platforms, decentralized applications (dApps), serverless computing, mobile applications, IoT devices, edge computing, and portable cloud applications	Web browsers, smart contracts, edge computing, plugin systems, isolation of untrusted components, lightweight containers, and IoT	Web browsers, standalone virtual machines, edge computing, distributed systems, and blockchain	Web browsers, IoT systems, cross-platform desktop applications, Node.js, and server-side runtimes.
Use-After-Free, Out-Of-Bound, Null-Pointer-Deref, Invalid-Memory-Access, Denial-of-Service, and heap-based-buffer-overflow	Integer overflows (e.g., i32.add overflow), out-of-bounds memory access, sensitive memory overwrites, and Cross-Site Scripting (XSS) attacks resulting from compromised data integrity.	Supply chain attacks, abuse of granted capabilities (privilege escalation), and the confused deputy problem stemming from ambient authority.	Buffer overflows, stack smashing, Use-After-Free, invalid pointer free operations, buffer underwrites, uncontrolled format strings, and remote code execution.
Malicious; the paper aims to uncover vulnerabilities that "malicious entities could exploit"	Malicious ("possibly malicious contexts")	Malicious (focusing on "compromised" components and malicious packages from third-party registries)	Malicious (attackers exploiting memory safety vulnerabilities to achieve remote code execution)
Use-After-Free in popular runtimes and vulnerabilities in critical systems like blockchains that handle sensitive transactions requiring high integrity	Vulnerabilities lacking mitigations like stack canaries and Address Space Layout Randomization (ASLR) are considered severe because Wasm lacks these native protections, making vulnerabilities like integer overflows and memory safety violations easier to exploit.	Supply chain attacks are cited as a rapidly rising and potentially disastrous threat because imported libraries can execute unrestricted code if their capabilities are not restricted.	Stack smashing, buffer underwrites, and invalid free operations are considered severe because they go undetected in Wasm due to the lack of hardware-level or compiler protections (like stack canaries) present in native environments.
WASM runtimes, specific modules related to initialization, error handling, and parsing, and the WASI (WebAssembly System Interface) module.	The linear memory, the rewriting-based semantics (evaluation contexts), and interactions with embedders (host functions).	The WebAssembly System Interface (WASI) and the Wasm component model's capability-based access control system.	The linear memory, the standard C library implementations (e.g., musl vs glibc), and function pointer tables (for indirect calls).
In the blockchain/critical systems domain, the complex and diverse WASM bytecode and runtimes are cited as containing potential attack surfaces. In web browsers, the integrated WASM engines (e.g., Chrome, Firefox) are listed.	In web browsers, the embedding context and imported host functions are vulnerable to malicious interactions. In smart contracts, the low-level nature of linear memory makes it difficult to soundly analyze and protect against vulnerabilities.	Not Defined	In Node.js/server-side applications, vulnerable Wasm code can be exploited to perform remote code execution through the host environment.
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Bytecode complexity and runtime vulnerabilities are investigated as threats to the high-level integrity required by blockchains.	The lack of sound and automated verification tools is noted as a gap for ensuring the high security required by this domain.	Not Defined	Not Defined
Web browsers are investigated for vulnerabilities that could allow malicious interpretation of modules within the sandboxed environment.	Web browsers were investigated for XSS-style attacks triggered by out-of-bounds writes to "trusted" data arrays.	Software supply chain attacks where malicious packages are distributed via central registries like Docker Hub or npm-like systems.	Server-side applications were investigated for vulnerabilities that lead to remote code execution when integrated with platforms like Node.js.
A structure-sensitive fuzzing framework (SwFuzz) that utilizes file structure mutation, instruction mutation, and Large Language Model (LLM)-assisted mutation to detect zero-day vulnerabilities in runtimes.	A sound and automated static analysis technique (WAPPLER) based on Horn-clause resolution and reachability analysis.	A transparent interposition framework for wrapping components with arbitrary logic and a Domain-Specific Language (DSL) for specifying fine-grained, declarative security policies.	Comparative analysis (cross-compilation) between x86 and Wasm to identify security-critical behavioral differences.
Not Defined	Not Defined (the paper focuses on verification)	Not Defined	Not Defined
Existing fuzzing solutions lack the specificity to detect structure nuances of WASM files and fail to accurately test module handling-related code (initialization, error handling), leaving security gaps.	A lack of sound and automated verification techniques at scale and the difficulty of capturing memory management soundly due to Wasm's low-level nature.	The lack of dynamic revocation for capabilities, the mixing of security and implementation logic in WASI, and the need for interposing logic for resource management and logging.	A mismatch between the security guarantees of native and Wasm applications, particularly the lack of compiler protections like Stack Smashing Protection (SSP) in Wasm.
SwFuzz, AFL, AFL++, Fuzzzz, and WasmFuzzer.	WAPPLER, WASP (concolic execution), Wassail (compositional analysis), Wasmali (static scanner), MinerRay, and MineSweeper.	Taint analysis is mentioned as a technique that could be combined with the proposed framework.	Manual inspection using radars2, the Juliet Test Suite (for benchmarking), and Wassail (static analysis library).
Structure recovery mechanisms to repair mutated files and ensure they remain executable candidate candidates for vulnerability detection.	Not Defined	Transparent component wrapping to inject security behavior without modifying the original binary	Code diversification to generate multiple program variants and mitigate attack vectors.
Not Defined	MSWasm (a memory-safe extension), vWasm, and rWasm (verified sandboxing compilers)	Not Defined	Extensions to Wasm to encode memory semantics and trusted runtimes using Intel SGX
Not Defined	Not Defined	Not Defined	Self-certifying compilers and hardening frameworks against Spectre attacks
extend SwFuzz to support more programming languages used in WebAssembly runtimes.	Future work includes implementing 32-bit value optimizations, supporting floating point numbers, analyzing multiple modules, and automatic reconstruction of attack traces.	future work involves evaluating the framework on large-scale real-world compositions and formalizing the interposition DSL using higher-order contracts.	future work includes investigating GCC/Emcripten, analyzing non-deterministic programs, and exploring other runtimes and architectures.
swfuzz	it introduces WAPPLER and discusses z3 (theorem prover) and the HORST framework.	it introduces a prototype interposition framework and discusses WASI-Virt, Shill, and BASTION	Wasmer, Swivel, Twine, and Wassail
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

18	19	20	21
Progressive Memory Safety for WebAssembly	WALTZZ: WebAssembly Runtime Fuzzing with Stack-Invariant Transformation	Static Semantics Reconstruction for Enhancing JavaScript WebAssembly Multilingual Malware Detection	Tagging WebAssembly for Numerical JavaScript Code Virtualization
10.1145/3337167.3337171	[Not Defined] (Zenodo artifact collection: https://zenodo.org/records/14718828)	https://doi.org/10.1007/978-3-031-51476-0_13	10.1109/ACCESS.2019.2953511
2019	2024	2024	2019
USA and UK	China and USA	China	China and United Kingdom
Craig Disselkoen, John Renner, Conrad Watt, Tal Garfinkel, Amit Levy, and Deian Stefan	Lingming Zhang, Binbin Zhao, Jiacheng Xu, Peiyu Liu, Qinge Xie, Yuan Tian, Jianhai Chen, and Shouling Ji	Yifan Xia, Ping He, Xuhong Zhang, Peiyu Liu, Shouling Ji, and Wenhai Wang	Shuai Wang, Guixin Ye, Meng Li, Lu Yuan, Zhanrong Tang, Huanling Wang, Wei Wang, Fuwei Wang, Jie Ren, Dingqi Fang, and Zheng Wang
Conference; HASP '19 (Workshop on Hardware and Architectural Support for Security and Privacy)	USENIX Security Symposium (Conference)	ESORICS 2023 (Conference)	IEEE Access (Journal)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Web browsers, server-side runtimes, IoT platforms, edge computing, video decoding, crypto libraries, and game engines	Web browsers (computation-intensive tasks), standalone runtimes, standalone platforms beyond web scenarios, and cloud deployments (wasmCloud).	Web browsers, NodeJS ecosystem, Open Source Software (NPM), Cryptojacking, and JavaScript Skimmers.	Numerical JavaScript kernels (image processing, cryptography, gaming), web-based applications, and mobile devices
Buffer overflows, Use-After-Free (UAF), pointer corruption, manufacturing pointers from non-pointer values, and control flow bending.	Sandbox escape, remote code execution (RCE), stack overflow, out-of-bounds read/write, heap buffer overflow, and global buffer overflow.	Cryptojacking, Drive-by-download attacks, JavaScript Skimmers, and Multilingual Malware (JWMM) that hides malicious behaviours in cross-language interoperations	Unauthorized reverse engineering, unauthorized modification/misuse of code (cheating, bypassing security), and injection-based attacks.
Malicious (attackers exploiting bugs to compromise Wasm code and the data it handles)	Malicious (attackers aiming for sandbox escape and remote code execution).	Malicious (attackers hiding malicious functionalities within cross-language interoperations to evade anti-virus detection)	Malicious (attacker with access to target code and modified browsers for static or dynamic analysis)
Memory-safety vulnerabilities (buffer overflows and UAF) are considered most severe because they allow attackers to bypass sandboxing isolation to compromise sensitive data like cryptographic keys within the Wasm memory.	Breaking the host security (sandbox escape and remote code execution) is considered the most severe because it can result in full control over the user system.	JavaScript-WebAssembly Multilingual Malware (JWMM) is considered severe because it successfully evades over half of the detection by leading anti-virus platforms like VirusTotal.	Unauthorized reverse engineering and modification of sensitive algorithms and data.
The linear memory ("untyped array of bytes") and function pointer tables (whose indexes live in memory).	WebAssembly runtimes (standalone and browser-embedded), host environments (JS engines like V8, JavaScriptCore, and SpiderMonkey), and virtual stacks/heap metadata.	Foreign language interfaces (FFI), interoperations between JavaScript and WebAssembly, and WebAssembly binaries (hiding sensitive data/instructions)	JavaScript source code (open visibility), WebAssembly binaries (as a target for obfuscation), and Virtual Machine (VM) interpreters.
Not Defined	Runtimes in web browsers (V8, JavaScriptCore) and non-web embeddings (wasm3, wasm-micro-runtime, wasmtime, wasm-interp) contain memory corruption vulnerabilities.	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Server-side use cases were investigated as critical environments where memory safety vulnerabilities are rampant	Not Defined	Multilingual Malware (JWMM) hiding JavaScript malware behaviours within WebAssembly units	Reverse engineering of numerical JavaScript kernels.
A proposed extension (MS-Wasm) that adds segment memory and handles to Wasm, allowing it to capture and enforce low-level memory semantics	A practical greybox fuzzing framework (WALTZZ) using stack-invariant transformation to maintain valid semantics and a skeleton-based generation algorithm to explore rare code constructs.	JWBinder, which uses language-specific data-flow analysis to identify interoperations and Static Semantics Reconstruction (SSR) to build an Inter-language Program Dependency Graph (IPDG) to reconstruct a pure JavaScript program for analysis.	JSPro, a code virtualization system that translates JavaScript into WebAssembly and performs code obfuscation at the lower binary level using a stack virtual machine.
Not Defined	Not Defined	Reconstruction of pure JavaScript programs to allow the reuse of existing monolingual anti-virus detection patterns.	Code obfuscation and virtualization to increase reverse engineering complexity for numerical kernels.
The loss of high-level semantic information (e.g., array sizes) during compilation to Wasm, which prevents Wasm from exploiting hardware safety features like tagged memory or pointer authentication.	Effective exploration of the input space and the maintenance of appropriate stack semantics during mutation.	Detection challenges due to interoperation complexity and semantics diversity.	Challenges in applying code virtualization to JavaScript due to source visibility and high performance overhead.
MS-Wasm backends utilizing hardware features (ARM MTE, Sparc ADI, Intel MPX) and software schemes like Baggy Bounds, Delta Pointers, and AddressSanitizer (ASan).	Greybox fuzzing (WALTZZ), wasm-smith (generative), Waplique (black-box splicing), and WasmFuzzer (structure-aware mutation).	JWBinder, MinerRay (static detector for malicious Wasm like cryptominers), and Wobfuscator (JWMM generation).	Not Defined
The MS-Wasm extension to bridge the gap between low-level code and hardware safety.	Not Defined	Not Defined	Code virtualization and obfuscation implemented at the compiled WebAssembly binary level
Segments and handles for spatial/temporal safety, Pointer Authentication (PAC) for integrity, and CHERI hardware capabilities.	Identifying and rectifying security bugs in runtimes (e.g., heap buffer overflows) to prevent exploits.	Not Defined	Not Defined
Not Defined	Targeting vulnerabilities in JIT tiers and JIT compilers (e.g., Fuzzilli for JS JIT vulnerabilities).	Not Defined	Not Defined
future work includes determining optimal handle sizing (64-bit), refining pointer semantics in C, and developing ways to protect the Wasm stack and function pointers.	Targeting diverse architectures (e.g., ARM), diversifying command-line arguments, incorporating ongoing proposals (GC, threads), and exploring the interplay between Wasm inputs and the host environment (WASI)	Threat of runtime code/data generation, advanced obfuscation, and developing more advanced multilingual analysis techniques based on the IPDG structure.	Further optimization of browser kernels to handle DOM or interactive operations without decreasing performance.
MS-Wasm and discusses V8 in Chrome, libsonum, and AddressSanitizer.	WALTZZ, Binayen (parsing), AFL++, wasm-smith, Waplique, WasmFuzzer, and RedQueen.	JWBinder, VirusTotal, Espirima, Wobfuscator, MinerRay, and Escodegen.	JSPro, Emscripten, JScrambler, and JShaman.
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

22	23	24	25
SFuzzer: An Efficient Way to Fuzz WebAssembly Interpreters	MineThrottle: Defending against Wasm In-Browser Cryptojacking	Bringing Binary Exploitation at Port 80: Understanding C Vulnerabilities in WebAssembly	Is This the Same Code? A Comprehensive Study of Decompilation Techniques for WebAssembly Binaries.
https://doi.org/10.3390/electronics13081498	https://doi.org/10.1145/3366423.3380965	10.5220/0012852400003767	10.1007/978-3-031-94448-2_6
2024	2020	2024	2026
China	Hong Kong, China	Italy	USA
Jideng Han, Zhaoxin Zhang, Yuejin Du, Wei Wang, and Xiuyuan Chen	Weikang Bian, Wei Meng, and Mingxue Zhang	Enmanuele Massidda, Lorenzo Pisu, Davide Maiorca, and Giorgio Giacinto	Wei-Cheng Wu, Yutian Yan, Halgrimur David Egilsson, David Park, Steven Chan, Christophe Hauser, and Weihang Wang
Electronics (Journal)	Proceedings of The Web Conference 2020 (Conference)	21st International Conference on Security and Cryptography (SECRYPT 2024)	SecureComm 2024
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Web browsers (Chrome, Firefox, Safari, and Edge), computationally intensive tasks, and smart contracts (EOSIO).	In-browser cryptocurrency mining (cryptojacking), websites, blockchain algorithms (CryptoNight, Monero), games, and graphics applications.	General web applications and commercial products such as AutoCAD and Figma	Internet of Things (IoT), mobile devices, smart contracts, and dedicated runtime environments.
Security vulnerabilities in WebAssembly interpreters (e.g., Webkit and WAVM)	In-browser cryptojacking (drive-by mining), CPU resource abuse, and code obfuscation to evade detection.	Buffer Overflow (BOF), Format String, Use After Free (UAF), Integer Overflow, Improper Validation of Array Index, and Redirecting Indirect Calls.	Stack overflow vulnerabilities are used as a motivating example.
Not Defined	Malicious (remote web attacker controlling a website containing arbitrary JavaScript and Wasm code)	Malicious; the paper demonstrates how these vulnerabilities can be actively exploited to achieve arbitrary read and write operations.	Not Defined
Not Defined	Drive-by cryptocurrency mining (cryptojacking) is considered an emerging threat because it consumes user computing resources without consent or action.	Remote Code Execution (RCE) is identified as most critical because it allows an attacker to inject and execute arbitrary code on a target server.	Not Defined
WebAssembly interpreters, stack-based virtual machines, validation algorithms (value and control stacks), and instruction parsing/execution.	V8 Wasm interpreter and compiler (used to hook and inject analysis code), browser threads, and Wasm instruction execution.	The stack and heap within linear memory, as well as indirect call tables.	Low-level Wasm bytecode and minified binaries that hinder manual inspection.
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Drive-by cryptocurrency mining (cryptojacking) in browser environments.	Client-side and server-side web applications (specifically focusing on XSS and RCE)	Not Defined
ESFuzzer, a fuzzing framework using the Equivalent-Statement concept and a Stack Repair Algorithm to address interpreter validation challenges.	MineThrottle, a browser-based defense using JIT code instrumentation and block-level program profiling to detect and throttle mining-related threads.	Systematic categorization and the development of Proof of Concepts (PoCs) to demonstrate exploitability.	Evaluation of decompilation techniques to recover high-level source code for vulnerability auditing.
Not Defined	Real-time throttling (penalising/suspending mining-related threads) based on user-configurable policies.	Not Defined	Sandboxing third-party libraries via auditing.
Inefficiencies in fuzzing tests caused by Wasm's static checking mechanism and low coverage due to lack of targeted mutation strategies.	Lack of effective defense mechanisms within user browsers to stop malicious drive-by mining.	A lack of built-in security features in Wasm and its compilers, such as Address Space Layout Randomization (ASLR) or stack canaries.	Existing decompilers lack correctness and readability, making manual inspection cumbersome.
ESFuzzer, libFuzzer, AFL, Honggfuzz, Angora, CSmith, and wasm-smith.	MineThrottle, MineSweeper (static code analysis + CPU measurement), Outguard (SVM classifier), CMTracker, SEISMIC (semantic signatures), and community-maintained blacklists (NoCoin).	Structured presentation and categorization of vulnerabilities to facilitate better understanding of security aspects.	A novel framework for empirically evaluating the correctness, readability, and structural similarity of decompiled C code.
Canary-based binary instrumentation (Fuzzm) to detect stack and heap overflows/underflows.	Not Defined	Not Defined	Use of decompilers for auditing binaries.
Not Defined	Not Defined	Identifying the absence of traditional protections like ASLR.	Not Defined
Language-specific fuzzing (CSmith) for testing compilers.	Instrumentation at the JIT compilation time to inject profiling triggers.	Not Defined	Analysis of how aggressive compiler optimization affects decompilation.
Improving the efficiency of fuzzing WebAssembly interpreters by increasing code coverage.	engineering work to resolve unknown implementation bugs that cause failure to load certain websites.	Implementing traditional mitigation techniques such as ASLR or stack canaries.	Developing a framework to assess decompiler correctness across a broader range of applications and languages.
ESFuzzer, libFuzzer, V8	MineThrottle	WABT	Evaluations of tools such as wasm2c, w2c2, and wasm-decompile.
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper.
Finalized	Finalized	Finalized	Finalized

26	27	28	29
RØB: Ransomware over Modern Web Browsers	POWERPOLY: Analyzing Multilingual Programs with the Aid of WebAssembly	LWDIFF: An LLM-Assisted Differential Testing Framework for WebAssembly Runtimes.	TCH: Fast Kernel Context Isolation with Implicit Context Sw
978-1-939133-37-3; identifies as USENIX Security 23	10.1007/978-981-95-3537-8_5	10.1109/ICSE55347.2025.00233	10.1109/SP46215.2023.00086
2023	2026	2025	2023
USA	China	Hong Kong, China; UK; USA	USA
Harun Oz, Ahmet Aris, Abbas Acar, Güliz Seray Tuncay, Leonardo Babun, and Selouk Ulugac	Zhuochen Jiang and Baojian Hua	Shiyao Zhou, Jincheng Wang, He Ye, Hao Zhou, Claire Le Goues, and Xiapu Luo	Dinglan Peng, Congyu Liu, Tapit Palit, Pedro Fonseca, Anjo Vahidek-Oberwagner, and Mona Vij
32nd USENIX Security Symposium	Conference, ICICS 2025	ACM, Journal	Conference; IEEE Symposium on Security and Privacy (SP)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Modern web browsers, cloud-integrated directories, and network-shared folders.	Multilingual programs (e.g., Rust/C, Go/C) and browsers.	Wasm runtimes (Wasmer, Wasmtime, WasmEdge, WAMR), browsers, and blockchain	library isolation in web browsers (e.g., Firefox) and as a general sandboxing platform
Browser-based ransomware (RØB)	Double-free, Use-after-free (UaF), Memory leak, Integer overflow, Null pointer dereference, and Buffer overflows.	Incorrect decoding (undefined opcodes), incorrect instruction implementation, and incorrect control flow handling.	Memory corruption, control flow hijack, remote code execution (e.g., CVE-2017-14632), and remote information disclosure (e.g., CVE-2019-9232). Additionally, attacks on underlying memory isolation systems (like MPK) via system calls such as sigsetum and process_vm_readv.
Malicious; an attacker lures victims to a web application to gain read/write access to local files for extortion.	Not Defined	Not Defined	Malicious; the paper assumes sandboxed functions execute arbitrary code and may contain vulnerabilities that an attacker could exploit.
Ransomware targeting cloud-integrated and network-shared folders is severe because existing antivirus solutions fail to detect it.	Vulnerabilities occurring at and across language boundaries (FFIs) are severe as single-language analysis cannot detect them.	Bugs in runtime implementation are critical as they lead to incorrect calculation results or runtime crashes.	Remote code execution (RCE) and remote information disclosure are highlighted because they allow attackers to compromise entire applications or steal sensitive user data across origins.
Wasm-based encryption libraries used to bypass platform security.	Foreign Function Interfaces (FFIs) and the function table.	Runtime phases including decoding, validation, and execution	Third-party libraries and the system call interface (kernel context) are primary sources of vulnerabilities.
The File System Access (FSA) API combined with Wasm enables access to various sensitive local directories.	Language boundary discrepancies in Rust/C and Go/C combinations.	Vulnerabilities within Wasm runtime implementations across various deployment contexts.	In web browsers, third-party audio (e.g., libogg) and video (e.g., libvpx) decoding libraries are vulnerable. In HTTP servers, components like network libraries (libevent) or script interpreters may compromise sensitive resources.
Not Defined	Not Defined	Not Defined	Not Defined
Ransomware encryption of cloud-integrated directories such as Google Drive and Dropbox.	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
External storage devices and network shared folders	Not Defined	Not Defined	In web browsers, information disclosure between origins and RCE through malicious images. In HTTP servers, arbitrary code execution with root privileges.
Malicious Modification Identification, Local Activity Monitoring, and New UI Design.	PowerPoly, utilizing Wasm as a unified Intermediate Representation (IR) for static and dynamic analysis.	LWDIFF, an LLM-assisted differential testing framework	The paper proposes µSWITCH, a fast kernel context isolation mechanism that uses implicit context switching and Intel MPK to isolate process memory and kernel resources into fine-grained domains.
Redesigned UI permission dialogs to explicitly warn users of permanent data loss risks.	User-supplied security plugins.	Comprehensive testing of all three runtime phases to identify and fix implementation defects.	For browsers, fine-grained library isolation (per-origin or per-component) using µSWITCH integrated with the RLibBox framework. For HTTP servers, isolating the network library and using Seccomp filters to minimize available system calls.
Existing browser UIs lack indicators to warn users of potential ransomware, and API countermeasures are ineffective.	Existing tools are specialized for specific language combinations (e.g., Java/C) and lack generalizability.	Existing testing frameworks fail to generate test cases that effectively cover decoding, validation, and execution phases together.	High performance overhead for fine-grained process isolation and the limited coverage of the system call interface in existing in-process virtualization techniques like WASI.
Entropy change and file size change identification via API hooking; monitoring FSA API function call sequences.	Static instrumentation for dynamic tracking of freed memory and integer overflow validation.	LLM-extracted mutation operators and a Multi-Armed Bandit (MAB)-based mutation selector.	Binary inspection is used to find and rewrite or abort unsafe instructions (e.g., WRPKRU, XRSTOR) that could circumvent memory protection.
Not Defined	Use of static instrumentation for vulnerability detection.	Not Defined	Binary inspection and dynamic rewriting of unsafe instruction sequences.
Not Defined	Using "dirty values" to track and detect Double Free bugs across language boundaries.	Not Defined	Hardware-assisted intra-process memory isolation using Intel MPK partitions.
Not Defined	Not Defined	Not Defined	Not Defined
More research is needed to enable web applications to interact with local files in a secure manner.	Supporting more language combinations (Python/C, Java/C) and investigating concurrency vulnerabilities like race conditions and deadlocks.	Not Defined	future work includes supporting serverless computing, automatically applying sandboxes, and handling JIT compilers by falling back to interpreters if unsafe instructions are found.
The RØB proof-of-concept and accompanying defense solutions code.	PowerPoly (a framework for holistic multilingual program analysis)	LWDIFF (differential testing framework for runtimes)	µSWITCH
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper.
Finalized	Finalized	Finalized	Finalized

30	31	32	33
WaSCR: A WebAssembly Instruction-Timing Side Channel Rep...	VMCANARY: Effective Memory Protection for WebAssembly via Virtual Machine-assisted Approach	WAMO: Toward Secure Browser Inference via Web Model Obfuscation in WebAssembly	WaSC: Hardening WebAssembly Sandboxes via System Interface Decoupling
10.1145/3696410.3714693	10.1109/QR560937.2023.00070	10.1145/3774904.3792297	10.1145/3795882
2025	2023	2026	2026
USA	China	China	China and USA
Liyan Huang, Junzhou He, Chao Wang, and Weihang Wang	Ziyao Zhang, Wenlong Zheng, Baoqian Hua, Qiliang Fan, and Zhizhong Pan	Yitong Wang, Pengfei Yu, Hao Han, Jingling Gu, Fengyuan Xu, and Xinyi Huang	Xingzi Yu, Tianlei Xiong, Wei Tang, Yufan Jiang, Zhixiang Wei, Hao Wang, Chen Chen, Yun Wang, Bo Peng, and Zhengwei Qi
Conference; ACM Web Conference 2025 (WWW '25)	Conference; IEEE International Conference on Software Quality, Reliability and Security (QRS)	Conference; ACM Web Conference 2026 (WWW '26)	Journal; ACM Transactions on Architecture and Code Optimization (TACO)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Used extensively in web applications to deliver near-native performance alongside JavaScript	Used in web browsers, edge computing, smart contracts, standalone runtimes, and serverless cloud computing.	Used as a backend for AI inference in browsers (machine learning), real-time translation, speech recognition, image enhancement, and interactive recommendation systems.	Currently used in serverless computing (FaaS), microservice architectures, IoT, blockchain, and deep learning
Instruction-timing side-channel attacks, Spectre, port contention, and cache attacks.	Buffer overflow attacks (stack-based) and potential heap-based attacks	Model theft (extraction), reverse engineering, tampering, adversarial manipulation, and side-channel attacks on memory/cache.	Sandbox escape (escape), host kernel compromise (cross-container compromise), filesystem privilege escalation, capability violations, DoS (via malformed packets), and memory-safety violations.
Malicious; the attacker is capable of executing programs repeatedly and measuring execution time at various granularities to infer sensitive information	Malicious; assumes that high-level source code and complex compilation toolchains are unreliable and may introduce or propagate vulnerabilities into Wasm binaries.	Malicious; assumes a strong adversary in an untrusted client environment who can monitor network traffic and access browser caches to extract model intellectual property.	Malicious; assumes untrusted serverless functions and malicious neighbors that can escape the Wasm sandbox to attack the VM or host kernel.
Instruction-timing side-channel attacks are critical because they allow attackers to infer sensitive data like passwords or encryption keys by measuring timing discrepancies in conditional branches.	Stack smashing/buffer overflows on the unmanaged data stack are severe because they can corrupt data stack frames or heaps, leading to system compromise.	Model stealing (extraction) is highly severe because it allows competitors or malicious actors to steal proprietary trained weights and architectures directly from the browser.	Cross-container compromise and host kernel vulnerability exploitation are most severe because vulnerabilities in the shared kernel allow containers to escape isolation and compromise the entire host.
Conditional branch instructions (if, br, if) dependent on sensitive data and linear memory (which can hide implicit data dependencies)	Wasm's linear memory (specifically the data/unmanaged stack) and complex compilation toolchains.	JavaScript frameworks (loading/caching assets), the data section of Wasm files (where weights reside), and the computation graph/operator logic.	The Wasm System Interface (WASI) implementation, the host OS kernel (due to a large syscall attack surface), the Wasm runtime itself, and I/O drivers.
In web applications, branching logic in authentication functions (e.g., password checkers) and cryptographic algorithms is vulnerable to timing leaks.	In edge computing and smart contracts, Wasm programs are vulnerable because the memory model allows overflows on the data stack to corrupt adjacent memory regions.	In browser-side AI inference, the exposed weight parameters and transparent operator call chains are vulnerable to reconstruction.	In serverless computing, the WASI API surface and the underlying host syscall ABI are the primary targets for exploitation.
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Stack buffer overflows resulting from unsafe source language practices.	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Stack buffer overflows propagated from sources to Wasm binaries	Not Defined	Not Defined
In web-based modules, timing attacks on encryption (e.g., DES, Twofish, SHA) and string comparison routines.	Not Defined	AI Inference: Systematic model stealing via intercepting .json or .bin files from traffic or local browser storage.	Serverless: sandbox escape via stackheap bugs, host kernel exploitation via vulnerable syscall paths, and DoS attacks targeting the I/O interface.
WaSCR, an automated tool that uses Program Dependency Graph (PDG) construction, sound static taint analysis, and rule-based code transformation to linearize vulnerable branches using constant-time selectors.	VMCANARY, a framework that uses VM-assisted stack canaries. It introduces novel Wasm instructions (canary.insert, canary.check) and uses automated binary instrumentation to detect overflows during execution.	WAMO, an obfuscation framework that converts AI models into hardened Wasm-native modules using weight encryption, operator path perturbation, computation graph obfuscation, and metadata obfuscation.	WaSC (WebAssembly Secure Container), which hardens isolation by decoupling the system interface (WASI) into a virtualization-based daemon (WaSID) protected by a triple-layer syscall barrier.
For web applications, branch linearization (replacing sensitive branches with the select instruction) and adaptive loop management to establish fixed execution times.	For edge and smart contracts, protecting the unmanaged stack by inserting canaries whose semantics are understood and verified by a tailored Wasm virtual machine.	For AI inference, shifting from JavaScript assets to hardened Wasm binaries, applying dummy operator injection, and operator dispatch indexing.	For FaaS, machine-level isolation for system interfaces, TCB minimization (using tiny Linux kernels and device delegation), and multi-mode isolation (monopolize, capability exclusion, or shared capabilities)
Existing tools often require platform-specific extensions, lack automation for repairs, or cannot easily adapt to Wasm's diverse runtimes and stack-based architecture.	Traditional canaries fail in Wasm due to incorrect placement (on the control stack instead of the data stack) and wrong spatial ordering.	Lack of conversion toolchains from standard model formats to Wasm-native representations and the difficulty of applying fine-grained, model-aware transformations directly at the Wasm layer.	WASI does not guarantee isolation from the host kernel, and current secure containers (MicroVMs) often compromise Wasm's lightweight memory efficiency and startup latency.
Sound static taint analysis on a PDG to identify branches control- or data-dependent on sensitive information.	Dynamic sanity checking of canary values by a virtual machine execution engine right before a function returns.	Evaluating resistance using cyclomatic complexity, Halstead complexity metrics, and normalized memory-distribution entropy during inference.	Not Defined
Rule-based transformation at the Wasm language level (WAT format) to ensure semantic equivalence after linearization.	Standalone binary instrumentation via AST-based rewriting that is neutral to the compilation toolchain.	Multi-dimensional Wasm-layer obfuscation (flattening execution paths, scrambling metadata, and injecting forged signatures).	Uses Software Fault Isolation (SFI) to protect linear memory within individual sandboxes.
Conservative matching of memory load and store instructions based on address variables and offsets during taint tracking	A code-based canary approach to protect the unmanaged stack within linear memory.	Not Defined	Decoupling and intercepting WASI function calls to prevent escaping the sandbox through host OS vulnerabilities.
Not Defined	Not Defined	Not Defined	Not Defined
Expanding analysis to imported function calls (e.g., JavaScript APIs), addressing multidimensional side channels (microarchitectural leaks), and mandating constant-time behavior in the official Wasm specification.	optimizing instrumentation positions via static analysis, providing protection for the unmanaged heap, and implementing memory protection for non-open-source VMs via dynamic hooks.	model watermarking, runtime integrity verification, and adaptive obfuscation	automating signature table generation, supporting cross-tenant sharing of daemons, implementing persistent state for failure-aware reattachment, and batching fine-grained I/O operations.
WaSCR	VMCANARY	Introduces WAMO and compares it with Tigress, Wasm-mutate, and WASMixer	Introduces WaSC and WaSID, and discusses Twine, Firecracker, and gVisor.
First-hand research paper.	First-hand research paper.	First-hand research paper.	First-hand research paper.
Finalized	Finalized	Finalized	Finalized

34	35	36	37
A Comprehensive Study of Bugs in Embedded WebAssembly Virtual Machines.	Stack-based static WebAssembly binary slicing and mutation for generating valid sub-binaries	SoK: Analysis Techniques for WebAssembly	Furina: A Light-weight WebAssembly Runtime for ICS
10.1109/CE60616.2023.10528174	https://doi.org/10.1038/s41598-026-45837-y	https://doi.org/10.3390/16030084	https://doi.org/10.1145/3733823.3764514
2023	2026	2024	2025
China	Republic of Korea	Norway	China
Wenlong Zheng and Baojian Hua	GyeongTaek Choi and Seunggho Jeon	Håkon Harnes and Donn Morrison	Hanwen Lei, Ding Li, and Yao Guo
Conference; 2023 3rd International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI)	Journal; Scientific Reports	Journal; Future Internet	Conference; 2025 Workshop on Re-design Industrial Control Systems with Security (RICSS '25)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Embedded applications, gateway API, edge computing, Internet-of-Things (IoT), Trusted Execution Environment (TEE), smart contracts, and cloud-native environments.	Web browsers, cloud computing, and embedded devices	Web browsers, desktop/mobile apps, cloud computing, blockchain/smart contracts, IoT, and embedded devices.	Industrial Control Systems (ICS), microservices, edge computing, AI/ML, and data storage.
Memory corruption (buffer overflows, heap metadata overwriting), sandbox escape (via WASI), data corruption, system crashes, and register usage errors in SIMD implementations.	General implementation vulnerabilities in Wasm runtimes discovered via testing.	vulnerabilities from memory-unsafe source languages (C/C++), and smart contract exploits (e.g., fake transfers, forged notifications)	Remote code injection, control flow redirection, and remote code execution (RCE)
Malicious	Not Defined	Malicious (involved in cryptojacking and smart contract attacks)	Not Defined
Memory corruption and sandbox escapes are highlighted as severe because they defeat the safety guarantees of Wasm, potentially leading to the corruption of hosting environments.	Not Defined	Smart contract vulnerabilities due to irreversible financial losses (e.g., USD 1.9 million lost in EOS tokens) and cryptojacking due to its massive prevalence.	Remote code injection and RCE in actively deployed systems
The runtime environment (execution engines and high-level language APIs), auxiliary tools, and the WebAssembly System Interface (WASI).	Wasm runtimes (execution environments)	Linear memory (buffer overflows), data sections (script injection), and smart contract code.	Shared linear memory (as an isolation boundary), and external environment connections (import section).
WASI: Incorrect implementations of file system interactions can break the sandbox, allowing files to be renamed outside pre-opened directories.	Not Defined	Blockchain: Forged notifications, missing authorization verification, and random number generation flaws. Web: XSS attacks through data section modification via Emscripten API.	ICS: Monolithic software stacks are susceptible to remote code injection and cascading outages.
Memory mode incompatibilities leading to crashes; Limited memory on embedded platforms can lead to system crashes if allocation is handled incorrectly.	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Resource-demanding binary vulnerabilities leading to outages (e.g., Fastly outage)	Not Defined
Not Defined	Not Defined	Not Defined	Remote code injection and RCE attacks
Not Defined	Not Defined	Fake EOS transfer, forged transfer notifications, and blockinfo dependency.	Not Defined
Not Defined	Not Defined	Cryptojacking (stealing hardware resources for mining)	Not Defined
Qualitative and quantitative empirical study of bug reports to identify root causes and develop fixing strategies.	Static program slicing, stack balance correction, and instruction-level stack-preserving backward mutation to generate diverse binaries for security verification.	Systematic review classifying analysis techniques into static, dynamic, and hybrid categories.	Furina framework, employing WebAssembly-based co-located sidecars and zero-copy communication via shared linear memory.
IoT: Leveraging specialized memory management and dealing with embedded platform incompatibilities. General: Adding memory boundary checks, adjusting allocators, and implementing missing WASI features.	Not Defined	Cryptojacking: MineSweeper (static), SEISMIC (dynamic), OutGuard (dynamic). Vulnerability Detection: Wassail (static), TaintAssembly (dynamic), WASP2 (hybrid). Smart Contracts: WANA (static), WASAI (dynamic).	ICS: Lightweight sandboxing, software fault isolation (SFI), and sidecar functions for rate limiting and access control.
A lack of automated generation of missing information in bug reports, which currently hinders dynamic debugging and reproduction.	Current static analysis algorithms for control dependency (like post-dominator) become inefficient as function graph complexity increases.	Security of Wasm in non-web environments; impact of code obfuscation on dynamic detection.	Existing container-based microservices are too heavyweight for resource-constrained ICS; lack of eBPF hooks in some ICS environments.
Manual inspection of bug reports and source code, complemented by script-based collection.	Differential testing and fuzzing of Wasm runtimes using generated sub-binaries	Signature matching, symbolic execution, taint tracking, binary instrumentation, and machine learning (CNN, SVM).	Sidecar-based monitoring, request logging, and metrics collection.
Not Defined	Generating valid, diverse sub-binaries to exercise broader range of instruction decoder paths.	Static analysis (CPG), inter-procedural control flow graphs (ICFG), and fuzzing (Fuzzm, WAFL)	Verifiable capability boundaries via explicit WASI declarations in the import section.
Adding range checks to fixed buggy implementations and releasing unused memory promptly.	Not Defined	Sandboxing, linear memory boundaries, and canary-based protection.	Language-level sandboxing and linear memory that prohibits raw host memory access.
SIMD proposal bugs caused by incorrect instruction lowering and selection.	Not Defined	Not Defined	Not Defined
Improving the bug-fixing ability for intractable bugs and addressing feature-specific challenges.	Introducing more efficient algorithms like Lengauer-Tarjan; researching control-flow-level mutation and crossover between functions across binaries.	Detecting other malicious uses like keyloggers; evaluating obfuscation effects on hash rates; prevalence of mining after CoinHive shutdown.	Not Defined
Datasets released at Zenodo for VM bug study.	Python implementation of slicing and mutation algorithms; WasmBench dataset, wabt (wasm-validate).	Surveyed tools include MineSweeper, Wasabi, Fuzzm, EVulHunter, and WANA.	Furina microservice runtime (built on Wasmtime)
First-hand research paper (empirical study)	First-hand research paper	Review (survey) paper: SoK	First-hand research paper
Finalized	Finalized	Finalized	Finalized

38	39	40	41
Ransomware Over Modern Web Browsers: A Novel Strain and a New Defense Mechanism	WASHADOW: Effectively Protecting WebAssembly Memory Through Virtual Machine-Aware Shadow Memory.	WebAssembly and security: A review	Vanilla: Sound Noninterference Analysis for WebAssembly
https://doi.org/10.1145/3708514	10.1109/TrustCom63139.2024.00055	10.1016/j.cosrev.2025.100728	10.1145/3719027.3765156
2025	2024	2025	2025
United States	China	Italy	Austria, Denmark
Harun Oz, Güliz Seray Tuncay, Ahmet Aris, Abbas Acar, Leonardo Babun, and Selcuk Ulugac.	Zhuochen Jiang, Baojian Hua	Gaetano Perrone, Simon Pietro Romano	Markus Scherer, Jeppe Fredsgaard Blaabjerg, Alexander Sjösten, Matteo Maffei
Journal; ACM Transactions on the Web	Conference; IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)	Journal; Computer Science Review	Conference; ACM SIGSAC Conference on Computer and Communications Security (CCS)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Web browsers, cloud-integrated directories (iCloud, Box), external storage, and network shared folders.	Edge computing, smart contracts, cloud computing, and server-side computing.	Web applications, data visualization, Internet of Things (IoT), AI, gaming, backend services, automotive, manufacturing, medical, smart contracts/blockchain, and edge computing.	Plugin systems, smart contracts, and edge computing.
ReB (browser-based ransomware), which uses the File System Access (FSA) API and Wasm-based encryption to encrypt user files.	Integer overflows, memory corruption, stack-based and heap-based buffer overflows, constant data overwrite, use-after-free, double free, null pointer dereference, ROP attacks, and control-flow hijacking.	Cryptojacking, memory attacks (stack/heap overflows), integer overflows, side-channel attacks (cache analysis, port contention), malware detection evasion (obfuscation), ransomware, XSS, RCE, and logic flaws in smart contracts.	Memory corruption, data overwriting (including "constant" data), confidentiality leaks (secrets disclosure), integrity breaches, XSS, and bad randomness in smart contracts.
Malicious (attacker Mallory).	Malicious; the paper assumes that both source code and compiler toolchains are unreliable and vulnerable, while an adversary attempts to trigger buffer overflows to corrupt adjacent memory.	Discusses various models including malicious users, untrusted cloud providers, and malicious WebAssembly binaries.	Malicious; an attacker monitoring public outputs to learn secrets or modifying untrusted inputs to alter trusted state.
ReB ransomware is severe because it requires no binary installation, evades commercial antivirus, and permanently encrypts files even in synced cloud folders without versioning (like iCloud).	Buffer overflows are highlighted as a significant and notorious threat because they can overwrite security-sensitive data in the unmanaged stack and corrupt heap metadata.	Memory-related vulnerabilities porting from C code are considered critical because they can lead to Remote Code Execution (RCE) and Cross-Site Scripting (XSS).	Memory corruption originating from memory-unsafe source languages is considered critical because it can break layout and control flow assumptions, leading to various exploits.
Attackers exploit Wasm-compiled encryption libraries (e.g., OpenSSL via Wasm) to hide activity from OS-level crypto API monitors.	Linear memory (including the unmanaged stack, heap, and static data areas) and compiler toolchains (e.g., Emscripten, LLVM, Clang).	Linear memory, compilers, runtime engines, interpreters, the WebAssembly System Interface (WASI), and CPU microarchitecture (vulnerable to side-channels).	Linear memory (due to lack of structure and shared mutability) and imported functions with observable side effects.
WebCloud: Abuse of the File System Access (FSA) API to gain read/write access to user directories through the browser.	Memory corruption vulnerabilities in linear memory are specifically noted as exploitable in edge computing and smart contract environments.	Mentions linear memory issues in browsers and logic/authorization flaws in the apply functions of smart contracts.	Shared mutable memory in a module is specifically dangerous for edge computing and smart contracts when compiled from unsafe languages.
Not Defined	Not Defined	General security disparities between Wasm modules and Docker containers.	Not Defined
Ransomware encryption of synced cloud folders (iCloud, Box Individual) lacking robust versioning.	Not Defined	Sensitive data disclosure in Trusted Execution Environments (TEEs) and host isolation breaches.	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Fake EOS transfer/receipt/notice, reentrancy, rollback, missing permission checks, pseudo-random number generator (PRNG) issues, and integer overflows.	Exploitation of "bad randomness" (using block information) to predict lottery outcomes and claim prizes.
Encryption of files in external storage (HDD/Flash) and shared network volumes.	Not Defined	Ransomware attacks over modern web browsers.	XSS attacks in web applications via privileged function calls. Browsers: Leaking secret session IDs through untrusted components.
ReBGuard, which uses function hooking to monitor FSA API calls and machine learning to detect malicious intent based on entropy and size changes.	Virtual machine-aware shadow memory combined with static binary instrumentation and dynamic sanity checking algorithms.	This is a review paper; it summarizes methods such as TEEs (Intel SGX/TruZone), static/dynamic analysis, language extensions, and fuzzing.	Vanilla, a sound and fully static noninterference analysis that tracks taints on values and uses value-sensitive relational reasoning.
Browser: ReBGuard (hooks FSA functions to pause web apps before permanent file swaps).	Not Defined	Integration with SGX/TruZone for cloud/edge; type-driven secure languages for cryptography; formal specification verification for compilers.	Not Defined
Misleading documentation regarding FSA API security countermeasures; difficulty in distinguishing between cryptomining (with consent) and cryptojacking.	Not Defined	Heavy reliance on retired benchmarks (Alexa); limited dark web investigation; lack of focus on cyber ranges; and a need to analyze source languages beyond C, such as Rust, Python, and JavaScript.	Not Defined
Monitoring entropy change and size change between original files and swap files; ML classifiers (Random Forest, KNN, DT, XGBoost)	Shadow memory layout tracking, canary instructions (canary.insert, canary.check), and heuristics-based algorithms for validating memory load, store, allocation, and deallocation operations.	Machine learning (SVM, CNN, Doc2Vec), static analysis (Control-Flow Graphs), symbolic execution, taint tracking, and runtime execution monitoring.	Taint tracking combined with abstract interpretation, relational reasoning over related runs, and Scope Extensions for analyzing implicit flows.
Not Defined	Static binary instrumentation and rewriting to insert instructions at function entry/exit blocks.	Code diversification, obfuscation, self-checking mechanisms, and binary rewriting.	Sound reachability and noninterference analysis to verify module integrity.
Not Defined	Use of a scale mapping (16 bytes of linear memory to 1 byte of shadow memory) to enforce access privileges and detect overflows.	MSWasm (segment memory), Vncanary, MPK-based isolation, and shadowing metadata in JavaScript.	Precise reasoning over addresses in linear memory to differentiate regions and identify corruption.
Not Defined	Not Defined	Hardening against Spectre, formal specification verification, and fuzzing (e.g., WasmCFuzz).	Not Defined
Investigating physical side channels (power usage, EM emissions) to detect ransomware; researching deep learning for lightweight real-time detection.	Improving accuracy via fine-grained shadow memory models, supporting concurrency/race condition detection, and applying the method to commercial VMs without available source code.	Developing updated benchmarks, investigating Wasm adoption on the dark web, and exploring cyber ranges for security training.	Extending the approach to robust declassification, improving composability for larger code bases, and verifying hyperproperties beyond noninterference.
ReBGuard defense solution.	WASHADOW	Summarizes tools like Wasabi, Wassail, SecWasm, WANA, EOSAFE, RLBox, Swivel, and MSWasm.	Vanilla
First-hand research paper	First-hand research paper	Review (survey) paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

42	43	44	45
WasmFuzzer: A Fuzzer for WebAssembly Virtual Machines	Resilient Web Code Execution: E-Graph Driven Obfuscation for Microarchitectural Attack Mitigation	WasmGuard: Enhancing Web Security through Robust Raw-Binary Detection of WebAssembly Malware	Cage: Hardware-Accelerated Safe WebAssembly
10.18293/SEKE2022-165	10.1109/ICSADL67539.2026.11451945	https://doi.org/10.1145/3696443.3714696	https://doi.org/10.1145/3696443.3708920
2022	2026	2025	2025
China, Hong Kong	USA	China and Canada	Germany, The Netherlands, and Finland
Bo Jiang, Zichao Li, Yuhe Huang, Zhenyu Zhang, W.K. Chan	Dhanush Gopal Battina	Yuxia Sun, HuiHong Chen, Zhixiao Fu, Wenjian Lv, Zitao Liu, and Haolin Liu	Martin Fink, Dimitrios Stavrakakis, Dennis Sprokholt, Soham Chakraborty, Jan-Erik Ekberg, and Pramod Bhatotia
Conference; International Conference on Software Engineering and Knowledge Engineering (SEKE)	Proceedings of the 5th International Conference on Sentiment Analysis and Deep Learning (ICSADL-2026)	Proceedings of the ACM Web Conference 2025 (WWW '25)	Published in a conference: Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization (CGO '25)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Web browsers, blockchain platforms, and embedded systems.	Web applications, serverless computing, blockchain smart contracts, and cloud-edge computing.	It is used in browsers, client-side, and server-side applications, as well as cloud platforms and IoT devices.	Wasm is used in mobile, edge, cloud, IoT devices, Function-as-a-Service (FaaS), as a Linux container alternative, and in WebOS-like scenarios.
Use-after-free, buffer overflows, stack overflows, memory leaks, and unique crashes/hangs in virtual machines.	Microarchitectural side-channel attacks, timing-based side-channel attacks, and specifically Spectre variants: Spectre-v1 (Bounds Check Bypass), Spectre-v2 (Branch Target Injection), and Spectre-v4 (Speculative Store Bypass).	Discovered attacks include cryptopacking, obfuscated malicious scripts, and adversarial attacks (such as FGSM or PGD-S0) designed to evade malware detection systems.	Attacks include buffer overflows, dangling pointers, use-after-free, double-free errors, control-flow hijacking (function pointer reuse), sandbox escapes (e.g., CVE-2023-26489), and Spectre-style attacks.
Malicious; the fuzzer aims to expose implementation bugs that could be exploited by malicious inputs.	Malicious; the paper describes an attacker exploiting hardware optimization features (speculative execution) to bypass software isolation and compromise sensitive data.	A malicious model is used, where the attacker tries to cause integrity violations by inducing the detection model to misclassify Wasm binaries.	A malicious model is used, involving untrusted input from outside the sandbox or untrusted/adversarial programs running within it.
Bugs in the implementation of Wasm virtual machines are considered severe because they can lead to security vulnerabilities or abnormal application behavior.	Microarchitectural attacks like Spectre are highlighted as a fundamental challenge because they can bypass traditional software isolation boundaries provided by Wasm's sandbox and compromise sensitive data.	Not Defined	Memory safety bugs are considered most severe as they constitute 70% to 75% of security vulnerabilities in large software projects and can be exploited to overwrite data or redirect control flow.
WebAssembly Virtual Machines (e.g., WAVM, Wasmtime, Wasmer, WAMR, and EOS-VM)	Hardware optimization features (speculative execution mechanisms), function tables (used for branch target injection), linear memory (facilitates layout manipulation), and structured control flow (can create predictable patterns for timing attacks).	Vulnerabilities can be exploited in raw binaries, specifically through manipulation of custom sections for perturbation or modifying instruction constants to evade detection.	Vulnerabilities exist in linear memory (lack of read-only regions), function tables (overwriting function pointers), and the call stack.
Execution engine logic in blockchain-specific VMs (EOS-VM) and embedded-focused VMs (WAMR)	In web, serverless, and blockchain domains, Wasm's linear memory and structured execution create consistent, measurable patterns in cache behavior and speculative execution that facilitate timing-based side-channel attacks.	Not Defined	In WebOS-like scenarios, leaking function pointers is particularly dangerous if instances share a common library.
Embedded system virtual machine crashes	Not Defined	Malware and adversarial attacks aimed at evading detection frameworks in edge/IoT environments.	Memory safety errors such as out-of-bounds access and use-after-free.
Not Defined	Timing-based side-channel attacks and speculative execution exploitation in serverless computing.	Malware execution and adversarial evasion of static detection models.	Sandbox escapes and memory corruption affecting multi-tenant or FaaS environments.
Not Defined	Not Defined	Not Defined	Not Defined
Bugs in blockchain execution engines leading to results inconsistency or abnormal exits.	Speculative execution attacks (Spectre) to reveal sensitive information.	Not Defined	Not Defined
Not Defined	Web applications in general are investigated for vulnerability to Spectre-type attacks.	Malicious Wasm files on webpages, including cryptojackers that execute in browsers	FaaS and Linux container alternatives are investigated for performance-related sandboxing vulnerabilities.
Bytecode-level fuzzing (WasmFuzzer) with valid seed generation, systematic mutation operators, and an adaptive random mutation strategy.	An automated code diversification framework based on e-graphs (equality graphs) that uses semantic-preserving transformation rules to generate structurally diverse program variants.	The paper proposes WasmGuard, a robust detection framework using FGSM-based adversarial training with prior-based initialization and adversarial contrastive learning to identify raw-binary malware.	The paper introduces Cage, a toolchain leveraging Arm's Memory Tagging Extension (MTE) for spatial/temporal safety and Pointer Authentication (PAC) to prevent function pointer reuse.
Not Defined	Systematic diversification at the binary level is proposed as a proactive defense to increase attack surface complexity for web, serverless, and blockchain applications.	For web security, the paper proposes and evaluates WebChecker, a browser plugin that provides real-time alerts for malicious Wasm files on webpages.	For edge/cloud and mobile, Cage provides hardware-accelerated sandboxing and memory safety for unmodified C/C++ applications on Arm platforms.
Not Defined	Current diversification suffers from limitations: source-level requires code/compiler, compiler-integrated solutions (like CROW) are tied to specific toolchains, and existing binary rewriting often lacks the semantic awareness needed for effective diversification.	Gaps include a lack of public datasets, reliance on labor-intensive feature engineering, and a lack of robust detection methods that defend against adversarial attacks in the evolving malware landscape.	Gaps include the lack of practical, low-overhead solutions for memory safety in C/C++ and the high performance cost of software-based bounds checks in 64-bit Wasm.
Coverage-guided grey-box fuzzing, grammar-aware mutation of bytecode, and the use of AddressSanitizer to capture memory-related software bugs.	Dynamic analysis to measure side-channel characteristics, instruction-level tracing (using Intel Pin), cache behavior monitoring, and differential testing to verify functional equivalence.	Techniques include static detection (e.g., MINOS), dynamic analysis (monitoring CPU cache, memory, and network), and machine learning-based detection using grayscale images or raw binaries.	Evaluated techniques include Wasm memcheck (wmemcheck) and Cage's hardware-based trapping on memory tag mismatches or signature validation failures.
Not Defined	Binary-level diversification using e-graphs involving instruction-level, control-flow, data-flow, and structural transformations (e.g., algebraic identities, if-consequence swapping, dead code insertion)	Adversarial training and binary rewriting techniques (e.g., Wasm-Mutate, BREWasm) to create robust detection models.	An LLVM-based compiler toolchain that instruments stack allocations and function pointers.
Integrating AddressSanitizer during VM compilation and testing to identify overflows and leaks.	Memory operation transformations that introduce redundant loads/stores and modify data layouts to disrupt cache-based side-channels.	Not Defined	Segmented memory and tagged pointers via Arm MTE to protect heap and stack regions.
Not Defined	Not Defined	Not Defined	Not Defined
Exploring new seed generation schemes and fuzzing input scheduling to improve effectiveness.	adaptive diversification strategies, exploring multi-variant execution to detect compromises, using machine learning to discover new transformation rules, and integrating diversification with runtimes for dynamic variant rotation (moving target defense)	It provides a large-scale dataset (WasmMal-15K) to serve as a resource for future research in robust Wasm malware detection.	Future work includes increasing the number of available sandboxes within a single process by combining guard pages with memory tagging.
WasmFuzzer	an e-graph based diversification engine implemented in Rust.	WasmGuard, WebChecker, WasmMal-15K	Cage, a hardened WASI-libc, a modified Wasmtime runtime, and an LLVM-based sanitizer.
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

46	47	48	49
Lurking Eyes: A Method to Detect Side-Channel Attacks on JavaScript and WebAssembly	A Survey of WebAssembly Usage for Embedded Applications: Safety and Portability Considerations	Exploring and Exploiting the Resource Isolation Attack Surface of WebAssembly Containers.	Owl: Performant Parallel Symbolic Execution Made Easy, an Application to WebAssembly
978-1-7281-8569-9/20	https://icseur-ws.org/Vol-3962/paper65.pdf	978-1-939133-52-6	10.22152/programming-journal.org/2025/9/3
2020	2025	2025	2024
Iran	Italy	China	France and Portugal
Mohammad Erfan Mazaheri, Farhad Taheri, and Siavash Bayat Sarmadi	Tancredi Orlando, Luca D'Agati, Francesco Longo, and Giovanni Merlino	Zhaofeng Yu, Dongyang Zhan, Lin Ye, Haining Yu, Hongli Zhang, and Zhihong Tian	Léo André, Filipe Marques, Arthur Carcano, Pierre Chambart, José Frago Santos, and Jean-Christophe Fillâtre
17th International ISC Conference on Information Security and Cryptology (ISCISC'2020)	Joint National Conference on Cybersecurity (ITASEC & SERICS 2025)	Published in a conference: Proceedings of the 34th USENIX Security Symposium	Journal: The Art, Science, and Engineering of Programming
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
It is used in browsers, server-side applications, and on any device with a JavaScript engine	It is used in desktops, servers, cloud, embedded systems, IoT, and industries such as healthcare and manufacturing.	Used as portable containers in servers, cloud environments, browsers, IoT, and cloud-edge computing.	Web applications, server-side runtimes, embedded systems, IoT platforms, cloud platforms, and blockchain smart contracts.
side-channel attacks (timing, cache, DRAM-based), memory deduplication attacks, and Spectre.	Attacks include buffer overflows, use-after-free, data corruption, stack-smashing, and threats to control-flow integrity.	Attacks include resource isolation attacks (resource exhaustion/DoS), file system exploitation (inode exhaustion, entropy depletion), network bandwidth monopolization, and multithreading amplification.	Buffer overflows, memory leaks, format string vulnerabilities, integer overflows, cross-site scripting (XSS), denial of service (DoS), code injection, and assertion failures.
A malicious model where the attackerExtracts secret data (like private keys) remotely through sandboxed environments.	A malicious model focusing on protecting users from malicious modules and unauthorized system access.	A malicious model where attackers exhaust the host's resources to interfere with other container instances.	Not Defined
Cache attacks are considered highly dangerous because they have a high success-rate in extracting secret data by sharing CPU cache levels.	Memory corruption vulnerabilities (buffer overflows, use-after-free) in the linear memory are considered severe because they remain possible despite Wasm's other safety features.	Not Defined	XSS, DoS, and code injection
Vulnerabilities can be found in CPU caches (L1, L2, LLC), SharedArrayBuffer, and WebAssembly memory in shared mode.	Components include linear memory (mutable/addressable) and the import/export mechanism (if not strictly controlled).	Components include WASIWASIX interfaces, linear memory, JIT compilers (e.g., Cranelift), and the runtime implementation (especially in Rust)	Wasm binaries, linear memory (heap), and the unmanaged data stack.
In browsers, the cache occupancy channel can be exploited through JavaScript and Wasm to perform website fingerprinting.	In embedded systems, the lack of hardware memory management units makes the software-based linear memory isolation critical.	In cloud and edge computing, insufficient restrictions in the WASIWASIX interfaces allow Wasm instances to break container isolation.	Not Defined
Not Defined	Buffer overflows, malicious input injection, and privilege escalation.	Resource exhaustion attacks through Wasm runtime interfaces	Not Defined
Not Defined	Not Defined	Denial-of-service through bypassing isolation boundaries to exhaust host resources (CPU, I/O, networking)	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Remote browser-based cache attacks and keystroke timing attacks in sandboxed environments.	Web browser attacks such as manipulation of sensitive user data	Browser-based side-channel attacks (e.g., port contention)	Not Defined
The paper proposes Lurking Eyes, a detection-based approach that analyzes JavaScript/Wasm code traces to find features indicating the occurrence of side-channel attacks.	The paper discusses sandboxing, static validation, type safety, and the use of a shadow stack to ensure safety.	The paper uses a static analysis framework (including Rust-oriented IR extraction and indirect call analysis) to identify attack surfaces and proposes exploitation strategies.	Owl, a toolkit for performant parallel symbolic execution using a monadic interpreter
For web browsing, the paper evaluates detection of malicious code on webpages to alert users without disabling useful JS/Wasm features.	For embedded systems, the paper evaluates Hardware Abstraction Layers (HALs) and WASI (WebAssembly System Interface) to standardize hardware access.	Evaluated mitigations include XFS file system quotas, Cgroups (CPU/bandwidth limits), and syscall interception (e.g., eBPF)	Not Defined
Gaps include the lack of effective software methods that don't degrade performance and the limitations of feature-stripping models like JavaScript Zero.	Gaps include the lack of native support for hardware resources (GPIOs, peripherals) in WASI and the fragmentation of HAL implementations.	Gaps include insufficient resource isolation in popular runtimes (Wasmtime, Wasmer) and a lack of systematic measurement of the Wasm attack surface.	Scaling symbolic execution to large codebases and the lack of parallel state space exploration in prior tools.
The paper evaluates Lurking Eyes, which uses feature searching and scoring to identify attacks like Spectre and memory deduplication.	Module validation (static verification of conformance to the specification) is evaluated.	Techniques include differential testing (WADIFF), Wasm runtime bug detection, and behavioral analysis using frequency-based or learning-based defenses.	Symbolic execution using SMT solvers (e.g., Z3, Colibri2).
Not Defined	Module validation and static type-checking.	Differential testing frameworks (e.g., WADIFF)	Symbolic execution and verification.
Not Defined	Shadow stack protection, Data Execution Prevention (DEP), and sandboxing.	Linear memory constraints and domain-isolated linear memory (PKUWA)	A lazy memory model resembling a copy-on-write strategy to manage state duplication efficiently
Not Defined	Not Defined	The Swivel compiler framework (hardening against Spectre) and analysis of inline assembly optimizations.	Not Defined
Employing machine learning to improve detection accuracy and utilizing JavaScript events to reduce error probability.	It identifies the need for lightweight, modular runtime environments specifically optimized for embedded workloads.	It suggests using eBPF for fine-grained monitoring and identifying actual resource consumers in the kernel.	Implementing a concolic monad, more efficient exploration strategies like A*, handling path explosion in loops using constrained Horn clauses, and integrating the WasmGC proposal
Lurking Eyes	WASI	WADIFF, PKUWA	Owl
First-hand research paper	Review (survey) paper.	First-hand research paper	First-hand research paper.
Finalized	Finalized	Finalized	Finalized

50	51	52	53
A Study of Timing Side-Channel Attacks and Countermeasures on JavaScript and WebAssembly	WASP: Stack protection for WebAssembly	An Empirical Study of Real-World WebAssembly Binaries	WasDom: An Efficient Write Protection for Wasm/JITed Code With ARM Domain
10.22042/secure.2021.263565.599	10.1016/j.sysarc.2025.103666	10.1145/3442381.3450138	10.1109/ACCESS.2025.3537756
2021	2026	2021	2025
Iran	United Kingdom	Germany	Republic of Korea
Mohammad Erfan Mazaheri, Siavash Bayat Sarmadi, and Farhad Taheri Ardakani	Ewan Massey and Pierre Olivier	Aaron Hilbig, Daniel Lehmann, and Michael Pradel	Suhyeon Song, Chaewon Shin, and Donghyun Kwon
Journal: ISeCure: The ISC International Journal of Information Security	Journal: Journal of Systems Architecture	Conference: Proceedings of the Web Conference 2021 (WWW '21)	Journal: IEEE Access
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Remote implementation of side-channel attacks in web browsers (client-side) and servers	Web browsers, JavaScript runtimes (Node.js), standalone interpreters, edge computing, IoT, cloud computing, and serverless computing	Client-side web applications, Node.js, browser extensions, standalone runtimes, cryptojacking, smart contracts (EOSIO), games, visualization, media players, and natural language processing	Web applications in major browsers, mobile environments, embedded devices, and tiny IoT systems
Timing side-channel attacks, cache attacks (Prime+Probe, Flush+Reload), memory/page deduplication attacks, Rowhammer (fault attacks), keystroke attacks, Spectre, shared event loop attacks, and GPU-based attacks	Stack smashing (stack-based buffer overflows), canary leak attacks, linear write overflows, cross-site scripting (XSS), sensitive data leaks, and arbitrary code execution	Memory vulnerabilities propagated from source languages (buffer overflows), cryptojacking/cryptomining, malicious crypto-stealing, heap metadata corruption (unsafe allocators), XSS (via DOM interaction), and speculative execution attacks	JITed code tampering/manipulation, code injection, memory vulnerabilities, multi-threading core-switch exploits, and unauthorized metadata alterations
Malicious (implementing "spy programs" remotely on victim systems to steal secret keys or track user behavior)	Malicious (attacker exploits a linear buffer overflow to gain read/write primitives over linear memory)	Malicious (attackers exploiting unmanaged stacks or unsafe memory allocators statically linked into binaries)	Malicious (attacker infiltrates the runtime to modify or overwrite the unprotected code space where JIT-compiled code is stored)
Spectre is described as a "serious security bug," and key extraction attacks (breaking RSA, ElGamal, ECDH) are noted for their high power	Attacks leading to arbitrary code execution (e.g., via indirect function calls or eval-like functions) or sensitive data tampering/leaks	Vulnerabilities that propagate from memory-unsafe languages like C and C++, which comprise two-thirds of the studied ecosystem	Code tampering that allows an attacker to inject and execute malicious behavior within the normal execution path
WebAssembly memory, shared memory features (SharedArrayBuffer), and event loops	Linear memory, specifically the unmanaged data stack used for non-scalar data	The unmanaged stack in linear memory, custom memory allocators, and imported security-critical APIs (e.g., eval, DOM-manipulation functions)	Just-In-Time (JIT) compiler, the "code space" memory region, jump tables, and the Domain Access Control Register (DACR)
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
cross-VM cache attacks	Mentions buffer overflow detection research for virtualized clouds	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	EOSIO smart contracts were found using potentially vulnerable non-default allocators	Not Defined
Remote timing attacks on modern browsers to track browsing history or extract keys	C and C++ programs compiled to WebAssembly	Browser-based cryptojacking (found to be significantly declining)	Not Defined
"Lurking Eyes," a detection-based countermeasure that searches JavaScript and Wasm code for attack-related features	WASP (WebAssembly Stack Protector), implementing traditional and dynamically randomized stack canaries in the compiler	Large-scale empirical study using automated program analysis (static analysis of imports/names/allocators), manual inspection, and VirusTotal.	WasDom, an efficient write protection mechanism for JITed code using ARM domains and randomized domain allocation
Software-level detection for client-side web platforms	Stack Smashing Protection (SSP) for Wasm programs	Not Defined	Enforcing the W*X (Write XOR Execute) policy in mobile/embedded Wasm environments.
The lack of effective software countermeasures that do not reduce performance.	The absence of common mitigations like stack protectors and Data Execution Prevention (DEP) in existing Wasm toolchains	The need for mitigations against memory vulnerabilities, automated security analysis tools, and techniques for reverse engineering minified Wasm code	Existing hardware protections like Intel MPK are inapplicable for mobile devices, and current methods lack fine-grained control for parallel compilation.
Lurking Eyes (checking for patterns like large array allocations, timing instructions, and Web Workers)	Stack canaries (static and randomized per function call)	Taint tracking, VirusTotal for cryptominers, and static analysis of security-critical imports	Software Fault Isolation (SFI) and fine-grained Control Flow Integrity (CFI) on JIT compilers
Malicious code detection using feature extraction	Binary-level stack canaries, ASLR, and guard page	Future work on decompiling and reverse engineering minified binaries	In-process isolation using ARM Domain
Hardware-level changes to address mapping, OS-level disabling of memory deduplication, and software-level "JavaScript Zero" permission models	Stack canaries, shadow stacks (SafeStack), fat pointers, and memory tagging	Stack canaries and providing secure memory allocation or GC APIs via the host environment	W*X policy, a randomized domain allocation strategy, and a kernel-based Secure Driver to manage metadata.
Not Defined	Implementing stack smashing protection directly within the Emscripten/LLVM compiler	LLVM toolchain needs security mitigations to protect a large fraction of the ecosystem	CFI techniques to isolate memory write operations during JIT compilation.
Dynamic online examination of events, using machine learning to bypass attacker deception, and developing timers based on full multi-threading once available in Wasm	Extending WASP with guard pages and ASLR, integrating it into more compilers, and evaluating it on runtimes like Wasmtime and Wasmedge	Decompiling minified Wasm to help analysts, hardening binaries against exploitation, and defending against propagated vulnerabilities	Extending the principles of WasDom to other JIT-based runtimes like major JavaScript engines.
Lurking Eyes (detection method) and Eagle timer (a new accurate Wasm-based timer)	WASP	WasmBench (the study's dataset), wasmparser, WABT, and VirusTotal	WasDom
First-hand research paper.	First-hand research paper	Empirical study(First-hand research paper)	First-hand research paper
Finalized	Finalized	Finalized	Finalized

54	55	56	57
NOS: A Lightweight Real-Time Cryptojacking Detection System	Performant Bounds Checking for 64-Bit WebAssembly	Aerogel: Lightweight Access Control Framework for WebAssembly-Based Bare-Metal IoT Devices	Wasabi: A Framework for Dynamically Analyzing WebAssembly
https://dx.doi.org/10.14722/ndss.2021.24444	https://doi.org/10.1145/3689490.3690400	https://doi.org/10.1145/3453142.3491282	https://doi.org/10.1145/3297858.3304068
2021	2024	2021	2019
USA	Germany	USA	Germany
Faraz Naseem, Ahmet Aris, Leonardo Babun, Ege Tekiner, and A. Selcuk Uluagac	Lukas Döllerer and Alexis Engelle	Renju Liu, Luis Garcia, and Mani Srivastava	Daniel Lehmann and Michael Pradel
Conference: Network and Distributed Systems Security (NDSS) Symposium	Conference: ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages (VML)	Conference: ACM/IEEE Symposium on Edge Computing (SEC)	Conference: 2019 Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
High-performing web applications running on browsers, cryptocurrency mining (specifically Monero), streaming services (e.g., YouTube), and various organisational websites.	Web browsers, embedded IoT devices, blockchain (smart contracts), databases (user-defined functions and operators), and standalone runtimes.	Bare-metal IoT devices, smart healthcare, smart city lighting, transportation, Unmanned Aerial Vehicles (UAVs), smart home security cameras (Blink XT2), and Nest Protect	web browsers, games, cryptography, machine learning, and medical applications.
Cryptojacking (drive-by-mining or coinjacking), CPU throttling and obfuscation of strings and function names to evade detection, and exploitation of vulnerable network devices like routers.	Out-of-bounds memory accesses, and vulnerabilities resulting from memory-unsafe languages like C and C++.	Unauthorized access to peripheral sensors and actuators, improper energy usage leading to battery draining, buffer-overflow, and Denial-of-Service (DoS) via resource exhaustion	Cryptomining (unauthorised use of computing resources) and illegal information flows (detected via taint analysis)
Malicious (attacker injects mining scripts without user consent via compromised devices, services, or websites)	Malicious (execution of "untrusted code")	Malicious (Aerogel does not trust any application code; apps are suspected of attempting unauthorized peripheral access or resource draining)	Malicious (implied by the focus on monitoring "untrusted web code," detecting malicious mining algorithms, and performing taint analysis to detect illegal flows)
Cryptojacking is highlighted as the major application among malicious Wasm use cases.	Out-of-bounds access is the primary focus as Wasm runtimes must guarantee it results in a trap.	Attacks draining critical device resources (battery and memory) or accessing sensors/actuators without authorization	Not Defined
Wasm binaries (bytecode) and Wasm modules.	Linear memory (out-of-bounds access), and address calculation (potential overflow in 65-bit addresses).	Wasm runtime (requires instrumentation for access control), linear memory (potential for overlapping with peripheral I/O addresses), and exported native functions.	Wasm binary (bytecode), linear memory, and control-flow structures (e.g., branch instructions and indirect calls)
Browser-based Web: Wasm binaries are exploited to execute cryptocurrency mining payloads (e.g., <code>cryptonight_hash</code>) covertly within user browsers.	Not Defined	Bare-metal IoT: Due to the lack of an MMU, all software models share the same memory space, allowing a malicious application to potentially perform memory attacks like buffer overflows to gain unauthorized peripheral access.	Not Defined
Exploitation of vulnerable routers and access points to inject cryptojacking scripts into web traffic.	Not Defined	Battery and memory resource exhaustion, and unauthorized sensing or actuation on bare-metal hardware.	Not Defined
Not Defined	Not Defined	Serverless function execution on the edge.	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Browser-based cryptojacking via malicious browser extensions and compromised third-party services.	Not Defined	UAV flight control system safety and security.	Browser-based cryptomining and illegal data flows (taint analysis)
MINOS, an image-based classification system using a Convolutional Neural Network (CNN) to classify grayscale image representations of Wasm binaries as malicious or benign.	Virtual memory techniques including two-level guard pages and macro guard regions to efficiently detect invalid memory accesses without heavy software overhead.	Aerogel, a runtime framework that instruments the Wasm execution environment to enforce fine-grained access control policies for peripherals, processor energy, and memory usage.	Wasabi, a general-purpose framework for heavyweight dynamic analysis based on binary instrumentation, which inserts analysis hooks into Wasm binaries.
Not Defined	Not Defined	Bare-metal IoT: Use of counting semaphores for peripheral access and strict energy allocations per application.	Not Defined
Obfuscation in cryptojacking is an unexplored area; future advanced obfuscation techniques or adversarial machine learning attacks could potentially affect current detection schemes.	The standardized behavior for 65-bit address wrapping in the memory64 proposal is noted as potentially impractical and a possible area for mitigation refinement.	Providing access control for non-memory-mapped I/Os (e.g., interrupt-based sensors) and adapting access control for dynamically streamed workloads from third parties.	The lack of general-purpose dynamic analysis frameworks for WebAssembly. Technical challenges addressed include tracing type-polymorphic instructions and resolving relative branch targets statically.
Blacklists, antivirus signature matching, dynamic feature analysis (CPU, memory, network, instruction counts), and the proposed grayscale image classification (MINOS).	Signal-based trap detection (SIGSEGV/SIGBUS handlers) and software-based bounds checks.	Runtime checkers for memory usage, processor energy consumption, and peripheral permission checks.	Instruction profiling (e.g., for cryptominers), basic block profiling, instruction/branch coverage, call graph extraction, memory access tracing, and dynamic taint analysis.
Conversion of binaries to images for instantaneous malicious activity classification.	Use of C++ exceptions or <code>siglongjmp</code> to stop execution upon a trap.	Sandboxing to isolate tenant applications from each other and from native code.	Ahead-of-time binary instrumentation to monitor execution.
Not Defined	Guard pages, micro guard pages, and macro guard regions.	Peripheral memory isolation by resolving I/O address collisions during memory allocation.	Memory shadowing using a separate JavaScript heap to track meta-information without interfering with the original WebAssembly heap.
Not Defined	Not Defined	Not Defined	Not Defined
Developing a Chrome extension that utilizes the MINOS framework to automatically close malicious webpages and generalizing the technique to detect other Wasm-based malware families.	Modification of source code and compiler toolchains to execute a broader subset of performance benchmarks.	Exploring autonomous policy enforcement and optimizing the impact of executing applications during over-the-air installation.	Cross-language dynamic analysis to simultaneously analyze web applications running both JavaScript and WebAssembly code.
MINOS	Not Defined	Aerogel, Wasm Micro-runtime (WAMR), and AccTee.	Wasabi. Other discussed tools include Pin, Valgrind, DISL, RoadRunner, and Jalangi (for other platforms), as well as Wasm-specific tools like TaintAssembly and SEISMIC.
First-hand research paper	First-hand research paper	First-hand research paper.	First-hand research paper
Finalized	Finalized	Finalized	Finalized

58	59	60	61
Concolic Execution for WebAssembly	Formally Verified Cryptographic Web Applications in WebAssembly	A Runtime Design of WebAssembly Container for Embedded Real-Time Operating Systems	Edgedancer: Secure Mobile WebAssembly Services on the Edge
10.4230/LIPIcs.ECOOP.2022.11	10.1109/SP.2019.00064	https://doi.org/10.1007/978-3-031-91853-7_17	10.1145/3434770.3459731
2022	2019	2024	2021
Portugal	USA (Microsoft Research) and France (Inria)	China	Germany
Filipe Marques, José Frago Santos, Nuno Santos, and Pedro Adão	Jonathan Protzenko, Benjamin Beurdouche, Denis Merigoux, and Karthikeyan Bhargavan	Wenzhuo Wang and Jian Li	Manuel Nieke, Lennart Almsedt, and Rüdiger Kapitza
Conference: European Conference on Object-Oriented Programming (ECOOP)	Conference: 2019 IEEE Symposium on Security and Privacy	Conference: ACIBD 2024 (LNICST 616)	Conference: 4th International Workshop on Edge Systems, Analytics and Networking (EdgeSys 21)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Client-side web applications, server-side runtimes (e.g., Node.js), IoT platforms, edge computing, and smart contracts.	Web applications requiring high-assurance cryptographic libraries (WHACL*) and secure messaging protocols (LibSignal*) for platforms like WhatsApp, Skype, and Signal	Embedded systems and Real-Time Operating Systems (RTOS) for resource-constrained edge devices such as UAVs or airborne systems.	Edge computing, mobile edge computing, and latency-sensitive services like cloud gaming, augmented reality, and connected cars
Buffer overflows, format string bugs, use-after-free errors, cross-site scripting (XSS), code injection, and malicious use of Wasm for web keyloggers or crypto-miners. A new heap-overflow bug was also detected.	The paper discusses timing side-channel attacks, network attacks, and memory-safety bugs such as HeartBleed.	Not Defined	Malicious host attacks on application integrity/confidentiality and malicious applications attacking hosts.
Malicious (targeting vulnerabilities that "cyber attackers" exploit to launch attacks on the Web)	Malicious (considering compromised or coerced servers and nation-state level access)	Not Defined	Malicious (assuming edge providers may falsify machine properties and that host software may be compromised)
vulnerabilities enabling code injection or XSS are emphasized as security-critical.	Not Defined	Not Defined	Not Defined
Wasm binaries (which can transpose unsafe language bugs), linear memory (specifically heap operations), and stack operations.	The interaction with the host environment via imports.	Not Defined	Linear memory and executed code, which a host can freely access or change without TEE protection
Web (Client-side): Vulnerabilities like buffer overflows transposed to Wasm enable XSS attacks. Web (Server-side): Targeting server-side vulnerabilities (e.g., Node.js) for code injection attacks.	In messaging applications, vulnerabilities can exist in the Wasm-to-JavaScript FFI (Foreign Function Interface) when passing session state or message components.	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Attacks by malicious hosts against the integrity and confidentiality of migrating edge services
Not Defined	Not Defined	Not Defined	Not Defined
WANA and Manticores are tools targeting smart contract vulnerabilities.	Not Defined	Not Defined	Not Defined
Malicious web keyloggers and crypto-miners.	In secure messaging, timing attacks on non-constant-time cryptographic implementations were investigated.	Not Defined	Not Defined
WASP, a concolic execution engine that instruments the Wasm reference interpreter to combine concrete and symbolic execution for path exploration and bug detection.	Formal verification of source code (Low*) using the F* language and verified compilation to WebAssembly.	Implementation of a lightweight container runtime based on Wasm to provide isolated execution environments.	Combining Wasm sandboxing with Trusted Execution Environments (TEEs) like Intel SGX
Not Defined	The use of high-assurance cryptographic libraries (WHACL*) and verified protocol cores (LibSignal*) to ensure memory safety, functional correctness, and secret independence.	Not Defined	Secure migration of execution contexts, transparent encryption of memory, and integrity checks
Noticeable lack of tool support for directly analyzing stand-alone Wasm binaries and the need for more precise information flow analysis.	The need for a mechanical proof of the translation from Low* to WebAssembly.	Many RTOSs lack the native resource management functions (like Linux Namespaces and Cgroups) required for traditional containerization.	Performance drops when the application state exceeds the Enclave Page Cache (EPC) limit
Concolic testing (WASP), static analysis (CT-Wasm, Wassail), dynamic taint tracking (TaintAssembly), and symbolic execution (Manticores, WANA)	A secret independence checker that analyzes generated WebAssembly code to ensure timing side-channel resistance.	Not Defined	Remote attestation to verify that code is running within a TEE.
Use of bounded verification proofs and test suite generation.	Verified compilation from formally proven source languages.	Not Defined	Integrity checks and encryption via TEE
Wasm Logic for establishing the safety of heap operations.	Leverages Wasm's inherent page protection mechanisms and formal proofs of spatial and temporal safety in Low*.	Not Defined	Wasm sandboxing (address-based memory access limited to linear memory)
Not Defined	Not Defined	Not Defined	Not Defined
Improving the precision of information flow analysis using self-composition, combining concolic execution with taint tracking, and increasing line coverage for industry-grade libraries.	The need for mechanical proofs of translation and integration with the CT-Wasm proposal.	Not Defined	Incorporating live-migration techniques, particularly post-copy migration
WASP, WASP-C, KLEE, VeriFuzz, Gillian-C, CT-Wasm, Wassail, and TaintAssembly.	KreMLin (a verified compiler) and the secret independence checker.	Not Defined	Edgedancer platform
First-hand research paper.	First-hand research paper	First-hand research paper	First-hand
Finalized	Finalized	Finalized	Finalized

62	63	64	65
The Security Risk of Lacking Compiler Protection in WebAssembly	Detecting WebAssembly-based Cryptocurrency Mining	ASMaint: Portable Multi-language Taint Analysis for the Web	Oron: Towards a Dynamic Analysis Instrumentation Platform for AssemblyScript
10.1109/QRSS4544.2021.00024	10.1145/3319535.3363287	10.1145/3759426.3760982	https://doi.org/10.1145/3464432.3464780
2021	2019	2025	2021
Belgium and New Zealand	Hong Kong and China	Belgium	Belgium
Quentin St'evenart, Coen De Roover, Mohammad Ghafari	Weikang Bian, Wei Meng, and Yi Wang	Abel Stuker, Aaron Munsters, Angel Luis Scull Pupo, Laurent Christophe, and Elisa Gonzalez Boix	Aaron Munsters, Angel Luis Scull Pupo, Jim Bauwens, and Elisa Gonzalez Boix
International Conference on Software Quality, Reliability and Security (QRS)	Conference; 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19)	Conference; 22nd ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes	Conference
Reviewer 2	Reviewer 1	Reviewer 1	Reviewer 1
Cross-platform desktop applications, IoT systems, and web browsers	Web browser environments for computation-intensive tasks like cryptocurrency mining	Web applications integrating JavaScript with languages like C, C++, or Rust, as well as server runtimes and embedded devices	Client-side web applications, performance analysis, and concurrency analysis
Stack-based buffer overflows,	In-browser cryptojacking, where attackers abuse computation resources for mining without consent	Confidentiality and integrity vulnerabilities (e.g., cross-language data leaks) and supply chain attacks via third-party Wasm dependencies.	The paper mentions information flow control and instances where malicious code might observe altered application behaviour to avoid discovery during security analysis.
Assumes the security model for Wasm and not specifically defining a threat model.	Malicious (remote attacker controlling a website visited by a victim)	Malicious (considering the implicit trust introduced by installing npm or third-party Wasm packages)	Malicious
Stack smashing is severe because it can occur silently in Wasm (allowing the program to continue execution), unlike in native code where it crashes	Not Defined	Not Defined	Not Defined
Wasm compilers (e.g., Clang, GCC) that lack mitigations like stack canaries	Wasm instruction execution traces which can be leveraged to profile and identify mining behavior	Shared linear memory, global variables, tables, and interlanguage function calls	AssemblyScript source code and compiled WebAssembly binaries
Not defined	Not Defined	Not Defined	Not Defined
Not defined	Not Defined	Not Defined	Not Defined
Not defined	Not Defined	Not Defined	Not Defined
Not defined	Not Defined	Not Defined	Not Defined
Not defined	Not Defined	Not Defined	Not Defined
Not defined	Web Browsing: Illicit mining using Wasm's efficiency to bypass traditional JavaScript-based detection	Web Applications: vulnerabilities arising from data flows between JavaScript and Wasm boundaries	Not Defined
Comparative empirical investigation of execution outcomes (divergences) between native x86 and Wasm	Modeling and detecting mining behavior using subsequences of Wasm instruction execution traces as signatures	Dynamic taint analysis using source code instrumentation and a novel analysis orchestrator	Oron, a source code instrumentation platform that inserts traps into AssemblyScript code to perform dynamic analysis
Not defined	A browser-based methodology to identify and detect cryptojacking by analyzing temporal mining speeds and instruction distribution	Precise tracking of taint information across language boundaries to mitigate integrity and confidentiality risks	Source code instrumentation for portable dynamic analysis in web environments
research is needed to correct these divergences that found in this paper, in order to enable the secure deployment of WebAssembly binaries compiled from C programs.	The need for obfuscation-robust detection, as attackers may use code obfuscation, throttling, or custom servers	Performance overhead vs. analysis precision, and the current lack of support for Wasm tables	The performance overhead introduced by source code instrumentation
Detection of divergent behavior (inconsistencies) between native and Wasm execution outcomes	Instruction sequence profiling and cosine similarity scoring	Dynamic taint analysis for uncovering vulnerabilities in multi-language applications	Dynamic analysis through intercepting program operations (function calls, property access, etc.) via traps
Identified the absence of stack canaries in decompiled Wasm binaries	Not Defined	Source code instrumentation to weave analysis directly into the module	Not Defined
Discussion on how Wasm's implicit return addresses only diminish, but don't eliminate, stack smashing risks	Not Defined	Byte-level taint tracking within shared linear memory	The paper discusses a global arguments buffer to handle runtime values that are unknown at compile time
Investigated for failing to introduce native-level protection mechanisms	Not Defined	Not Defined	Not Defined
Researching divergent behavior in other source languages and vulnerability types beyond buffer overflows	improving the methodology against Wasm obfuscation attacks	reducing performance overhead and adding support for Wasm tables.	Conducting benchmarks to identify expensive operations and exploring more efficient strategies for handling function call arguments to reduce overhead
Not defined	Not Defined	JASMaint	Oron, Linvall, Aran, and NodeProf
first-hand research	First-hand	First-hand	First-hand research paper
Finalized	Finalized	Finalized	Finalized

66	67	68	69
Evaluating Legacy and Modern Cryptography on the Web: RSA, Hybrid AES and Ed25519 in Wasm and JavaScript	Multi-modal Learning for WebAssembly Reverse Engineering	SeeWasm: An Efficient and Fully-Functional Symbolic Execution Engine for WebAssembly Binaries	Method for Detecting Anomalies in WebAssembly Using Category
10.12720/jcm.20.6.681-693	https://doi.org/10.1145/3650212.3652141	https://doi.org/10.1145/3650212.3685300	https://doi.org/10.1007/s42979-025-04599-6
2025	2024	2024	2026
Bosnia and Herzegovina and Germany	USA	China	Germany and Brazil
Imad Buljić, Esad Kadušić, Nataša Žvič, and Elmin Marevac	Hanshan Huang and Jishen Zhao	Ningyu He, Zhehao Zhao, Hanqin Guan, Jikai Wang, Shuo Peng, Ding Li, Haoyu Wang, Xiangqun Chen, and Yao Guo	Tiago Heinrich, Newton Carlos Will, Rafael Obelheiro, and Carlos Maziero
Journal of Communications	Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis	Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis	Journal
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Web applications, secure communication (TLS), blockchain (consensus mechanisms), IoT endpoints, and mobile devices	Web browsers, serverless computing, edge computing, IoT, cryptography, and machine learning.	Web browsers, standalone runtimes, edge-side devices (SQLite, Nghttp), blockchain (EOSIO smart contracts), and SGX applications	Web services, web browsers, containers, IoT, and mobile devices
Bleichenbacher's padding oracle attack, Spectre (speculative execution), and type-confusion exploits in JIT engines	Malicious WebAssembly modules distributed through third-party services.	0-day vulnerabilities, security threats in source code (C, Go, SGX), and sensitive data leakage	Cryptojacking, obfuscation of malicious software, and intrusions in web services, Potentially Unwanted Applications (PUAs).
Malicious	Malicious (focus on auditing potentially malicious modules)	Malicious (attackers exploiting vulnerabilities to obtain primitives and affect hosting environments)	Malicious (detecting threats that compromise system security)
Padding-oracle attacks (due to plaintext recovery) and speculative execution channels like Spectre that leak secrets	Not Defined	0-day vulnerabilities and cross-boundary pointer vulnerabilities in SGX applications	Interactions that offer direct system control or resource manipulation (e.g., A-level risk calls like path_unlink_file)
Linear memory (leaks via Spectre), host-function imports (bypassing same-origin policy), and sandboxed environments	WebAssembly bytecode/modules and numeric data types (which conceal precise type information)	Linear memory, stack, control flow (e.g., br_table, call_indirect), and imported functions.	WASI interface/calls, sandbox environment, WebAssembly binary, and the compiler
Resource-constrained IoT/mobile devices are vulnerable to energy-inefficient implementations	Not Defined	Smart contracts (EOSIO) have vulnerabilities that symbolic executors can identify	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Investigating potentially malicious modules distributed without source code	Not Defined	Not Defined
Benchmarking and comparative performance analysis to guide the selection of secure and efficient algorithm-implementation combinations	WasmRev, a multi-modal pre-trained language model designed for reverse engineering tasks like type recovery, function purpose identification, and summarization	SeeWasm, an efficient and fully-functional symbolic execution engine	Dynamic intrusion detection by classifying and categorising WASI calls generated during runtime using machine learning
Transitioning to OAEP/PSS for RSA and adopting Ed25519; implementing strict CSP headers with COOP/COEP for Wasm sandboxes	Providing programmers with high-level abstractions (e.g., precise types, summaries) to enhance code comprehension and facilitate auditing for malicious code	Sound static analysis via symbolic execution to detect vulnerabilities and generate exploitations	Online (real-time) and offline evaluation strategies
Fine-grained power profiling across domains such as blockchain validation and video encryption.	Inherent scarcity of labeled data for specific WebAssembly analysis tasks	Efficiency issues due to state forking in linear memory and limited functionality in existing executors regarding imported functions	Lack of standardised security models and limited public availability of representative malicious content/datasets
Performance measurement (latency/throughput) and statistical analysis using Welch's t-tests and Cohen's d.	Self-supervised learning with three multi-modal tasks: M3LM (Masked Language Model), SSI (Similar Semantics Identification), and RII (Reordered Instructions Identification)	Symbolic execution, instruction emulation, SMT-querying (z3 solver), and memory modeling	Anomaly detection using categorical data, sliding window approaches, and ML models like XGBoost, Decision Tree, and AdaBoost
Compiling constant-time Rust/C code to Wasm	Normalization of instructions (e.g., string literals and large constants) to handle the out-of-vocabulary problem	Behavioral constraints considered instruction emulation for soundness	Reinforcing the sandbox with Trusted Execution Environments (TEEs)
Wasm's linear memory avoids GC-induced jitter but requires LFENCE/BTI barriers to mitigate Spectre.	Not Defined	State merging based linear memory modeling algorithm to avoid overhead while ensuring soundness	Operating within a sandboxed environment to provide isolation
Compilers must explicitly emit LFENCE/BTI barriers to prevent secret leakage	Not Defined	Not Defined	Not Defined
Exploring multi-threaded Wasm for parallel crypto; conducting mobile energy profiling and direct measurement of joule-per-operation metrics.	Exploring WasmRev for function name prediction and precise decompilation	The paper identifies current limitations in other tools that SeeWasm addresses, but does not explicitly list future research directions	Investigating advantages of data categorisation in more intricate environments and adapting the methodology for Intrusion Prevention Systems (IPS) to proactively block behavior
wasm-bindgen, WebCrypto API, Docker (for isolation), and libraries like rsa, aes-gcm, and ed25519-dalek	WasmRev, SnowWhite, WASPur, WABT, and Emscripten	SeeWasm, WANA, EOSAFE, Manticores, Eunomia, SymGX, and KLEE	WasmEdge, Atmo, WASim, EOSafe, WasmView, and Wobfuscator
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

70	71	72	73
WASMBOX: A Lightweight Wasm-Based Runtime for Trustworthy Multi-Tenant Embedded Systems	WBSan: WebAssembly Bug Detection for Sanitization and Binary-Only Fuzzing	Twine: An Embedded Trusted Runtime for WebAssembly	Everything Old is New Again: Binary Security of WebAssembly
https://doi.org/10.1109/TETC.2024.3409817	https://doi.org/10.1145/3696410.3714622	DOI 10.1109/ICDE51399.2021.00025	www.usenix.org/conference/usenixsecurity20/presentation/lehmann
2025	2025	2021	2020
Italy and Belgium	China, United States	Switzerland	Germany
Luigi Coppolino; Salvatore D'Antonio; Giovanni Mazzeo; Roberto Nardone; Luigi Romano; Mathieu Schmitt	Xiao Wu, Junzhou He, Liyan Huang, Cai Fu, Weihang Wang	Jāmes Minētrey, Marcelo Pasin, Pascal Felber, Valerio Schiavoni	Daniel Lehmann, Johannes Kinder, Michael Pradel
IEEE Transactions on Emerging Topics in Computing	Conference; Proceedings of the ACM Web Conference 2025	Conference; 37th International Conference on Data Engineering (ICDE)	Conference; 29th USENIX Security Symposium
Reviewer 2	Reviewer 1	Reviewer 1	Reviewer 1
Multi-tenant embedded systems, Edge cloud, IoT, Space applications, ML	IoT, smart contracts, cloud computing, and browsers	Cloud data centres, edge computing, IoT devices, and database services (SQLite)	Browsers, server-side (Node.js), serverless cloud computing, IoT/embedded devices, and smart contracts
Sandbox escape attacks, unauthorized resource/data access, zero-day attacks using system calls, and privilege escalation	Memory bugs (Heap-buffer-overflow, Stack-buffer-overflow, Use-After-Free, Memory-Leak) and undefined behaviors (Shift-overflow, Integer-overflow, Implicit-integer-sign-change, Float-cast-overflow, Null-dereference, Float-divide-by-zero)	Attacks from malicious system administrators, compromised OSs, memory-dumping processes, binary tampering, rollback attacks, and side-channel attacks leaking metadata/usage patterns	Stack-based buffer overflow, stack overflow (unmanaged stack), heap metadata corruption, overwriting "constant" data (string literals), redirecting indirect calls, and code injection into host environment (eval/exec)
Tenants are considered untrusted/malicious and may attempt to escape the sandbox via arbitrary host calls. TEE hardware is trusted	Evaluating bug detection on malicious/problematic samples	Semi-honest	Active adversary aiming to building end-to-end exploits would be a malicious model.
Sandbox escape attacks are severe because they allow a malicious tenant to interfere with others or take over the hosting device	Not Defined	Not Defined	Not Defined
WASI interface (system call risk), linear memory (adjacent object overwrites), and the Wasm runtime	Unmanaged stack, linear memory, and specific binary instructions (anchor instructions)	WASI interface (communication layer), the file system interaction, and the memory allocator	Linear memory (unmanaged stack and heap), indirect call table, and host environment APIs
Not defined	Not Defined	Not Defined	Not Defined
Not defined	Not Defined	Not Defined	Not Defined
Not defined	Not Defined	Not Defined	Not Defined
Not defined	Not Defined	Not Defined	Not Defined
Not defined	Not Defined	Not Defined	Not Defined
Not defined	Not Defined	Database management (SQLite): reading fetched/decrypted data in memory via malicious processes or system administrators	Not Defined
Not defined	Browsers: cross-site scripting (XSS) facilitated by compiling vulnerable third-party C libraries into Wasm	Database management (SQLite): reading fetched/decrypted data in memory via malicious processes or system administrators	Browsers (XSS via libpng buffer overflow); Node.js (Remote Code Execution via heap overflow); Stand-alone VM (Arbitrary file write via overwriting filename string literals)
Sandboxing with the Wasm runtime, a patched WASI interface, and an anomaly-based detection monitoring layer	WBSan, a binary sanitizer that uses static analysis (control and data dependency) and binary instrumentation (shadow memory and red zones) to detect bugs	Twine, a trusted runtime nested within an Intel SGX enclave using a secure software sandbox and an encrypted file system (IPFS)	Systematization of attack primitives and empirical risk assessment on real-world binaries to identify missing standard mitigations
Use of Arm TrustZone (TEE) for further isolating tenants and integrity verification for Wasm modules in Edge computing	Not Defined	Use of Trusted Execution Environments (TEEs), authenticated encryption (AES-GCM/AES-CCM) for persisted data, and remote attestation	Not Defined
Application compilation challenges due to current limitations in Wasm compilers	Difficulty in extracting legal memory bounds from binaries due to information loss during compilation and the unique memory structure of Wasm	Lack of protection against malicious file deletion/swapping, rollback attacks, and metadata leakage (side-channels) in current protected file systems.	WebAssembly re-enables several formerly defeated attacks (like stack buffer overflows) because it lacks modern mitigations found in native binaries.
Real-time behavioral analysis of system calls using filtering/detection profiles and SIEM systems	WBSan (binary sanitization), Wasmalt (static scan), fuzzm-canary (canary instrumentation), ASAN, and UBSAN (source-level sanitizers)	Remote attestation to establish trust and cryptographic hashes for EPC (enclave page cache) pages to detect modifications from outside	Static analysis to measure unmanaged stack usage and CFI (control-flow integrity) equivalence classes. Multiple linear memories to separate stack/heap/data; reference types for fine-grained table protection; MS-Wasm segments for memory safety.
Measured dynamically to produce attestation statements	Binary rewriting and static/dynamic instrumentation techniques	Delivery via secure channels and loading into reserved enclave memory to ensure confidentiality and prevent tampering.	Compiler-level mitigations: FORTIFY_SOURCE, stack canaries, CFI defenses, and safe unlinking in allocators.
Region-level bounds checking for linear memory and protected call stacks	Wasm shadow memory with red zones to verify memory validity	Memory preallocation buffers to optimize performance and avoid the expensive/complex BGX memory allocator.	Identifies unmanaged stack as fundamentally unprotected in single-memory models
Not defined	Not Defined	Not Defined	Not Defined
Advancements in Wasm compiler technology to improve usability	Aims to extract more program semantic information to obtain more accurate memory boundaries from binaries	Extending the WASI layer to support encrypted communication through sockets.	Calls for further hardening of the Wasm language, its compilers, and the ecosystem to implement standard binary security features
WASMBOX	WBSan	Twine	wasmtime, WAVM, Wasabi, and Emscripten
first-hand research	First-hand research	First-hand research	First-hand research
Finalized	Finalized	Finalized	Finalized

74	75	76	77
Promise and Pitfalls of WebAssembly: Perspectives from the Industry	A Categorical Data Approach for Anomaly Detection in WebAssembly Applications.	Wasmati: An efficient static vulnerability scanner for WebAssembly	EOSAFE: Security Analysis of EOSIO Smart Contracts
https://doi.org/10.1145/3696630.3728570	DOI 10.5220/0012252800003648	https://doi.org/10.1016/j.cose.2022.102745	Not Defined
2025	2024	2022	2021
China (Hong Kong SAR, Beijing, Wuhan)	Brazil	Portugal	China, Qatar
Ningyu He, Shanglong Cao, Haoyu Wang, Yao Guo, Xiapu Luo	Tiago Heinrich, Newton C. Will, Rafael R. Obelheiro, Carlos A. Maziero	Tiago Brito, Pedro Lopes, Nuno Santos, José Fragoso Santos	Ningyu He, Ruiyi Zhang, Haoyu Wang, Lei Wu, Xiapu Luo, Yao Guo, Ting Yu, Xuxian Jiang
Conference; 33rd ACM International Conference on the Foundations of Software Engineering	Conference; 10th International Conference on Information Systems Security and Privacy (ICISSP 2024)	Computers & Security (Journal)	USENIX Security Symposium (Conference)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
3D graphic rendering, video decoding, industrial software (Photoshop, AutoCAD), Google Earth, font processing, cryptography, and game console emulation.	Web Services (WasmEdge, Atmo) and mainstream Web browsers	Browsers, server-side web applications (e.g., Node.js), IoT apps, edge computing, smart contracts, and browser extensions	Blockchain - EOSIO smart contracts
Cryptomining (malware like Iccryptomining), phishing scams (impersonating technical giants/banks), malware obfuscation, buffer/stack overflows, and abuse of imported security-related APIs.	Cryptojacking, obfuscation of malicious applications, and anomalous application behaviors deviating from normal statistical models.	Buffer overflows, format strings, use-after-free, double free, cross-site scripting (XSS), and code injections	Fake EOS, fake receipt, rollback, and missing permission check
Malicious	Malicious (adversaries exploring Wasm features for unwanted actions)	crafted malicious input can trigger vulnerabilities in affected modules	Malicious (any non-privileged account capable of interacting with deployed smart contracts)
Not Defined	Not Defined	Not Defined	Attacks targeting gambling DApps (e.g., fake EOS and fake receipt) that have resulted in millions of dollars in financial loss
Unmanaged call stack, statically linked allocators, and imported security-related APIs	The sandbox runtime, the WASI interface (system calls), and the shared-memory interaction model (exposed functions)	Wasm bytecode/binary instruction format and linear memory, which is untyped and byte-addressable, allowing exploits to affect the stack, heap, and data sections.	Wasm bytecode and the EOS Virtual Machine (EOS VM)
Not Defined	Not Defined	Not Defined	Smart contracts using EOS VM features such as global variables, floating-point operations, and advanced jump instructions (e.g., br_table)
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Fake EOS, fake receipt, rollback, and missing permission checks
Web browsing/Phishing; large-scale scams mimicking Facebook and financial service providers (banks)	Web Services; high-risk operations including full system compromise and denial-of-service (DoS) via abused WASI calls	Browser extension: buffer overflow in a Firefox extension	Not Defined
Large-scale empirical measurement study using urlscan.io, Internet Archive, and VirusTotal to characterize in-the-wild binaries and their threats	Dynamic anomaly detection using categorical data models and Machine Learning (ML) applied to interactions between the application and its runtime (WASI calls)	Static analysis based on the generation of a Code Property Graph (CPG)	Static analysis framework using a native symbolic execution engine and library emulation
Not Defined	Not Defined	Not Defined	Narrowing the scope of accepted tokens or using assertion statements to force action failure however these can often be bypassed
Lack of ecological diversity (controlled by a small part), and developers mistakenly treating Wasm as a data packaging format rather than a compute core.	Intrusion detection efficacy can be reduced in real-time (online) scenarios when only partial information is available via sliding windows	The lack of automated validation tools for reported vulnerabilities and the need to analyze interactions with external components like JavaScript	The path explosion problem in symbolic execution and the high manual effort required for binary-to-C conversion solutions
VirusTotal (malware scanning), BREWasm (binary rewriter for parsing), and heuristic templates to extract unmanaged call stack frame sizes.	ML models (XGBoost, Decision Tree, Nu-Support Vector, MLP, AdaBoost, and SGD) evaluated through online and offline strategies.	Wasmati, a tool using a domain-specific language (WQL) or other backends (Native, Neo4j, Datalog) to traverse CPGs	EOSAFE, which includes a symbolic execution engine, a customized library emulator, and heuristic-driven detectors
Not Defined	Categorizing interaction data to detect malicious binaries (offline evaluation)	Direct static analysis to root out security flaws regardless of the target application	Not Defined
Not Defined	Shared-memory interaction models for exposed functions.	Not Defined	A memory-merging algorithm to address memory overlap problems during bytecode emulation
Not Defined	Not Defined	Not Defined	Not Defined
suggests designing tools to recover binary semantics (like symbolic executors) and performing more security-related research on the unmanaged call stack.	aims to improve detection in more complex environments and explore other data types that benefit from data categorization for IDS	Extending the tool to model JavaScript calls to exposed Wasm functions and accesses to the JavaScript array buffer used as linear memory	Adopting advanced symbolic execution techniques to alleviate path explosion and covering more general security bugs like buffer overflows
Not Defined	Not Defined	Wasmati and the Wasmati Query Language (WQL)	EOSAFE
First-hand research	First-hand research	First-hand research	First-hand research
Finalized	Finalized	Finalized	Finalized

78	79	80	81
WasmCFuzz: Structure-aware Fuzzing for Wasm Compilers	WASIF: In-depth detection of vulnerabilities in Wasm smart contracts via information flows and function invocation sequences	WaLi: Control-flow-based Analysis of Wasm Smart Contracts	Binding Bugs in WebAssembly Interface Type Binding Generators
https://doi.org/10.1145/3643662.3643959	https://doi.org/10.1016/j.bcr.2025.100287	Not Defined	10.1109/ASE63991.2025.00025
2024	2025	Not Defined	2025
China, Australia	China	China	USA
Xiangwei Zhang, Junjie Wang, Xiaoning Du, Shuang Liu	Chu Chen, Xuan Wang, Bin Yu, Yumo Tian, Xiaoyu Lu, Pinghong Ren, Jin Liu	Shuo Yang, Huizhong Li, Zibin Zheng	Ethan Stanley, Eric Eide
ACM/IEEE 4th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCris) (Workshop)	Blockchain: Research and Applications (Journal)	Not Defined	ASE (IEEE/ACM International Conference on Automated Software Engineering) - Conference
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Web browsers, cloud computing, IoT, and edge computing	Blockchain (EOSIO and Ethereum 2.0) smart contracts	Blockchain-Liquid smart contracts on FISCO-BCOS	WebAssembly Component Model for interoperable software, cloud applications (wasmCloud, Spin), web app servers (NGINX Unit), editor extensions (Visual Studio Code, Zed)
Compiler vulnerabilities (e.g., flaws in JIT or Wasm compilers leading to browser escapes)	Fake EOS, fake notification, missing authorization verification, block information dependency, and rollback	Access control vulnerabilities	Crashes and silent data corruption due to errors in binding code generation; data-conversion bugs between high-level languages
Not Defined	Malicious (mentions "malicious attackers" and "attacker agent contracts" used to trigger vulnerabilities)	Malicious ("hostile attackers")	Bugs in binding code can be exploited when handling untrusted code; Malicious
Remote Code Execution (RCE) (e.g., a vulnerability in the JavaScriptCore Wasm compiler exploited to gain RCE in Safari)	Not Defined	Access control flaws	Latent code-generation bugs (silent data corruption) because they result in incorrect behavior without raising exceptions, making them very hard to detect.
Wasm compilers within browser engines (e.g., JavaScriptCore, V8, SpiderMonkey)	Wasm bytecode, multi-indexed tables (persistent data storage), and sensitive APIs (e.g., send_inline, tapos_block_num)	Wasm bytecode and FISCO-BCOS Environment Interfaces (FBEIs) such as getCaller and setStorage	WebAssembly Interface Type (WIT) binding generators (e.g., wit-bindgen), binding code (adapters), Wasm compilers (e.g., TinyGo), and runtimes.
Not Defined	Smart contracts: focuses on data dependencies and state transitions within the contract's persistent storage.	Liquid smart contracts and their interaction with FBEIs	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Cloud/Edge integration and editor extensions: incorrect data translation/encoding across language boundaries in components
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Fake EOS, fake notification, missing authorization, block info dependency, and rollback	Access control vulnerabilities	Not Defined
Browsers (vulnerabilities causing crashes or RCE)	Not Defined	Not Defined	Semantic bugs in binding code generation
Structure-aware fuzzing	Concolic fuzzing utilizing information flows (taint analysis) and function invocation sequences	Static analysis based on control flow and critical path identification, followed by simulation in a Wasm virtual machine	Random differential testing
Not Defined	Verifying token issuers/recipients, ensuring authorization via authAPIs, and using deferred actions to prevent rollback.	Implementing access control settings (e.g., address verification via if statements before sensitive operations)	Not Defined
The difficulty of producing "semi-valid" samples that bypass initial structural checks but are unique enough to probe for compiler corner cases	Inefficiency in generating invocation sequences, failure to trace sensitive data propagation (leading to false positives), and path explosion in complex contracts.	Path explosion problems in control flow graphs and a scarcity of Wasm smart contract datasets	Neglect of the correctness of the binding-code layer compared to runtimes; lack of automated program reducers for WIT
WasmCFuzz, which utilizes AFL-generated random bytes to create structurally valid Wasm formats for testing	WASIF, featuring a sequence generator, a symbolic executor with heuristic branch selection, and dependency-based taint analysis	WaLi, which uses a virtual machine to monitor stack changes, memory, and variables while tracing critical paths	Random differential testing framework (Atlas) for WIT binding generators
Not Defined	Use of concolic execution and taint tracking to precisely pinpoint vulnerabilities	Use of a critical path identification method to improve detection efficiency by narrowing the search scope	Differential testing of binaries generated with different bindings
Not Defined	Not Defined	the tool uses symbolic values to simulate stack/memory opcodes	Not Defined
Identification and reporting of unique bugs to vendors for patching	Not Defined	Not Defined	Testing binding code generators and compilers (TinyGo) using fuzzing and differential testing
Not Defined	Expanding the types of detectable vulnerabilities, optimizing symbolic execution/pruning, and broadening support to other smart contract platforms	Including more vulnerability types from Solidity, developing precise path trimming methods, and creating a more scalable Wasm virtual machine	Detecting "self-consistent" bugs, improving source program generation for all types, and applying the approach to other middleware (gRPC, SWIG)
WasmCFuzz	WASIF	WaLi	Atlas (test case generator), wit-smith, wit-bindgen, wit-bindgen-go, WAC (composition tool), Wasmtime (runtime), TinyGo (compiler)
First-hand research	First-hand research	First-hand research	First-hand research
Finalized	Finalized	Finalized	Finalized

82	83	84	85
metaSafer: A Technique to Detect Heap Metadata Corruption in WebAssembly	Scaling Instruction-Selection Verification against Authoritative ISA Semantics	Flexible Non-intrusive Dynamic Instrumentation for WebAssembly	AFL: Binary-Only WebAssembly Fuzzing with Fast Snapshots
10.1109/ACCESS.2023.3327817	10.1145/3764383	10.1145/3620666.3651338	https://doi.org/10.1145/3503921.3503924
2023	2025	2024	2021
Republic of Korea	USA	USA	Germany
Suhyeon Song, Seonghwan Park, Donghyun Kwon	Michael McLoughlin, Ashley Sheng, Chris Fallin, Bryan Parno, Fraser Brown, Alexa VanHattum	Ben L. Titzer, Elizabeth Gilbert, Bradley Wei Jie Teo, Yash Anand, Kazuyuki Takayama, Heather Miller	Keno Haßler, Dominik Maier
IEEE Access - Journal	Proceedings of the ACM on Programming Languages (OOPSLA) - Journal	Conference ACM	ROOTS'21 (Reversing and Offensive-oriented Trends Symposium) - Conference
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Image/video processing, AI graphical applications, games, video editing, machine learning, and database management (SQLite)	Web browsers, OS kernels, cloud services, and sandboxing for untrusted code execution	Web, Cloud, Edge, IoT, embedded, and industrial systems; program analysis (code coverage, execution frequency, taint tracking, fuzzing, debugging)	imaging, gaming, cryptography (libsodium), compression (brotli, lzma), audio conversion (libsndfile)
Heap memory overflow, metadata corruption (unlink exploit), arbitrary memory writes, control flow diversion, buffer overflow	Miscompilations, sandbox escapes (e.g., CVE-2023-26489), failed overflow checks (sdv bug), and errors in instruction selection that undermine isolation.	Program bugs and performance bottlenecks; risk of "intrusive" instrumentation breaking program state	Memory corruptions, vulnerabilities in memory-unsafe source code (e.g., C) translated to Wasm
Malicious	Malicious	Honest : focus is on developer tools for debugging and optimization, monitors are sandboxed and non-intrusive	Malicious
Control flow diversion and arbitrary code execution because they allow an attacker to take full control of a system	Wasm sandbox escape because it allows untrusted code to break isolation and compromise the host	Not Defined	Code execution
Linear memory (heap area and metadata), memory allocators (e.g., emmalloc)	Wasm-to-native compilers (Cranelift), instruction selection phase (SLE rules), machine code IR (Minst), and the assembler	Wasm bytecode, JIT compiler, interpreter, execution frames (state materialization), and deoptimization mechanism	Wasm binaries
Not Defined	Miscompilation bugs in instruction lowering	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Sandbox escape using instruction selection errors	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Metadata corruption via heap overflow, Browser-based high-performance applications: unlink exploit on the allocator free list.	Miscompilation of division leading to silent wrong results	Not Defined	Not Defined
Shadowing metadata from Wasm linear memory to JavaScript VM memory and conducting metadata verification (metaSafer)	Formal verification using "rule chaining," lightweight state modeling, and auto-generated ISA semantics (Arrival)	Dynamic, non-intrusive instrumentation using "probes" and FrameAccessor for state introspection	Coverage-guided fuzzing (WAFL) using AFL++ and lightweight VM snapshots
Not Defined	Not Defined	Not Defined	Not Defined
Security solutions for existing domains (x86, C/C++) are not practical for Wasm's linear memory, lack of ASLR support in Wasm	Industrial compilers are too large for clean-slate verification; existing tools struggle with deep rule graphs and stateful operations	Wasm engines lack standard APIs for dynamic instrumentation and detailed insight into executing programs	Tooling to uncover memory corruptions in Wasm binaries is limited; lack of tooling for binary-only Wasm without source access.
metaSafer library to detect metadata corruption	Arrival (instruction-selection verifier)	Dynamic instrumentation framework in the Wizard Research Engine	WAFL fuzzer for binary-only WebAssembly
Deterministic metadata protection in compiled code	Proving the absence of miscompilations in instruction lowering paths	Bytecode overwriting for local probes to minimize overhead	Fuzzing binaries to uncover bugs
Shadowing metadata to JS VM memory for spatial memory safety	Not Defined	FrameAccessor API to robustly separate monitor data from program data	Not Defined
Not Defined	Formal verification of compiler instruction selectors against authoritative ISA semantics	JIT intrinsification of probes to reduce transitions between JIT code and the engine	Verifying Wasm VMs for both compilers and runtimes using fuzzing with randomly generated binaries
Supporting standalone runtimes (without JS) and full metadata migration out of linear memory	Automating IR semantics generation from SpecTec, expanding coverage to all of Wasm core (SIMD, stack, control flow), and applying machine-readable specs to Intel x86-64.	Multi-threaded instrumentation, supporting Wasm exception handling unwind events, and using Wasm bytecode as an IR for inlining monitor code.	Improving SanitizerCoverage callback design, adding CompCov (multi-byte compare splitting), and CmpLog support for better coverage
metaSafer (APILibrary), emmalloc (allocator), emscripten (toolchain)	Arrival (verifier)	open-source Wizard Research Engine	WAFL
First-hand research	First-hand research	First-hand research	First-hand research
Finalized	Finalized	Finalized	Finalized

86	87	88	89
Eunomia: Enabling User-Specified Fine-Grained Search in Symbolically Executing WebAssembly Binaries	WebAssembly diversification for malware evasion	Wasm: A General Static Binary Rewriting Framework for WebAssembly	Hardware-Assisted Fault Isolation: Going Beyond the Limits of Software-Based Sandboxing
https://doi.org/10.1145/3597926.3598064	https://doi.org/10.1016/j.cose.2023.103296	https://doi.org/10.1007/978-3-031-44245-2_8	10.1109/MM.2024.3422977
2023	2023	2023	2024
China, USA	Sweden and Estonia	China	USA and Germany
Ningyu He, Zhehao Zhao, Jikai Wang, Yubin Hu, Shengjian Guo, Haoyu Wang, Guangtai Liang, Ding Li, Xiangqun Chen, and Yao Guo	Javier Cabrera-Artega, Martin Monperus, Tim Toady, and Benoit Baudry	Shangtong Cao, Ningyu He, Yao Guo, and Haoyu Wang	Shravan Narayan, Tal Garfinkel, Mohammadkazem Taram, Joey Rudek, Daniel Moghimi, Evan Johnson, Chris Falin, Arjo Vahdicki-Oberwagner, Michael LeMay, Ravi Sahita, Dean Tullsen, and Dean Stefan
Conference; 32nd ACM SIGSOFT	Journal: Computers & Security	Conference	Journal: IEEE Micro
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Web applications, blockchain apps, and serverless applications	Web browsers for computation-intensive activities like gaming and image processing. A primary use case is cryptojacking (malicious in-browser cryptocurrency mining)	web browsers (3D engines, scientific operations), IoT, serverless computing, edge computing, blockchain, and Web 3.0	Currently used in web browsers (Firefox, Photoshop, Zoom, Figma), serverless computing/FaaS (Fastly, Cloudflare, Akamai), microservice data planes (Istio), real-time databases (SingleStore), data streaming (RedPanda), and SaaS (Shopify)
Division by zero, slice out of range, and implementation errors in data structure functions (e.g., deque and array reverse functions).	Cryptojacking (unauthorized mining) and malware evasion via binary diversification	Buffer overflow, stack overflow leading to out-of-bound read/write, cryptojacking, and malicious mutations	Spectre attacks (speculative execution side-channels), memory safety vulnerabilities in third-party C libraries, control flow hijacking, and supply chain attacks.
Not Defined	Malicious (Attacker creates functional malware variants to evade existing detection systems)	Malicious	Malicious (Paper discusses sandboxing untrusted native binaries and resisting Spectre attacks)
Logic bombs and zero-day bugs in popular libraries are highlighted because they are hard to reach for existing symbolic execution tools and can lead to exploitable vulnerabilities once triggered.	Cryptojacking is highlighted as a lucrative and widespread form of malware because it consumes victim resources for the attacker's gain without needing to steal credentials.	Buffer overflows and stack overflows	Spectre attacks are highlighted because software-based mitigations for Wasm incur performance overheads of 62%–100%.
WebAssembly binaries, WebAssembly Interface (WASI), linear memory, and symbolic pointers.	WebAssembly binaries, custom sections (used for metadata), function bodies (instruction sequences), and linear memory layout.	Linear memory, global variables (used as stack pointers), Wasm binaries, and indirect call mechanism (table and element sections)	Wasm sandboxes (subject to Spectre), Wasm runtimes (virtual memory scaling limits), 32-bit address spaces (SFI design limits), and compilers (lack of speculative isolation).
Not Defined	Browser-based Wasm is vulnerable to malware evasion techniques where attackers diversify the binary to bypass static and dynamic detectors.	Not Defined	Browser: Wasm runtimes are vulnerable to Spectre attacks. Serverless/FaaS: Subject to scaling limitations as each Wasm instance consumes large virtual address spaces (8 GiB).
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Web browser cryptojacking (evasion of meta-antiviruses like VirusTotal and specialized detectors like MINOS).	Cryptojacking in web programs.	Web browsers (Spectre pattern history table/branch target buffer attacks and vulnerabilities in third-party libraries), JIT compilers (memory safety bugs and hijacking)
Symbolic execution with fine-grained search strategies guided by a Domain Specific Language (DSL) named Aes	Focuses on Software Diversification for evasion. It uses MCMC sampling to search for bytecode transformations that evade oracles.	Static binary rewriting using the BREWasm framework to perform binary instrumentation, software hardening, and mutation-based testing.	Hardware-assisted fault isolation (HFI), which provides minimal architecture extensions (regions) for secure in-process isolation, Spectre mitigation, and system call interposition.
Not Defined	Code canonicalization is proposed as a potential mitigation to represent diverse variants in a unified form for detectors.	Software hardening via stack canaries to detect and prevent buffer overflows.	Browser/Serverless: Replacing software-based guard regions with HFI implicit/explicit regions to eliminate overheads. JIT Hardening: Using per-thread memory permissions to enforce write-XOR-execute (WXX) without system calls
Not Defined	Current detectors (VirusTotal and MINOS) are fragile against automated bytecode-level transformations, and WebAssembly malware evasion is “nearly unexplored”.	Lack of a general-purpose static binary rewriting framework for Wasm, with challenges like complicated binary structure and strict static syntax verification.	SFI limits scaling in high-concurrency environments; Wasm/SFI cannot protect against Spectre without extreme penalties; and Wasm lack compatibility with unmodified native binaries.
Eunomia (the proposed framework), Manticores, and KLEE primitives.	VirusTotal (meta-antivirus aggregator), MINOS (image-based detection using CNN), MineSweeper (static analysis), and dynamic analysis tools like SEISMIC, MineThrottle, and MinerRay.	Binary instrumentation for runtime information collection and stack canary validation.	HFI's microarchitectural region bounds checks (comparators and masking) and near-zero-cost system call interposition.
Use of symbolic execution for automated bug finding	Proposed binary canonicalization (using e-graphs) as a robust preprocessing stage for detection.	Automatic patch generation, code obfuscation (opaque predicates, name obfuscation), and software optimization	Native sandboxing of unmodified binaries using HFI regions.
Use of a fully symbolic memory model to handle symbolic pointers efficiently	Not Defined	Insertion of stack canaries and wrapper functions around instructions to detect integer overflows.	Implicit regions for pervasive access checks and explicit regions for granular memory layout control and zero-copy sharing
Not Defined	Not Defined	Not Defined	Software-based Spectre mitigations like Swirel-SFI
Features like multi-threading (planned for Rust-to-Wasm) are not yet supported by Wasm	Suggests improving evasion fitness functions with malware program properties and using diversification for data augmentation to strengthen machine learning-based detectors.	Suggests researching flexible and general rewriting of structured control flow (e.g., blocks and loops)	Discusses the path to adoption on other architectures like RISC-V and ARM and JIT hardening for modern browsers.
Eunomia	wasm-mutate	BREWasm	HFI
First-hand research	First-hand research	First-hand research	First-hand research
Finalized	Finalized	Finalized	Finalized

90	91	92	93
Bringing WebAssembly to Resource-constrained IoT Devices for Seamless Device-Cloud Integration	WasmA: A Static WebAssembly Analysis Framework for Everyone	ZKWASM: A ZKSNARK WASM Emulator	New Kid on the Web: A Study on the Prevalence of WebAssembly in the Wild
https://doi.org/10.1145/3498361.3538922	DOI: 10.1109/SANER56733.2023.00085	https://doi.org/10.1109/TSC.2024.3422798	https://doi.org/10.1007/978-3-030-22038-9_2
2022	2023	2024	2019
China	Germany	Australia and China	Germany
Borui Li, Hongchang Fan, Yi Gao, and Wei Dong	Florian Breithfelder, Tobias Roth, Lars Baumgartner, Mira Mezini	Sinka Gao , Guoqiang Li , Member, IEEE, and Hongfei Fu	Marius Musch, Christian Wressnegger, Martin Johns, and Konrad Rieck
Conference: 20th Annual International Conference on Mobile Systems	IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)	IEEE TRANSACTIONS ON SERVICES COMPUTING	DIMVA 2019 (Conference)
Reviewer 1	Reviewer 2	Reviewer 2	Reviewer 1
IoT (city-scale sensing, wild monitoring, smart healthcare, industrial automation), Device-Cloud integration, serverless/FaaS on IoT, V2X computing, and personal area networks (PAN)	Wasm is used in web browsers for complex applications, as a backend technology on servers (e.g., Wasmer, Wasmtime), and in specialized areas like medicine, games, and IoT devices	Wasm is used for providing a zero-knowledge proof to convince a verifier that the execution result is trustworthy. This is a necessary feature for distributed serverless functions in cloud, edge, and grid computing; financial systems; private data handling; anonymous verifiable voting; cybersecurity (reporting bugs without disclosing exploits); and blockchain extensions for off-chain computation.	Wasm is used for CPU-intensive web applications such as games, custom experimental code (e.g., animations), publicly available libraries (e.g., Highentropy), and environment fingerprinting. It is also heavily used for malicious purposes, specifically cryptocurrency mining (cryptjacking) and code obfuscation.
Arbitrary jumps, illegal read/write access outside linear memory, stack overflow, buffer overflow, and malicious code modification.	Discovered attacks include malicious cryptomining, malware (often obfuscated through code and function-name obfuscation), and the exploitation of vulnerabilities in benign Wasm code	The paper addresses the threat of untrustworthy computation where a cloud provider might return invalid results or leak private data.	Attacks include cryptojacking (unauthorised mining of currencies like Monero), code obfuscation to hide malicious JavaScript (e.g., pop-unders and tracking), and the potential for stealthy malware that defeats static and dynamic JS analysis.
Malicious (Addressing security where code is transmitted over networks and third-party programs are untrusted)	Not defined specifically but the paper considers malicious contexts, specifically referring to malware authors and the execution of harmful code that might attempt to break out of a sandbox	The paper assumes an untrusted/malicious cloud service provider	Malicious
Illegal memory access and control-flow violations are considered most severe because resource-constrained IoT devices often lack memory management units or privileged levels to contain them.	Malicious cryptomining is highlighted as a significant threat, as it constitutes more than 50% of Wasm's usage on the web. It is severe because Wasm's performance boost and obfuscation capabilities make it ideal for malware authors.	Attacks that violate semantic specifications are severe in safety-critical scenarios like autonomous driving.	Code obfuscation as a severe rising threat because it allows attackers to hide exploits from existing defensive mechanisms and analysis tools.
Wasm modules (transmissible code), linear memory, stack, and control-flow building blocks (functions and nested blocks).	The stack-based nature of Wasm poses challenges for analysis; specifically, jumps and function calls depend on stack values that cannot always be determined purely statically	The virtual machine is considered a point of potential failure if it is hosted on untrusted hardware.	The memory section is used to hide malicious payloads, and import/export sections are used to interact with the host environment for malicious deeds.
IoT devices are vulnerable due to lack of hardware protection (MMU)	For web applications, the high performance and obfuscation potential of the Wasm bytecode are exploited for cryptomining	Due to the lack of native privacy for variables can lead to data leakage, which the paper addresses.	In the domain of web-based malware, the linear memory section is exploited to embed and hide HTML/JavaScript code from security filters.
Memory safety and control-flow integrity violations on constrained MCUs like ATmega128.	Not defined	Not Defined	Not Defined
Vulnerabilities in FaaS functions deployed as Wasm modules on IoT devices	Not defined	Investigates the risk of dishonest execution by service providers	Not Defined
Not Defined	Not defined	Not Defined	Not Defined
Not Defined	Not defined	Mentions Wasm as an execution environment (like ZKEVM) where computation must be verifiable	Not Defined
Not Defined	The paper specifically investigates cryptomining detection in general web usage	Not Defined	Not Defined
WAIT runtime, which uses Ahead-of-Time (AOT) compilation with two-phase sandbox checks (compile-time and runtime) for memory and CFI safety.	The authors developed WasmA, a static analysis framework that generates call, control-flow, and data-flow graphs to reveal security issues without requiring an execution environment	Generates ZKSNARK (Zero-Knowledge Succinct Non-interactive Argument of Knowledge) proofs	The paper uses a large-scale analysis involving an instrumented browser to collect Wasm code, combined with a performance profiler and n-gram bytecode similarity analysis to categorise modules and detect malicious usage patterns.
IoT: Streamed look-back compilation to handle jumps, stack restoration, and linear memory jump tables for safe access. Smart Contracts: Sandboxed execution facilitated by WAIT to minimize the attack surface compared to native code.	The paper evaluates a cryptomining detection tool built on top of WasmA using n-gram analysis of bytecode instructions	Artificially constructed circuits to enforce Wasm semantics and proof batching for long execution times.	It mentions existing research on cryptojacking countermeasures and suggests that future defense mechanisms must incorporate Wasm-specific analysis.
Lack of efficient AOT runtimes for resource-constrained (KB-level RAM) IoT devices; existing runtimes (WAMR, Wasmer) have unacceptable footprints for small MCUs.	Existing static analysis tools like Wassail and Wasmp were found to be unsound, often neglecting indirect calls or lacking essential details in their graphs	The inability of existing programs to leverage ZKSNARKs without invasive source code modifications.	A major gap exists in the analysis of intertwined code where malware is split between JavaScript and Wasm, as current systems cannot track interactions across both domains.
Compile-time safety analyzer (checking function signatures, branch indices, and immutable variables) and runtime safety analyzer (checking stack depth and memory addressing indices)	The paper proposes an n-gram based static fingerprinting approach for cryptominers using the control-flow graph (CFG)	Correctness is "detected" through the succinct verification of generated zero-knowledge proofs	Detection techniques evaluated include CPU profiling to measure Wasm execution time relative to JS and hierarchical clustering of bytecode samples to identify known malicious families.
Lightweight AOT compilation that transforms bytecode into native assembly with embedded safety checks.	The framework provides an intermediate representation of analyzed bytecode for any static client analysis	Not Defined	Manual analysis of binaries to find embedded strings or decryption logic
Run-time memory access checker and constant remapping to Flash to save RAM.	Not defined	Memory access log table with Plonkish constraints to ensure all memory reads and writes are valid.	Not Defined
Not Defined	Not defined	Not Defined	Not Defined
Automatic porting to new hardware platforms and expanding support for post-MVP Wasm features like 128-bit SIMD and threading.	The paper suggests that as code complexity increases, there is a continued need for efficient, high-precision static analysis tools that can handle large binaries ranging from several KB to MBs	Deployment scheme, adopting better parallel computing strategies, and using SNARKs for verification.	The need for sophisticated Wasm analysis in future security tools to counter evolving generations of malware obfuscation.
WAIT	The paper introduces WasmA, a static analysis framework implemented in Go	The paper introduces ZKWASM, the first ZKSNARK Wasm virtual machine	Not Defined
First-hand research	First-hand	First-hand	First-hand research paper
Finalized	Finalized	Finalized	Finalized

94	95	96	97
Empowering WebAssembly with Thin Kernel Interfaces	EST: Specification-Based Test Generation for WebAssembly	JABBERWOCK: A Tool for WebAssembly Dataset Generation and Its Application to Malicious Website Detection	Time Hybrid Approach to Combat In-Browser Cryptojacking
https://doi.org/10.1145/3689031.3717470	10.1109/ASE63991.2025.00119	10.2197/ipsjip.32.298	https://doi.org/10.3390/app13042039
2025	2025	2024	2023
USA	South Korea	Japan	Pakistan and Italy
Arjun Ramesh, Tianshu Huang, Ben L. Titzer, and Anthony Rowe	Dongjun Youn, Wonho Shin, and Sukyoung Ryu	Chika Komiya, Naoto Yanai, Kiyosuke Yamashita, and Shingo Okamura	Muhammad Haris Khan Abbasi, Subhan Ullah, Tahir Ahmad, and Attaullah Buriro
Conference	Conference	Journal of Information Processing	Applied Sciences (Journal)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
web browsers (e.g., Photoshop, Unity) and cloud and edge, IoT, automotive, and industrial systems. Virtualizing userspace applications and system daemons.	Web applications, cloud platforms, edge computing, blockchain systems, and standalone embedded runtimes.	Web browsers for various applications and investigated for malicious website detection, including phishing and cryptojacking detection.	web browsers for cryptomining (Monero), blockchain and decentralized systems.
Remote code injection/execution (RCE), control-flow redirection, filesystem escapes (e.g., via <code>proc/self/mem</code>), and signal trampoline gadgets (sigreturn exploits).	The paper investigates engine implementation bugs, including runtime crashes, internal panics, and validation mismatches, particularly in features like SIMD, Garbage Collection (GC), and Exception Handling.	Phishing, cryptojacking, and the use of Wasm for obfuscating JavaScript malware.	In-browser cryptojacking (unauthorized mining), parasitic computing, and the use of Wasm code instructions to bypass traditional JS analysis.
Malicious	Malicious	Malicious	Malicious
Remote Code Execution (RCE) is considered severe because it can break the inherent sandboxing and compromise the entire process.	Bugs in interactions between new proposals (e.g., GC and SIMD interaction causing runtime crashes) are severe because they are easy to overlook in manually written tests.	Phishing and cryptojacking are identified as major cyber crimes caused by malicious websites.	Cryptojacking
System call interfaces, memory management (e.g., mmap), and signal handling mechanisms can be exploited if not properly sandboxed.	Wasm runtime engines (e.g., V8, SpiderMonkey, Wasmtime) and their validation and execution logic.	JavaScript engines in major browsers and the interaction between JS and Wasm modules.	Browser resources (CPU time)
In edge software, the filesystem interface (<code>proc/self/mem</code>) and memory-mapping (<code>mmap/mremap</code>) are high-risk components that can be used to generate executable code segments or access host memory.	In browser-based engines, garbage collection (GC) and SIMD instruction components may contain interaction bugs leading to crashes.	In web security, obfuscate code through Wasm.	In web browsers, the CPU usage component of the host is targeted by Wasm modules for authorized computational tasks.
Safety-critical software execution and resource control.	Not Defined	Not Defined	Not Defined
Virtualization overhead	Not Defined	Not Defined	Not Defined
execution in critical physical environments.	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
RCE and CFI violations in system daemons.	Not Defined	Not Defined	Not Defined
The paper proposes thin kernel interfaces (WALI/WAZI) that enforce intra-process sandboxing, address-space translation, name-bound syscalls, and safe-point-based signal delivery.	The WEST framework uses mechanized specifications (SpecTec) to automatically generate valid, diverse test cases with runtime assertions derived directly from execution semantics.	The JABBERWOCK tool generates Wasm datasets by converting real-world JavaScript into Wasm (via Wobfuscator) and then using natural language processing (Doc2Vec) for ML-based detection.	A real-time hybrid approach combining blacklisting, signature-based statistical code analysis, and dynamic CPU usage profiling.
Proposes layering complex security policies (like WASI) over stable kernel interfaces to sandbox high-level API implementations.	Not Defined	Evaluates machine learning models (e.g., Random Forest, XGBoost, LightGBM) to detect malicious sites using Wasm vectors.	Immediate blocking of malicious URLs, static blocking of scripts communicating with mining providers, and suspending malicious code execution using <code>sleep()</code> intervals.
The lack of stable, effective system interfaces is a key limiting factor for adopting Wasm in critical legacy software and edge domains.	There is a gap in effectively testing complex feature interactions as the specification grows and engine support for different feature subsets varies.	The lack of publicly available Wasm datasets with labeled benign and malicious samples.	Existing methods are ineffective against non-Wasm service-providing scripts and advanced obfuscation techniques.
Static validation of modules (with enumerating syscall imports) and parameter type-checking inherent in Wasm.	Automated top-down and bottom-up test generation, indirect interpretation of semantics for behavior extraction, and differential testing of engines.	Machine learning-based detection using vectorized Wasm code as a feature.	Statistical code analysis for mining-related instructions and dynamic CPU speed measurement against established baselines.
Enforcing non-executable memory and implicit Control Flow Integrity (CFI)	Semantic-aware binary generation for testing.	Not Defined	Mitigates exploitation of CPU resources by calling the <code>sleep(t)</code> function on identified malicious code blocks. Fingerprinting and signature-based matching.
Bounds-checked linear address-spaces and sandboxed memory management mappings.	Enforcing static validation rules during program synthesis.	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Full-stack software verification (tower of proofs) and the expansion of syscall interfaces to support unified ISA-agnostic dynamic security policies.	Integrating the framework into the specification authoring process, exploring coverage-guided fuzzing, and applying it to other domains beyond Wasm.	Improving processing time for real-time detection, handling multiple JS files, and designing browser-based security apps.	Incorporating network behavior analysis into the dynamic analysis layer to further improve accuracy.
WALI (Linux interface), WAZI (Zephyr interface), and WebAssembly Micro Runtime (WAMR) for evaluation	WEST	JABBERWOCK and Wobfuscator	Not Defined
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

98	99	100	101
Wasm-IO: Enabling Low-Level Device Interaction in WebAssembly for Industry Automation	Vivienne: Relational Verification of Cryptographic Implementations in WebAssembly	Dynamic-Aware Pruning-Quantization Compression for Browser-Based Anomaly Detection via WebAssembly Optimization	Instruction Vulnerability Prediction for WebAssembly with Semantic Enhanced Code Property Graph
https://doi.org/10.1145/3760287	10.1109/SecDev51306.2021.00029	10.1142/S0218001426550013	https://doi.org/10.1145/3686410.3714723
2025	2021	2026	2025
Germany	Sweden	China	China
Maximilian Seidler, Alexander Krause, and Peter Ulbrich	Rodothea Myrsini Tsoupdi, Musard Balliu, and Benoit Baudry	Jun Yang and Ziming Fan	Bao Wen, Jingling Gu, Hao Han, Pengfei Yu, and Yang Liu
Journal - ACM Transactions on Embedded Computing Systems	Conference - IEEE Secure Development Conference (SecDev)	International Journal of Pattern Recognition and Artificial Intelligence (World Scientific Publishing Company)	Conference - Proceedings of the ACM Web
Reviewer 1	Reviewer 1	Reviewer 2	Reviewer 1
Industrial automation (manufacturing, rail, and infrastructure), Train Control Systems (TCS), and embedded safety-critical domains.	Cryptographic libraries (libodium, HACL*), web browsers, server-side computations, and standalone applications.	Wasm is primarily used for browser-based execution of deep learning tasks, specifically for anomaly detection. Key use cases include detecting fraudulent transactions in e-commerce, monitoring user behavior in real-time dashboards, and applications in intelligent transportation systems	Web systems, high-performance web applications (e.g., Figma, Google Earth), IoT, and embedded systems.
Malicious memory access, malicious access to device registers, monopolization of priority levels (e.g., E14), timing interference, and error propagation	Timing side-channel attacks (e.g., Lucky13), cache attacks, and speculative leaks (e.g., Spectre)	The paper does not focus on cyberattacks on WASM (like buffer overflows) but uses WASM for security to address anomalies such as fraudulent transactions and malicious user behavior. It also highlights security risks associated with transmitting sensitive data to servers, which client-side execution seeks to mitigate	Bit flips (caused by RowHammer, DVFS attacks, or clock glitching), and corruption of memory data or instruction sequences.
Untrusted third-party code and malicious Wasm services; Malicious	Malicious (an attacker seeking to leak secret information through timing variations)	malicious	malicious
Executing untrusted code in interrupt prologues is considered the most severe because a program failure in this stage results in non-recoverable device failure and invalidates system determinism.	Timing side channels because they allow attackers to recover sensitive secrets like plain text or cryptographic keys through observation of execution time.	While not explicitly ranked, fraudulent transactions are presented as a primary threat that necessitates immediate response and high accuracy to ensure application "security and reliability"	Bit flips in physical memory or registers because they can cause catastrophic system failures or errors in web applications that are not detected by the OS.
Wasm runtime (interpreters/compilers), linear memory, imported functions, call stacks, and operand stacks.	Control-flow instructions (br_if, if, br_table, call_indirect), memory operations (load, store), and the value stack.	The paper identifies the computational and memory management systems of the browser environment as critical points. It notes that static compression strategies (pruning and quantization) are vulnerable to accuracy degradation or increased latency because they cannot adapt to real-time input variations	Memory data, registers (numerical carriers), instruction sequences, stacks, opcodes, and operands.
In industrial automation, Memory Mapped I/O (MMIO) registers and interrupt lines are critical components that can be exploited.	For cryptography, secret-dependent branch conditions and secret-dependent memory indices are the primary vectors.	In the domain of browser-based deep learning, the Wasm execution engine (specifically how it interacts with CPU/GPU resources) and memory limits are the primary vulnerable components that can lead to high latency or system instability	Web applications - registers operated by instructions that propagate errors to subsequent code.
Not Defined	Not Defined	This is not the primary focus, but the paper addresses efficiency for edge devices and resource-constrained scenarios	Not Defined
Not Defined	Not Defined	Investigates the migration of tasks from server-side to client-side to mitigate the risks of transmitting sensitive data and reduce server load	RowHammer and bit flips in serverless computing environments
Corruption of register values, malicious priority elevation via system calls, and timing interference in interrupt handling.	Not Defined	Not defined	Not Defined
Not Defined	Not Defined	Not defined	Not Defined
Railway systems (Train Control Systems) where variations in sensor/actuator models require secure, isolated driver updates	Cryptography - leaks resulting from secret-dependent program control flow and memory access patterns.	Investigates e-commerce fraud and malicious real-time user behavior	Web browser code segments (e.g., flash) where corruption of jump instructions leads to high vulnerability.
Wasm-IO framework, which uses a deferred priority model (E1/4) for interrupt isolation, devicetree bindings for register identification, and interception of memory accesses to hardware.	VIVIENNE tool, utilizing Relational Symbolic Execution (RelSE) and automated loop invariant generation to verify constant-time properties.	The paper proposes DAPQ-AD (Dynamic-Aware Pruning-Quantization for Anomaly Detection), a compression algorithm optimized for Wasm. It uses input sensitivity evaluation with an importance score to adaptively adjust pruning thresholds and quantization levels in real-time	MPSEG paradigm, which uses Large Language Models (LLMs) for semantic extraction, Code Property Graphs (CPG), GNNs, and attention diffusion.
For industrial automation, the paper proposes isolated device drivers within Wasm containers to confine the impact of modifications to single modules.	Adoption of the constant-time programming discipline for cryptographic implementations.	For browser-based anomaly detection, the paper evaluates the DAPQ-AD algorithm, which integrates dynamic pruning and quantization to maintain high accuracy while reducing model size and latency	Identification of high-vulnerability positions in program positions where redundancy can be implemented to minimize costs.
Lack of formally proven interpreters for resource-constrained devices and the fact that verification remains an open question for AOT and JIT compilers.	Lack of compilers that preserve constant time and the gap between verification at the Wasm level versus machine-level code.	A major gap is that most existing compression strategies are static and fail to adapt to real-time input variations like noise or sparse patterns. Additionally, prior hardware-aware methods like AMC or NetDepth lack per-input adaptive mechanisms	Fine-grained modeling of instruction vulnerabilities caused by bit flips and the lack of hierarchical structure/semantics in traditional program representations.
Measurement-based timing analysis and characterization of Worst-Case Execution Times (WCETs) and Worst-Observed Execution Times (WOETs).	Relational Symbolic Execution (RelSE), SMT solvers, and relational invariants.	The proposed detection technique is the DAPQ-AD algorithm, which leverages SIMD instructions and multithreading within Wasm to parallelize mask generation and forward propagation for low-latency inference	Semantic enhanced code property graphs, GNN-based prediction, and LLM-driven semantic mining.
Embedding unique register labels in binaries for load-time verification against a hardware configuration.	Verification at the Wasm binary level to identify vulnerabilities introduced by the LLVM compiler.	Not defined	Predictive analysis of instruction text sequences to identify error-prone code.
Use of linear memory isolation, static type checks, and DMA configuration management to prevent unauthorized access.	Accurate modeling of memory to detect if memory indices are secret-dependent.	The paper addresses memory consumption and excessive footprint through model compression (pruning/quantization) to improve system stability and real-time responsiveness	Modeling register and memory bit flips as carriers for data transfer error propagation.
Not Defined	Investigation of how compiler optimization levels (e.g., -O3) can introduce unsafe branch instructions for the select opcode.	Not defined	WasmC-Fuzz, a structure-aware fuzzing tool for Wasm compilers.
Developing formally correct runtimes for resource-constrained devices	Developing direct analysis for machine code generated by runtimes to ensure constant-time properties are preserved.	The authors suggest that their input-aware adaptive methodology could be further applied to design deep learning models for intelligent transportation systems and other edge-device scenarios	Implementing targeted redundancy in high-vulnerability code segments to enhance resilience.
Wasm-IO	VIVIENNE	The paper introduces the DAPQ-AD algorithm. It also mentions WebDNN as a related framework for fast DNN execution in browsers	MPSEG and LLFI
First-hand research paper	First-hand research paper	first-hand	First-hand research paper
Finalized	Finalized	Finalized	Finalized

102	103	104	105
WebMO: WebAssembly Memory Bitwise Operation Obfuscation	WasmEye: Language- and Platform-Independent Anomaly Detection for WebAssembly	Detecting WebAssembly Runtime Bugs With Grammar-Guided Program Mutation	Characterizing and Detecting WebAssembly Runtime Bugs
10.1109/COMPSAC61105.2024.00189	https://doi.org/10.1145/3721462.3730957	10.1109/TR.2025.3614352	10.1145/3624743
2024	2025	2025	2023
Republic of Korea	Germany	China	China, UK
Jeongpil Park and Eun-Sun Cho	Arne Vogel, Timothée Glörfeld, Alexander Szekely-Schenker, Thomas Trenner, Rene Emler, and Rüdiger Kapitza	Xinyi Lu, Zhide Zhou, Xiaochen Li, Jifeng Xuan, Hao Wu, He Jiang, and Zhilei Ren	Yixuan Zhang, Shanglong Cao, Haoyu Wang, Zhenpeng Chen, Xiepu Luo, Dongliang Mu, Yun Ma, Gang Huang, and Xuanzhe Liu.
Conference - IEEE COMPSAC	Conference - ACM Middleware Conference	IEEE Transactions on Reliability (Journal)	ACM Transactions on Software Engineering and Methodology (Journal)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Web browsers, virtual currency trading, gaming, and web development	IoT, edge computing, cloud environments, and industrial software	Smart contracts, cloud or edge computing, and the Internet of Things (IoT)	Web applications, desktop applications, mobile applications, IoT, blockchain, serverless computing, and edge computing.
Memory vulnerabilities (e.g., buffer overflows), Cross-Site Scripting (XSS), stack-based buffer overflows, heap metadata corruption, data overwriting, and Use-After-Free	Memory corruption (buffer overflow), control flow manipulation (call_indirect), supply chain attacks, cryptominers, side-channel attacks, and backdoors.	Bugs, crashes (segmentation faults), and memory overflows.	Not Defined
Malicious (attackers stealing personal information, reverse engineering, or illegal intrusion)	Malicious (untrusted third-party module creators, malicious external input, and supply chain compromise)	Not Defined	Not Defined
Memory vulnerabilities (e.g., buffer overflows) because they are among the most dangerous bugs and allow for data theft or large-scale computer infection	Memory corruption and control flow manipulation (indirect calls) because they allow attackers to escape the sandbox context or manipulate program execution.	memory overflow	Not Defined
Linear memory, stack, heap metadata, load/store instructions, i32 pointers, and the V8 engine	Linear memory, function tables (call_indirect), stack pointers, imported functions, and exported functions.	runtime, the backend compiler or interpreter.	Backend compilers (JIT and AoT), interpreters, runtime environment (memory allocation, stack), WebAssembly System Interface (WASI), and auxiliary tools
Web browsers - linear memory shared between JavaScript and Wasm via the V8 engine.	Industrial systems - untrusted modules that can contain hidden logic (backdoors) triggered by malicious external input.	Not Defined	Not Defined
Not Defined	Manipulation of IoT module behavior with external inputs	Not Defined	Not Defined
Not Defined	Malicious code triggering abnormal behavior during dynamic module migration across cloud infrastructures	Not Defined	Not Defined
Not Defined	Exploitation of backdoors in third-party industrial modules	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Virtual currency trading and Gaming: unauthorized reverse engineering and stealing of proprietary data from Linear Memory.	Web browsers - detection of drive-by cryptomining attacks	Not Defined	Not Defined
WebMO framework, using Bitwise Memory Operation (BMO) obfuscation (Memory Splitting, Indirect Addressing, and Offset-based Addressing)	WasmEye framework, utilizing ensemble learning (OSVM, ISO, LOF, and EE) directly embedded into Wasm modules to detect anomalies.	The paper proposes WAGEN, a novel runtime fuzzing approach that combines grammar-guided program mutation with coverage-guided mutation rule selection and differential testing to detect runtime bugs.	A pattern-based bug detection framework based on a taxonomy of 31 bug categories distilled from 311 real-world bugs.
Data obfuscation to protect copyrights and hide vulnerabilities in web-based applications.	On-device anomaly detection for industrial software and IoT to detect unusual module behavior during production execution.	Not Defined	Not Defined
Lack of research into Wasm defenses compared to JavaScript; absence of standard protections like ASLR, unmapped pages, or stack canaries in Wasm.	Current solutions are limited to specific attacks (like cryptominers), require source code access, or necessitate modifying the runtime.	Traditional mutation techniques (like bit-flipping) are inadequate for Wasm's strict binary format, and there is a challenge in generating diverse test programs that explore deep logic and edge cases in runtimes	There is a general lack of understanding of runtime-specific bugs, their root causes, fix patterns, and how to detect them in emerging runtimes.
obfuscation for protection	Ensemble machine learning, tracing of internal function calls, memory access pattern analysis, and bag-of-words vector representation.	Fuzz testing (generation-based and mutation-based) and differential testing.	Static vulnerability scanners (Wasmal), symbolic execution (WANA), and pattern-based bug detection.
Using BMO patterns to increase binary code complexity and prevent reverse engineering	Direct instrumentation of modules with tracing and ML inference logic.	Grammar-guided mutation to generate valid, complex program variants for testing.	Not Defined
Manipulating and merging memory bits via splitting data across contiguous or non-contiguous space to defeat overflow and Use-After-Free attacks.	Using the multi-memory feature to store security logs in a separate memory index that is inaccessible to the main program.	Not Defined	Fixes for memory allocation, leaks, release, and growth
Not Defined	Not Defined	Fuzzing the backend compiler implementations within runtimes.	Fixing compilation rules, register allocation, and data operation logic in backend compilers.
Storing fragmented data in non-contiguous spaces to further strengthen obfuscation.	Performing forensic analysis offline by replaying inputs that caused detected anomalies.	Incorporating reinforcement learning for seed generation, designing advanced mutation rules like equivalence modulo inputs (EMI), and using large language models to automate mutation rule generation.	Future work includes automated testing of runtimes and extending the detection framework with fuzzing and differential testing.
WebMO	WasmEye	WAGEN	A pattern-based bug detector (the paper's tool), TWINE (trusted runtime), and Wasmachine (secure OS for IoT)
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

106	107	108	109
Robust Stack Smashing Protection for WebAssembly	Secure and Flexible WebAssembly Deployment Infrastructure for Automotive Cyber-Physical Systems	WASMixer: Binary Obfuscation for WebAssembly	Iris-Wasm: Robust and Modular Verification of WebAssembly Programs
10.1109/FNW63303.2024.11028722	10.1109/IOLT865288.2025.11116815	10.1007/978-3-031-70896-1_5	https://doi.org/10.1145/3591265
2024	2025	2024	2023
France	Italy	China	UK, Denmark
Quentin Michaud, Yohan Pipereau, Olivier Levillain, and Dhouha Ayed	Tancredi Orlando, Michele Arena, Luca D'Agati, Giovanni Merlino, and Francesco Longo.	Shanglong Cao, Ningyu He, Yao Guo, and Haoyu Wang	Xiaojia Rao, Aina Linn Georges, Maxime Legoupil, Conrad Watt, Jean Pichon-Pharabod, Philippa Gardner, Lars Birkedal
IEEE Future Networks World Forum(Conference)	Conference; IEEE 31st International Symposium on On-Line Testing and Robust System Design	ESORICS 2024 (Conference)	Conference; PLDI (Proceedings of the ACM on Programming Languages)
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Browsers (e.g., 3D workloads), standalone runtimes, cloud computing, and 5G/6G service orchestration for edge and far-edge environments	Automotive Cyber-Physical Systems (CPS), software-defined vehicles (SDVs), ADAS, V2X communication, navigation, and infotainment.	Computation-intensive jobs, browsers (3D engines, multimedia), mobile apps, and serverless computing	Web browsers (running C/C++ applications), cloud sandboxing platforms, and library isolation tools (RLBox).
Buffer overflows, stack smashing, memory corruption, and heap data corruption via stack-based overflows	Buffer overflows within linear memory and vulnerabilities in connected vehicle attack surfaces like V2X, CAN, and Ethernet	Reverse engineering, exploitation of classic vulnerabilities exposed in Wasm binaries, and cryptomining malware	Interaction with unknown, untrusted, or adversarial code from other modules. The paper also notes that classical attacks against memory-unsafe languages can target Wasm programs.
Malicious	Malicious (compromised third-party applications or unauthorized software deployment in a connected vehicle environment)	Malicious	Malicious
Memory corruption attacks (e.g., buffer overflows) because Wasm currently offers weaker protection mechanisms (like the historical absence of SSP) compared to native binaries.	Buffer overflow	Buffer overflow	untrusted code breaking module isolation
Linear memory (where stack, heap, and data zones reside), specifically when it lacks permission controls (making everything writable).	Linear memory and the host function interfaces if they do not properly control hardware access.	Binary structure (readable names, plain-text string literals), the control flow (logic), and linear memory.	Modules (specifically the exploitation of weak encapsulation) and linear memory (susceptible to classical memory attacks)
5G/6G and edge environments are specifically noted as vulnerable to these buffer overflow exploits due to reliance on Wasm for lightweight orchestration.	In automotive CPS, electronic control units (ECUs) and microcontroller units (MCUs) are vulnerable if applications are not properly isolated.	Not Defined	Modules in edge/sandboxing platforms can be exploited if they interact with malicious modules.
Not Defined	Buffer overflows and the deployment of unauthorized/compromised software	Not Defined	Not Defined
Buffer overflows targeting memory corruption	Not Defined	Not Defined	Malicious code interaction in edge platforms like Compute@Edge
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
The paper evaluates and hardens Stack Smashing Protection (SSP) by improving secure storage for the canary reference value and handling random generator failures.	A Kubernetes-based orchestration architecture that utilizes Wasm sandboxing, Zero Trust (ZT) principles, and Reproducible Builds.	WASMixer, a general-purpose binary obfuscator that uses data-level (name/memory) and code-level (control flow flattening, alias disruption, opaque predicates) obfuscation.	Formal verification using Iris-Wasm (a mechanized higher-order separation logic) and logical relations to prove functional correctness and robust safety.
Hardened SSP implementation for standalone runtimes.	Wasm sandboxing (isolation), capability-based permissions, remote attestation, and secure boot	For cryptomining evasion, using name and memory obfuscation together allows malware to fully escape VirusTotal detectors.	Verification of module boundaries and encapsulation properties to enforce strong isolation.
Existing SSP implementations have weaknesses: canaries can be predictable if the runtime fails to provide randomness, and the reference value is stored in writable linear memory where it can be overwritten.	Wasm lacks native system calls and hardware abstraction on bare-metal platforms, and WASI has limited support for embedded systems.	Wasm binaries are weakly protected because they can be easily converted to readable text (wat), and no Wasm-specific obfuscator existed to handle its structured control flow.	Verification of large-scale, real-world programs.
SSP (canaries) and the CookieCrumble tool (adapted for Wasm) to assess implementation robustness.	Trusted software attestations, cryptographic signatures, and policy-based trust evaluation.	virus detectors (VirusTotal), symbolic executors (marcicos, WANA), concolic executors (wasp), and static analysis tools (Binaryn).	Not Defined
Standardizing SSP as a default protection	Reproducible builds and functional build systems (Nix) for supply chain integrity.	Binary-level obfuscation to resist human and automated analysis.	Not Defined
Moving the canary reference value to VM-managed global variables (safe memory) and aborting the program if randomness fails.	Sandboxed execution to isolate application memory from the host and other applications	On-demand runtime memory decryption to protect string literals and data in the memory area.	Verification of isolation properties at module boundaries
Implementing robust SSP prologue/epilogue generation in the LLVM compiler.	Not Defined	Not Defined	Not Defined
It argues that SSP should become a default in all WebAssembly binaries	Enhancing the framework with secure attestation, runtime verification, and formal policy enforcement.	Integrating random control flow and Mixed Boolean-Arithmetic (MBA) to further resist symbolic execution.	Extending Iris-Wasm with automation techniques and using it to validate extensions like MSWasm
Not Defined	CLOVER-LEAF project (reference architecture) and a Kubernetes-based orchestration framework	WASMixer	Iris-Wasm
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

110	111	112	113
WASMOD: Detecting vulnerabilities in Wasm smart contracts	Port Contention Goes Portable: Port Contention Side Channels in Web Browsers	Silverline: Lightweight Virtualization and Orchestration of Distributed Systems	MADVEX: Instrumentation-Based Adversarial Attacks on Machine Learning Malware Detection
10.1049/bic2.12029	10.1145/3488932.3517411	10.1109/RTAS65571.2025.00042	https://doi.org/10.1007/978-3-031-35504-2_4
2023	2022	2025	2023
China	France, Israel	USA, Portugal, Germany	Germany
Jianfei Zhou, Ting Chen	Thomas Rokicki, Cl��mentine Maurice, Marina Bobirnik, Yossi Oren	Arjun Ramesh, Tianshu Huang, Emily Ruppel, Dakshina Dasari, Behnaz Pourmohseni, Fedor Smirnov, Marco Gianti, Paolo Pazzaglia, Charles Shelton, Nuno Pereira, Arne Hamann, Dirk Ziegenbein, Anthony Rowe	Nils Loose, Felix M��chtle, Claudius Pott, Volodymyr Bezmernyi, Thomas Eisenbarth
Journal; IET Blockchain	ASIA CCS '22 (ACM Asia Conference on Computer and Communications Security)	Conference; RTAS (IEEE Real-Time and Embedded Technology and Applications Symposium)	Conference; DIMVA
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Blockchain (EOSIO smart contracts)	Web Browsers (Mozilla Firefox, Google Chrome)	Industrial Automation, Automotive Systems, Edge Computing, and IoT	Web Applications and browser-based cryptojacking
Integer overflow and stack overflow attacks	Microarchitectural side-channel attacks (specifically port contention), covert channels, ASLR derandomization, and transient execution attacks.	Remote code injection and potential crashes/leaks from untrusted modules	Adversarial attacks on machine learning malware detectors and cryptojacking
Malicious	Malicious (attackers running code via malicious websites or ads)	Malicious (specifically "untrusted distributed Wasm modules")	Malicious
Integer overflow, due to its ability to cause massive financial losses (e.g., \$0.85 million loss in one case)	Not Defined	Not Defined	Cryptojacking, as it drains victim resources for profit without consent
Linear memory (where function stack frames are simulated) and the VM interpreter (lacking default overflow checks)	CPU execution ports (P1, P5), SMT/Hyper-Threading, and SharedArrayBuffer (used as a high-resolution timer)	Host Runtime (interfacing with untrusted modules) and Channel API	Code section of Wasm binaries and floating point constants (exploited for gadgets)
Linear memory in smart contracts is vulnerable to overflows as Wasm lacks stack canaries	Execution ports shared between logical cores on the same physical core	Untrusted modules accessing industrial Datalayers (WAXI) or automotive Vehicle Signal Specifications (VSS)	Manipulation of the code section in cryptojacking binaries to bypass CNN-based image classification
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	cross-VM covert channels	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Propagation of source-level memory-unsafe vulnerabilities and lack of VM-layer defenses	Not Defined	Not Defined	Not Defined
Not Defined	Cross-browser covert channels	Not Defined	Evasion of the Mnos cryptojacking detection system
WASMOD, which combines bytecode instrumentation, run-time validation, and grey-box fuzzing	Creating port contention using specific Wasm instructions (e.g., i64.rem_u) and measuring timing via SharedArrayBuffer-based auxiliary clocks.	Silverline framework, providing spatial sandboxing (via WAMR) and temporal isolation (using OS features like cgroups and SCHED_DEADLINE)	MADVEX, which uses binary instrumentation to insert semantic-preserving gadgets containing adversarial payloads
Run-time validation of operands and results for integer operations	Not Defined	Split data and control plane orchestration and sidcar modules for autonomous, low-latency failover.	defensive distillation and adversarial training, though notes they can be bypassed by thoughtful attackers
Difficulty in fuzzing functions that require specific transaction signatures not present in bytecode	Security implications of Wasm compilation on microarchitecture are not fully evaluated	Dynamic network topology capture from controllers	Improving the robustness and security of ML-based malware classifiers
Grey-box fuzzing with test oracles and stain tracking to recover operand types	PC-detector, a framework to dynamically detect Wasm instructions that cause port contention.	Not Defined	CNN-based Mnos (targeted for evasion) and shadow/substitute models
Bytecode instrumentation for recording stack information	Not Defined	Sandboxing and control-flow integrity by design	Designing size-efficient (SE) and optimization-resistant (OR) gadgets
Extracting stack pointer and frame size at runtime to detect overwrites	Not Defined	In-process memory sandboxing	Not Defined
Not Defined	JIT compilation makes code system-dependent	Not Defined	gadgets that can bypass AOT compiler optimizations
Combining fuzzing with symbolic execution and pre-executing contracts to reduce false positives	Creating a pipeline benchmark for Wasm and studying more generic hardware contention side channels	Improving network topology handling via CNC protocols	Ongoing efforts to improve ML robustness against evasion techniques
WASMOD	PC-detector (Selenium-based framework)	Silverline (virtualization and orchestration framework)	MADVEX
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

114	115	116	117
Static Stack-Preserving Intra-Procedural Slicing of WebAssembly Binaries	Potential of WebAssembly for Embedded Systems	WasmRef-Isabelle: A Verified Monadic Interpreter and Industrial Fuzzing Oracle for WebAssembly.	Formal Verification Platform as a Service: WebAssembly Vulnerability Detection Application
https://doi.org/10.1145/3510003.3510070	10.1109/MECO52332.2021.9460215	https://doi.org/10.1145/3591224	10.32604/csee.2023.027680
2022	2021	2023	2023
Belgium, USA	Germany, Switzerland	UK, Netherlands, Germany	China, France
Quentin Stévenart, David W. Binkley, Coen De Roover	Stefan Wallentowitz, Bastian Kersting, Dan Mihai Dumitriu	Conrad Watt, Maja Trela, Peter Lammich, Florian Märkl	LiangJun Deng, Hang Lei, Zheng Yang, WeiZhong Qian, et al.
44th International Conference on Software Engineering (ICSE '22) - Conference	10th Mediterranean Conference on Embedded Computing (MECO) - Conference	PLDI (Proceedings of the ACM on Programming Languages) - Conference	Computer Systems Science & Engineering - Journal
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Web applications (game engines, natural language processing, media players), desktop applications, smart contracts, IoT backends, and embedded software.	Embedded systems, IoT, deeply embedded microcontrollers, smart cards, vision-based sensors, and edge processing.	Web browsers, blockchain (Wasmi runtime), and software testing/industrial fuzzing infrastructure	Blockchain (Ethereum, EOS, Dfinity, Hyperledger Fabric), smart contracts, and transportation systems (Railway-Port-Aviation)
Not Defined	Not Defined	Runtime bugs/crashes identified through fuzzing (e.g., a specific crash bug found in Wasmtime)	Division by zero (demonstrated in a test case)
Not Defined	Malicious	The context is automated test generation (fuzzing) to find flaws in runtimes	Malicious
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Runtimes (interpreters/engines), specifically inefficient handling of control flow, function frames, and memory representation (lists vs. arrays)	Smart contracts (logic flaws) and redundant housekeeping/runtime codes that lead to path explosion during verification.
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Logic vulnerabilities in Hyperledger Fabric chaincode (e.g., WriteOrder contract)
Not Defined	Not Defined	Not Defined	Not Defined
Static, intra-procedural, backward slicing of WebAssembly binaries.	Sandboxing and strong isolation provided by application virtual machines	A verified monadic interpreter (WasmRef-Isabelle) proven correct using Isabelle/HOL refinement proofs, used as a fuzzing oracle	Formal verification (Symbolic execution) combined with optimized translation to prune redundant code using an external API table.
Not Defined	Not Defined	Not Defined	Not Defined
Precise alias analysis for WebAssembly memory and inter-procedural slicing	Standardized APIs for embedded hardware (like image sensors/ISPs) and robust dependency management for large-scale IoT fleets.	Automation for proofs involving monadic state aliasing and performance optimizations for verified runtimes	Development of complex smart contract check models for formal verification
Binary slicing used as a building block for reverse engineering and vulnerability detection.	Not Defined	Differential fuzz testing using a verified oracle to identify semantic discrepancies in industry runtimes.	Symbolic execution using the Microsoft Z3 Prover
A reconstruction phase in the slicing algorithm that inserts synthetic instructions (i32.const, drop) to ensure the slice remains a valid, executable program that passes the Wasm validator.	Not Defined	Not Defined	Code size reduction (pruning housekeeping code) to prevent "path explosion" and memory exhaustion during symbolic execution.
Not Defined	Careful implementation of AOT/JIT methods on devices without virtual memory	Mechanized formal semantics to guarantee the correctness of memory access and stack transitions	Not Defined
Not Defined	Not Defined	fuzzing often targets JIT compilers	Not Defined
Overcoming current implementation limitations related to the br_table instruction and disconnected control-flow graph sections.	standardization of embedded APIs (WASI-compatible) and development of management tools for heterogeneous systems.	Supporting Wasm 2.0 features (Reference Types, Bulk Memory), Threads, and Garbage-collected types	Enhancing vulnerability detection capabilities and check models.
Wassail	Not Defined	WasmRef-Isabelle, WasmCert-Isabelle	FVPS (Formal Verification Platform as a Service)
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

118	119	120	121
WaVe: a verifiably secure WebAssembly sandboxing runtime.	SecWasm: Information Flow Control for WebAssembly	Swivel: Hardening WebAssembly against Spectre	When Does Wasm Malware Detection Fail? A Systematic Analysis of Their Robustness to Evasion.
10.1109/SP46215.2023.00114	https://doi.org/10.1007/978-3-031-22308-2_5	https://www.usenix.org/conference/usenixsecurity21/presentation/narayan	Not Defined
2023	2022	2021	2025
USA	Sweden and Austria	USA	Republic of Korea and USA
Evan Johnson, Evan Laufer, Zijie Zhao, Dan Gohman, Shrawan Narayan, Stefan Savage, Deian Stefan, Fraser Brown	Iulia Bastys, Maximilian Algehed, Alexander Sjösten, and Andrei Sabelfeld	Shrawan Narayan et al.	Taeyoung Kim et al.
IEEE Symposium on Security and Privacy (S&P) - Conference	SAS 2022 (Static Analysis Symposium), Conference	30th USENIX Security Symposium, Conference	Not Defined
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Edge clouds (Fastly, Cloudflare), embedded devices, blockchains, browsers, and network proxies (Envoy)	Web applications, Node.js, decentralised cloud computing, smart contracts, and IoT	Edge clouds, Function-as-a-Service (FaaS) platforms, browser library sandboxing, serverless computing, IoT, games, Trusted Execution Environments (TEEs), and OS kernels.	Web applications, cryptojacking, malicious advertisements, keyloggers, malware, and smart contracts.
Sandbox escape, memory isolation violations (e.g., integer overflow in bounds checks), filesystem isolation violations (symlink attacks), lago attacks, and TOCTOU (Time-of-check-to-time-of-use) bugs.	Malicious modules leaking sensitive data (e.g., passwords) via URL parameter encoding, implicit flows, and flows via memory. It also addresses memory vulnerabilities that are information-flow unsafe.	Spectre attacks (hijacking control flow via PHT, BTB, or RSB), sandbox breakout, sandbox poisoning, and host poisoning.	Cryptojacking, resource theft, and malware evasion via semantic-preserving structural and code perturbations (obfuscation)
Malicious	Malicious	Malicious	Malicious
Sandbox escapes, because they break the primary isolation contract between the guest and the host.	Not Defined	Spectre attacks are highlighted as severe because they bypass isolation on almost all superscalar CPUs and cannot be fixed entirely in hardware	Malicious web programs like cryptojackers that steal computing resources and use diversification to evade all current detectors.
Runtime systems, specifically WASI implementations, hostcall handlers, and system call wrappers.	Unstructured linear memory, operand stack, and global variables	Control flow predictors (CBP, BTB, RSB), linear memory, and host-sandbox transition points (springboards/trampolines)	Binary sections (Type, Import, Function, Global, etc.), instruction distribution, and control/data-flow paths.
Not Defined	Not Defined	FaaS platforms rely on sandboxing isolated tenants, which is vulnerable to Spectre breaking the isolation boundary	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Sandbox escapes with buggy WASI implementations in runtimes like Wasmer and Wasmtime.	Not Defined	Spectre-based isolation bypass on multi-tenant FaaS and edge clouds	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Cryptojacking
Not Defined	A password meter website scenario where a malicious third-party module leaks data	Not Defined	Cryptojacking malware in browser
Verifiably secure sandboxing runtime using the Prusti verification framework to prove hostcall safety contracts.	SecWasm, a hybrid IFC system combining a mostly static security type system with dynamic security checks on memory loads to enforce termination-insensitive noninterference.	Swivel framework, providing software-only (Swivel-SFI) and hardware-assisted (Swivel-CET) hardening using linear blocks, BTB flushing, separate return stacks, Intel MPK/CET, and register interlocking.	Introduces SWAMPED, a framework to evaluate detector robustness using 22 perturbation methods including instruction substitution, opaque predicate insertion, and data encryption.
Enforcing centralized safety policies for memory, filesystem, and network isolation at the hostcall boundary.	Not Defined	Hardening FaaS platforms against cross-tenant data leakage.	Not Defined
Verified dynamic library loading, support for multi-threaded sandboxes, and atomic path resolution.	Handling non-determinism (NaN values, resource exhaustion), timing side-channels, and handling imported host functions	Spectre-STL, and transient attacks like Meltdown, MDS, and LVI on legacy systems.	The systematic robustness of Wasm malware detection against automated diversification
Differential fuzzing of WASI hostcalls against multiple industrial runtimes.	Static information-flow tracking using a stack of security levels and type stacks to detect illicit information flows.	Not Defined	Instruction frequency analysis (MineSweeper), behavior modeling (MinerRay), CNN-based classification (Minos, WasmGuard, MalConv, AvastNet)
Not Defined	Not Defined	Compiling to linear blocks to restrict speculative control flow	Evaluates binary-level structural and code transformations to harden or evade detectors.
Verification of end-to-end memory isolation (ensuring hostcalls only read/write to the sandbox's linear memory)	Flow-sensitive fine-grained memory labeling	Register interlocking to prevent cache-based leaks during mispredicted paths	Not Defined
verifying the runtime removes the compiler from the trusted computing base	Not Defined	Not Defined	Not Defined
Verifying the loader, improving asynchronicity support, and handling more complex OS semantics.	Implementation performance benchmarks, declassification mechanisms, and addressing side channels from micro-architectural states.	optimizing data-dependent loops, compiler secret tracking, and hardware extensions like tagging predictor state or dedicated interlock instructions.	improving detector robustness against distribution-shifting perturbations and investigating multi-section dependency modifications.
WaVe runtime	SecWasm	Swivel	SWAMPED
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

122	123	124	125
Verified Compilation Using a Concrete Memory Model	Bento: Fine-Grained Memory Isolation for COTS Wasm Binaries	Issues and Their Causes in WebAssembly Applications: An Empirical Study	Put Your Memory in Order: Efficient Domain-based Memory Isolation for WASM Applications
10.4230/LIPIcs.ITP.2024.36	10.1145/3774904.3792439	https://doi.org/10.1145/3661167.3661227	https://doi.org/10.1145/3575915.3575205
2024	2026	2024	2023
USA	Germany	Finlan, China, Germany	China
Andrew Tolmach, Chris Chihak, and Sean Anderson	Oussama Draissi and Lucas Davi	Muhammad Waseem, Teerath Das, Aakash Ahmad, Peng Liang, Tommi Mikkonen	Hanwen Lei, Ziqi Zhang, Shaokun Zhang, Peng Jiang, Zhineng Zhong, Ningyu He, Ding Li, Yao Guo, Xiangqun Chen
ITP 2024 (International Conference on Interactive Theorem Proving), Conference	ACM Web Conference 2026	International conference on evaluation and assessment in software engineering	Conference: ACM SIGSAC
Reviewer 1	Reviewer 1	Reviewer 2	Reviewer 1
WebAssembly virtual machine, and legacy C programming idioms.	Web applications, image processing (Libpng), file management (PDFalto), cryptography, scientific computing, and natural language processing.	runtime environments (Wasmer), specifications/proposals, toolchains/compilers, games, blockchain/smart contracts, and UI/frontend frameworks	Standalone runtimes and browsers. Specific use cases include data compression (gzip), speech synthesizers (eSpeak), face detection, chess games (GNU-Chess), memory databases (WhiteDB), and GNU operating system utilities.
Control-flow attacks (corrupting return addresses), buffer overflows, and data-only attacks.	Data-only attacks, Cross-Site Scripting (XSS), memory corruption (buffer overflows, Use-After-Free), Heartbleed, and cross-region exploits.	authentication issues (certificate integration, cryptographic operations, role-based auth), compliance issues, and general security vulnerabilities that expand an application's attack surface	Modifying static variables through stack overflow, arbitrary memory access, sequential buffer overflow, arbitrary file writes, and HeartBleed vulnerabilities.
Malicious	Malicious (adversaries exploiting memory corruption bugs to escalate to XSS or full web application compromise)	The paper focuses on developer issues and not directly on security. So no adversarial model is defined	Malicious
Control-flow attacks	Data-only attacks that corrupt DOM-bound data to enable XSS sandbox escape.	Security Vulnerabilities due to Wasm's Structure and Execution Model: Wasm's structure and execution model are inherent causes of new security vulnerabilities in applications.	Memory corruption vulnerabilities; they are considered more severe in WebAssembly than in native code because the linear memory architecture allows arbitrary access to the entire memory space without unmapped spaces (gaps) between sections.
Memory model (specifically the co-location of public and private data), and data pointers	Monolithic linear memory (co-locating stack, heap, and globals) and the multi-memory feature if not used for isolation.	Vulnerabilities reside in infrastructure and compatibility (28.16% of reported issues), error management, and memory management (including access failure and exhaustion)	Linear memory (lacks permission differentiation and unmapped gaps) and function tables (vulnerable to hijacking)
Not Defined	Commercial-off-the-shelf (COTS) libraries like Libpng and PDFalto co-locating sensitive metadata with untrusted buffers.	The paper does not directly talk about this but (through references) mentions: Smart contracts suffer from vulnerabilities exploitable by various attacks .IoT environments face challenges in maintaining isomorphic microservice architectures	Not Defined
Not Defined	Not Defined	Mentions security concerns in heterogeneous IoT environments	Not Defined
Not Defined	Not Defined	Not defined	Not Defined
Not Defined	Not Defined	Not defined	Not Defined
Not Defined	Not Defined	Investigates attacks exploiting vulnerabilities in Wasm-based smart contracts	Not Defined
Legacy C code that relies on undefined behaviors (UB) like out-of-bounds writes.	Image conversion: Libpng overflow, file recovery: PDFalto overflow, and cryptography: Heartbleed information leak	Not defined	Investigated HeartBleed in OpenSSL (Server-side/Security), and heap-based buffer overflows in PDFalto and libjpeg (Document/Image processing)
Verified compilation using concrete memory semantics and Public vs. Private (PvP) data protection enforced by tagging or sandboxing.	Bento, a static binary rewriting framework that automatically hardens commercial-off-the-shelf (COTS) WebAssembly modules. Bento focuses on mitigating data-only attacks and cross-region memory corruption	The paper provides a taxonomical classification of 120 types of issues and 278 causes to guide developers in diagnosing and addressing problems (not just security, but also any issues related to software design)	PKUWA, a runtime utilizing the Domain Isolated Linear Memory (DILM) model and Intel MPK hardware to provide function-level memory isolation.
PvP protection for Wasm-like environments.	Fine-grained memory isolation for COTS Wasm binaries.	Suggestions include using external libraries as interim solutions for operational issues and implementing rigorous security protocols (not just security focused)	Not Defined
Current verification does not protect against data-only attacks.	Temporal safety issues (e.g., use-after-free within the same region) and intra-frame isolation for local variables.	Significant gaps include the lack of academic literature on Wasm's operational networking capabilities and a lack of mature toolsets for code implementation and optimization (not just security focused)	A "Memory Model Mismatch" exists between Wasm's linear memory and the page-based memory model of hardware features like MPK
Not Defined	Not Defined	The study itself acts as a detection guide by documenting recurring issue-cause patterns, but does not only focus on security	PKUWA uses the MPK hardware to quickly detect unauthorized memory access based on assigned protection keys.
A verified compiler from a Register Transfer Language (RTL) to a protected machine code (Mach)	Static binary rewriting and function specialization to enforce monomorphic memory accesses	Not defined	Not Defined
Tag-based memory protection to separate private control information from public data.	Mapping logical memory regions to isolated memory instances (software-based MMU)	Identifies the need to address mismanagement of memory resources and garbage collection issues	Domain-based memory isolation (DILM) and the default provision of guardian pages between stack, data, and heap sections.
Not Defined	Not Defined	Notes that the diversity of source languages causes discrepancies and anomalies in compiled Wasm code	Not Defined
Incorporating function pointers, achieving vertical compositionality, and building a complete UB-free Concrete C compiler.	intra-frame isolation and granular static analysis capable of deconstructing stack frames without debug symbols.	Future work includes proposing a taxonomy of solutions, validating findings through industrial surveys, and investigating the priority levels of identified issues	Combining PKUWA with libmpk to virtualize more than 16 protection keys and applying the DILM model to other hardware implementations besides Intel MPK.
Not Defined	Bento	The paper provides a dataset and taxonomies for researchers and practitioners	PKUWA
First-hand research paper	First-hand research paper	This is a review (empirical study) paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

126	127	128	129
WebAssembly Memory Tagging	Efficient Data Exchange between WebAssembly Modules	Waplique: Testing WebAssembly Runtime via Execution Context-Aware Bytecode Mutation	Broken Promise: Differential Analysis of Functional Discrepancies Between WebAssembly and Native Binaries
https://doi.org/10.1145/3733812.3765536	https://doi.org/10.3380/fi.16090341	https://doi.org/10.1145/3650212.3680340	https://doi.org/10.1145/3774904.3792193
2025	2024	2024	2026
USA	Portugal	China	China and United States
Shengdun Wang, Aravind Prakash	Lucas Silva, José Metrôlho, Fernando Ribeiro	Wenxuan Zhao, Ruijing Zeng, Yangfan Zhou	Xiao Wu, Alan Romano, Lijun Huang, Qiwen Yan, Cai Fu, Weihang Wang
Conference: Cloud Computing Security Workshop	Journal: Future Internet	Conference: ACM SIGSOFT International Symposium on Software Testing and Analysis	Conference: The ACM Web Conference
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Progressive Web Apps (PWAs), migrating C/C++ applications (games, DOSBox, emulators, remote desktop), Android applications, and cryptocurrencies (e.g., Ethereum)	Web browsers, cloud computing, edge computing, embedded devices, blockchain, and multiplayer games.	Cloud services	IoT's, smart contracts, cloud computing, bioinformatics (Kalign), image processing (Libtiff), audio processing (Llibndiff), and cryptography (Libtomcrypt)
Buffer overflows, code reuse attacks, control flow hijacking, Cross-Site Scripting (XSS), ransomware, and null pointer dereference.	Deserialization of untrusted data (leading to logic abuse, denial of service, or arbitrary code execution), unauthorized data access, and resource exhaustion.	Not Defined	Buffer overflow, heap smashing, and stack smashing
Malicious	Malicious	Not Defined	Malicious
Memory-safety vulnerabilities (specifically use-after-free and buffer overflows); they represent ~70% of high-severity vulnerabilities in major projects and can lead to cryptocurrency theft or XSS.	Deserialization of untrusted data; it is listed among the top 10 security issues by OWASP and can lead to arbitrary code execution.	Not Defined	Functional discrepancies
The flat (linear) memory model, which lacks page protections and Address Space Layout Randomization (ASLR)	Shared memory regions (conflicting access) and module interfaces (imported/exported functions)	Decoder (parsing), Validator (static safety/type checks), and Executor (AOT compilation and machine code generation)	Standard Libraries (musl-libc, wasilibc vs glibc), runtime environments (Wasmtime, WAVM, WasmEdge), and compilers (Emscripten, WASI SDK)
Memory-unsafe cryptocurrency applications are vulnerable to XSS and subsequent theft.	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Investigated out-of-bounds reads in zlib (Data compression) and null dereference bugs in WASI Libc.	Out-of-bounds reads and resource exhaustion in the context of multiplayer games and inter-module communication.	Not Defined	Not Defined
A software-based memory tagging solution for Wasm (inspired by ARM Memory Tagging Extension) implemented via modified LLVM and WAVM	Karmem, which includes a custom interface description language (KarmemIDL), an optimized ABI, and a code generator to establish standardised, secure data organisation.	Waplique, a fuzzing tool using execution context-aware bytecode mutation and differential testing to generate valid and diverse test cases.	WasmDiff, an automated differential fuzzing framework using AFL++ to identify input-triggered behavioral differences between platforms.
Pre-deployment testing and evaluation using memory tagging to find bugs before release.	Timeouts to prevent resource exhaustion from untrusted modules and memory size limits for guests.	Not Defined	Not Defined
Wasm lacks support for mainstream memory safety detection tools like Address Sanitizer (ASan) and Hardware-assisted Address Sanitizer (HWASan) for various architectures.	The Object-Relational Impedance Mismatch (ORIM) across paradigms and the potential for Use-After-Free (UAF) issues when avoiding data copies.	The "diversity/efficiency dilemma" in generating test cases that are both varied and valid enough to pass Wasm's strict safety checks.	Isolating legitimate functional discrepancies in UB-free (valid) code from expected differences caused by undefined behavior.
Memory tagging (color-coding memory regions to detect mismatched pointer access) and comparisons with host Address Sanitizer.	An integrated testing framework that uses malicious producers to generate malformed data to ensure the system behaves as expected.	Differential testing (comparing behavior deviations in return codes and execution time across multiple runtimes like WasmEdge, Wasmer, Wasmtime, and WAMR)	Coverage-based differential fuzzing that compares final outputs (stdout, files, errors) between native and Wasm binaries.
Extending Wasm ISA with dedicated memory tagging instructions.	Not Defined	Context-aware mutation of Wasm bytecode.	Not Defined
Random tagging of the zero address, stack memory, and heap memory regions during allocation.	Internal runtime bound checks to prevent unauthorized access beyond defined memory zones.	Runtime error mitigation strategies, such as adding patch code to map potentially out-of-bound memory addresses to valid ones.	Not Defined
Not Defined	Not Defined	Investigated functional bugs in the code generation (CodeGen) phase, such as register misuse in WAMR and bad optimizations in Wasmtime.	Identified discrepancies arising from different optimization choices and incorrect binary instruction generation (e.g., sign extension issues)
Implementing global tagging once supported in LLVM and exploring alternative memory tagging designs, such as using a B-tree.	Development of the Language Server Protocol (LSP) for KarmemIDL, improving test coverage for unsafe features, and implementing continuous fuzzing.	Incorporating the v128 data type (SIMD) and developing methods to handle or eliminate infinite loops generated during mutation.	Enhancing WasmDiff to record and compare intermediate states (e.g., return values or memory layout) to detect "silent" discrepancies.
Not Defined	Karmem	Waplique	WasmDiff
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

130	131	132	133
Modular Abstract Definitional Interpreters for WebAssembly	WARD: Efficient Memory Protection for WebAssembly on Tiny Embedded Systems	WASDAM: Effectively Detecting Vulnerabilities in Wasm Smart Contracts Based on the Data Access Model.	Segue & ColorGuard: Optimizing SFI Performance and Scalability on Modern Architectures
10.4230/LIPIcs.ECOOP.2023.5	10.1109/ACCESS.2025.3650447	10.1109/QRS65678.2025.00079	10.1145/3669940.3707249
2023	2026	2025	2025
Germany	Republic of Korea	China	USA, Germany, China
Katharina Brandl, Sebastian Erdweg, Sven Keidel, Nils Hansen	Chaewon Shin, Gowon Han, Suhyeon Song, Jinsun Park, Yoon-Ho Choi, Donghyn Kwon	Chu Chen, Xuan Wang, Pinghong Ren, Bin Yu, Jin Liu	Shravan Narayan, Tal Garfinkel, Evan Johnson, et al.
Conference; 37th European Conference on Object-Oriented Programming	Journal; IEEE Access	Conference; IEEE International Conference on Software Quality	Conference; 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems
Reviewer 1	Reviewer 1	Reviewer 1	Reviewer 1
Computation-intensive JavaScript libraries (e.g., game engines), web browsers, and portable computation.	Tiny embedded systems, web browsers, IoT devices, and resource-constrained platforms.	Blockchain smart contracts (EOSIO, Ethereum 2.0/eWasm)	Web browsers (Firefox), datacenter infrastructure, SaaS applications, databases, and high-scale low-latency edge-compute platforms (e.g., Fastly, Cloudflare)
OS-level routine susceptibility, information-flow violations, tampering with memory accesses, and vulnerabilities resulting from the lack of compiler protection mechanisms like stack canaries.	Memory safety violations (spatial and temporal), buffer overflows (inter-object, intra-region), corruptions, out-of-bounds reads/writes, buffer overreads, use-after-free, and use-after-reallocation.	Fake EOS, Fake Notification, Missing Authorization Verification, Block Information Dependency, Rollback, "Groundhog Day" vulnerabilities, and "FakeTransfer"	Memory safety vulnerabilities (mitigated through library sandboxing), out-of-bounds memory accesses, and Spectre
Not Defined	Malicious	Malicious	Malicious
Not Defined	Memory errors such as buffer overflows within linear memory because they can silently corrupt adjacent data and undermine the integrity of the entire memory space	Rollback vulnerabilities, as they involve complex multi-function logic and strict triggering conditions.	Not Defined
OS-level routines, linear memory, indirect function calls, operand stacks, call frames, global variables, and function tables.	Linear memory (including global variables, stack frames, and heap objects)	Multi-index tables (dynamic external state data) and Wasm bytecode.	SFI (Software-based Fault Isolation) compiler instrumentation, memory allocators (common source of CVEs breaking isolation), and virtual address space exhaustion.
Not Defined	Not Defined	Smart Contracts: Execution state data stored in tables that are modified by transactions or function calls.	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Not Defined	Not Defined	Not Defined	Investigated scaling limits where concurrent Wasm instances exhaust the 47-bit user-space address space due to large guard regions.
Not Defined	Vulnerabilities in stack frames and heap objects within the linear memory.	Not Defined	Not Defined
Not Defined	Not Defined	Security flaws including state dependency (hard to manipulate external states for symbolic execution), cross-function state transfer (difficulty inferring relationships among multiple function calls), and path selection (depth-first search overlooking important paths)	Not Defined
Not Defined	Not Defined	Not Defined	Not Defined
Abstract definitional interpretation to perform static whole-program analysis, specifically for dead code identification, constant propagation, and taint analysis.	WARD, a redzone-based memory protection technique utilizing a randomized linear-to-physical (L2P) address translation mechanism.	WASDAM, a concolic fuzzing approach integrating data access modeling (DAM), dynamic sensitive code tracing, and shortest path optimization.	Segue (leveraging x86-64 segmentation to reduce SFI overhead) and ColorGuard (leveraging MPK memory tagging to improve scaling)
Not Defined	Embedded Systems: Software-based address translation to create large, scalable, and non-mapped "external" redzones without requiring hardware extensions.	Smart Contracts: Tracking load and store instructions to trace sensitive data flow and solve symbolic constraints for branch exploration.	Browsers: Segue reduces the performance tax of SFI for sandboxed libraries like media-decoding. EdgeCloud: ColorGuard stripes memory with MPK colors to pack up to 15x more Wasm instances concurrently in a single address space.
Lack of existing static whole-program analyses for Wasm due to complex language features	Prior approaches either suffer from limited detection coverage or are unsuitable for the strict memory/hardware constraints of tiny embedded systems.	Difficulties in revealing vulnerabilities due to lack of source code scrutiny; limitations of symbolic execution in identifying transaction dependencies and handling fragile branch constraints.	Efficiency gaps (SFI overhead of 20%-30%) limiting adoption in latency-sensitive tasks; scaling limits that force multi-process architectures, which sacrifice throughput.
Context-insensitive dead code analysis, callsite-sensitive constant propagation analysis, and callsite-sensitive taint analysis.	Detecting illegal memory access through failed address translation in a hashed translation table.	Concolic fuzzing using automated test oracles and shortest sensitive path evaluation to find vulnerabilities in contract execution logs.	Not Defined
Reduction of binary size by safely eliminating dead instructions.	Not Defined	Not Defined	Use of Segment Prefix and address-size override prefix to reduce the instruction count for memory sandboxing.
Taint analysis to guarantee memory accesses cannot be user-influenced (tamper-proof)	Randomized L2P translation to isolate memory at the granularity of individual stack frames and heap objects; delayed reuse of freed memory to mitigate use-after-reallocation	Not Defined	Implicit bounds checking using virtual memory guard regions; MPK-based domain isolation; and x86-64 segmentation to store linear memory base addresses.
Not Defined	Not Defined	Identification of sensitive code segments obscured by compilation optimizations that fragment functions and strip semantic context	Identified CVEs arising from code generation flaws in Wasmtime's Cranelift backend and verified the pooling allocator using refinement type checkers.
Refining memory abstractions, tracking additional sinks/sources for taint values, and implementing more precise abstract domains such as intervals or relational domains.	Not Defined	Adding detection rules for other vulnerability types and expanding the variety of vulnerability types in the dataset.	Modifying compilers to dynamically choose between Segue and explicit base addition using a cost function.
Not Defined	WARD	WASDAM	Not Defined
First-hand research paper	First-hand research paper	First-hand research paper	First-hand research paper
Finalized	Finalized	Finalized	Finalized

134	135
A First Look at Code Obfuscation for WebAssembly	WASMDYPA: Effectively Detecting WebAssembly Bugs via Dynamic Program Analysis
10.1145/3507657.3528560	DOI: 10.1109/SANER60148.2024.00037
2022	2024
USA	China
Shrenik Bhansali, Ahmet Aris, Abbas Acar, Harun Oz, A. Selcuk Uluagac	Wenlong Zheng and Baojian Hua
Conference, 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks	IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)
Reviewer 1	Reviewer 2
Mobile computing, web development, mobile agents, IoT device containers, and edge computing	Wasm is used in blockchain, edge computing, clouds, cryptography, smart contracts, embedded devices, IoT, video transcoders, and game engines
Cryptojacking, Wasm-based malware, browser computational resource abuse, ransomware, and remote code execution	Attacks investigated include integer overflows, memory vulnerabilities (buffer overflows, double-free, use-after-free), and heap metadata corruption (unlink exploits)
Malicious	The paper utilizes a malicious input model, where an attacker provides specific data to trigger deep vulnerabilities like integer overflows
Malicious programs with unrestricted access to browser functionality through JavaScript APIs because they can perform remote code execution or acquire all user data	Integer overflow to buffer overflow (IO2BO) is considered highly severe because an undetected arithmetic overflow can lead to a zero-length buffer allocation, making subsequent memory corruption inevitable
JavaScript API, browser functionality, and jump tables	The Wasm type system (which lacks integer overflow checks) and the linear memory model (which allows heap metadata corruption) are identified as key vulnerable components
Not Defined	In multi-tenant cloud scenarios, vulnerabilities in the intra-process isolation/sandboxing could potentially compromise tenant separation
Not Defined	Not defined
Not Defined	Investigates vulnerabilities that could compromise safe binary execution
Not Defined	Not defined
Not Defined	Addresses vulnerabilities like integer overflows and memory bugs in contracts
Exploiting browser computational resources while using obfuscation to evade static analysis tools.	Not defined
Applying and evaluating seven high-level obfuscation techniques (e.g., Virtualization, Flattening) to test their effectiveness against state-of-the-art detectors.	The paper introduces WASMDYPA, a dynamic analysis framework that uses static instrumentation hooks to collect runtime data and security plugins to detect bugs
Cryptojacking: Using obfuscation as a deterrent against reverse engineering and static analysis.	They propose a heuristic approach, pre-condition instrumentation, involving generating specific inputs by instrumenting security predicates as pre-conditions, designed within an extendable framework to apply to other possible bugs.
Feasibility and effectiveness of Wasm-specific obfuscation techniques were previously uninvestigated; lack of analysis methods resistant to obfuscation.	A systematic study of effective bug detection for Wasm programs (as opposed to just VM or compiler testing) was previously missing
MINOS, a cryptojacking detector using grayscale image representations of Wasm binaries and CNN-based classifiers, was evaluated for subversion.	Dirty value propagation (assigning "dirty" values to overflow results to track their sink into memory allocations) and hook-based memory tracking
Obfuscation at the C-level source code to produce distinct and incomprehensible Wasm binaries	Not defined
Not Defined	The tool tracks heap and stack allocations via hooks to detect use-after-free and double-free vulnerabilities
Identified how the two-tier compilation process of browsers interacts with obfuscated code	Not defined
Creating analysis methods that cannot be circumvented by obfuscation; evaluating the effect of stacking multiple obfuscation techniques together.	Suggested future work includes supporting new instructions (SIMD, GC, reference types), investigating Rust-specific bugs in Wasm, and detecting concurrency bugs
MINOS, Tigress (obfuscator), Emscripten	The paper introduces WASMDYPA, an automated dynamic bug detection system
First-hand research paper	First-hand
Finalized	Finalized